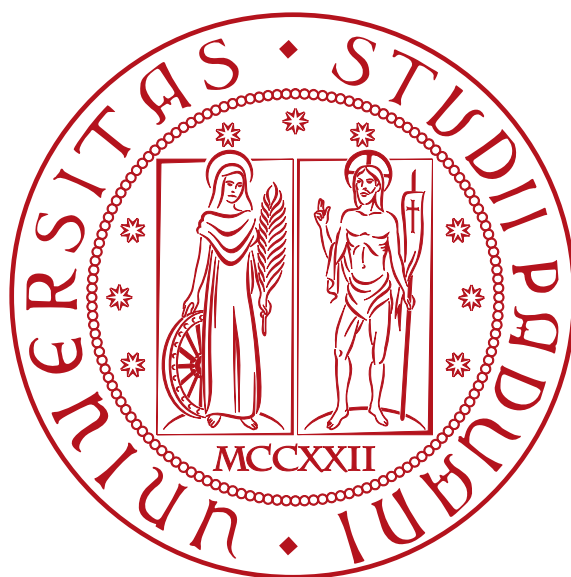


Università degli studi di Padova

DIPARTIMENTO DI MATEMATICA "TULLIO LEVI-CIVITA"

CORSO DI LAUREA IN INFORMATICA



Analisi di sicurezza e integrità crittografica di un sistema di versionamento distribuito

Tesi di laurea triennale

Relatore

Prof. Tullio Vardanega

Laureando

Michele Stevanin

Matricola 2101741

«Chi non conosce la storia è condannato a ripeterla,
ma chi la conosce è condannato a guardare gli altri che la ripetono.»

— Primo Levi

Sommario

Questa relazione descrive il lavoro svolto durante lo stage curricolare presso Zucchetti S.p.A., della durata di trecentoquattro ore, sotto la supervisione del tutor aziendale Gregorio Piccoli e del tutor accademico Prof. Tullio Vardanega.

Lo stage ha riguardato l'analisi della sicurezza di *RVC_G*, un sistema di versionamento distribuito sviluppato internamente da Zucchetti S.p.A. come alternativa a *Git_G*. A differenza dei sistemi tradizionali, *RVC_G* non richiede un server centrale per la verifica di autenticità dei commit: i commit vengono distribuiti come archivi firmati, navigabili direttamente tramite filesystem. L'integrità dei contenuti è garantita attraverso la verifica crittografica degli *hash_G* di ogni commit, con l'obiettivo di permettere a qualsiasi utente di accertare autonomamente l'autenticità della *repository_G* ricevuta, indipendentemente dalla fonte.

Il lavoro ha previsto lo studio delle tecnologie crittografiche alla base del sistema — chiavi *SSH_G Ed25519_G*, firma-digitale e cifratura asimmetrica con *AGE_G* — seguito dalla definizione di un modello di sicurezza formale composto da quattordici requisiti classificati per priorità e tipologia. Il modello introduce una gerarchia di fiducia a tre livelli (amministratore, responsabile di progetto, dipendente), un progetto speciale `_rvc_root` come radice di fiducia verificabile autonomamente tramite chiave master, e cinque livelli di sicurezza configurabili per progetto — dall'accesso anonimo senza firma (livello 0) alla cifratura dei contenuti tramite *AGE_G* (livello 4). Ogni commit è vincolato a una catena crittografica basata su un `cumulativeHash` progressivo che incorpora l'intera storia precedente, permettendo la verifica dell'ordine dei commit indipendentemente da qualsiasi server centrale.

Il lavoro include la simulazione di quattro scenari di attacco con livelli crescenti di accesso dell'attaccante — dall'esterno senza credenziali fino alla compromissione della chiave dell'amministratore — e la successiva implementazione dei miglioramenti nel codice sorgente in linguaggio *CPL_G*. Tra le funzionalità realizzate figurano il verificatore di integrità della catena crittografica, il controllo preventivo delle autorizzazioni per progetto tramite file `allowed_Dipendenti` versionato, la protezione dei file speciali di configurazione

e il meccanismo di redazione-trasparente per la rimozione forense di contenuti dalla storia. Gli stessi scenari di attacco vengono quindi ripetuti sulla versione aggiornata del sistema per documentare l'efficacia dei controlli introdotti.

Organizzazione del testo

- Il primo capitolo** presenta l'azienda ospitante, introduce il progetto *RVC_G* e illustra le motivazioni che mi hanno portato a scegliere questo stage, con gli obiettivi definiti, la pianificazione e l'analisi dei rischi;
- Il secondo capitolo** descrive l'organizzazione del lavoro durante il tirocinio, l'ambiente di sviluppo, gli strumenti utilizzati e l'approccio metodologico seguito;
- Il terzo capitolo** illustra le tecnologie e i concetti teorici alla base del progetto, dalla crittografia-asimmetrica e *SSH_G* fino all'architettura di *RVC_G* e al confronto con *Git_G*;
- Il quarto capitolo** definisce il modello di sicurezza che un sistema di versionamento distribuito moderno dovrebbe soddisfare analizzando i requisiti formali, gerarchia di fiducia, livelli di sicurezza configurabili e meccanismo di Redazione-trasparente — concludendo con l'analisi del divario rispetto alla versione iniziale di *RVC_G*;
- Il quinto capitolo** documenta gli scenari di attacco simulati in ambiente controllato, con livelli di accesso crescenti, distinguendo le vulnerabilità rilevabili solo tramite verifica esplicita da quelle bloccate preventivamente dal motore;
- Il sesto capitolo** descrive i miglioramenti implementati nel codice sorgente di *RVC_G*, tra cui il sistema di configurazione dinamico, la firma *SSH_G* integrata, la verifica dell'integrità della catena e i meccanismi di controllo delle identità autorizzate;
- Il settimo capitolo** ripete i quattro scenari di attacco del quinto capitolo sulla versione aggiornata di *RVC_G*, documentando per ogni tecnica il comportamento dei nuovi controlli preventivi — firma obbligatoria, verifica dell'autorizzazione per progetto, protezione dei file speciali — e confrontando i risultati con quelli della versione iniziale per valutare l'efficacia dei miglioramenti;
- L'ottavo capitolo** riassume i risultati raggiunti, valuta il grado di soddisfacimento degli obiettivi prefissati e propone una riflessione personale sull'esperienza e sugli sviluppi futuri.

Convenzioni tipografiche

Durante la stesura del testo sono state adottate le seguenti convenzioni tipografiche:

- Gli acronimi, le abbreviazioni e i termini di uso non comune vengono definiti nel [glossario](#), situato alla fine del documento (p. 115);
- I termini presenti nel glossario sono indicati con la notazione: *RVC_G*;
- I termini in lingua straniera non di uso comune o appartenenti al gergo tecnico sono evidenziati in *corsivo*;
- I nomi di funzioni, variabili o comandi sono scritti con carattere `monospaziato`;
- I riferimenti bibliografici sono indicati con il numero identificativo della fonte, es. [1];
- I blocchi di codice sorgente sono rappresentati nel seguente modo:

```
proc ReportInfo(FileManifest fm, bool files:=false)
  if fm.tag <> nil
    ? '    tag:', fm.tag
  end
  if fm.prev <> nil
    ? '    prev:', fm.prev
  end
end
```

cpl

Esempio di funzione CPL estratta dal sorgente di *RVC_G*.

Indice

1	Introduzione	1
1.1	L'azienda	1
1.2	Il progetto	2
1.3	Scelta del progetto	2
1.4	Obiettivi dello stage	3
1.5	Pianificazione	4
1.6	Analisi dei rischi	5
2	Descrizione dello stage	7
2.1	Organizzazione del lavoro	7
2.2	Ambiente di sviluppo	8
2.3	Approccio metodologico	9
3	Tecnologie e fondamenti teorici	11
3.1	Crittografia asimmetrica e firma digitale	11
3.2	SSH e ssh-keygen	13
3.3	AGE	14
3.4	Sistemi di controllo versione	15
3.5	RVC: architettura e funzionamento	18
4	Requisiti e modello di sicurezza	21
4.1	Sicurezza strutturale nei sistemi di versionamento distribuito	21
4.2	Requisiti di sicurezza formali	23
4.3	Gerarchia di fiducia	27
4.4	Livelli di sicurezza configurabili	38
4.5	Gestione delle identità e ciclo di vita delle chiavi	44
4.6	Gestione dei branch	47

4.7	Redazione Trasparente	51
4.8	Analisi del divario	56
5	Simulazione scenari di attacco	61
5.1	Ambiente di simulazione e metodo	61
5.2	Scenario 1 — Attaccante esterno senza credenziali	62
5.3	Scenario 2 — Dipendente con chiave SSH valida ma non autorizzato	70
5.4	Scenario 3 — Chiave privata di un dipendente compromessa	75
5.5	Scenario 4 — Chiave operativa dell'amministratore compromessa	80
5.6	Sintesi complessiva	83
6	Miglioramenti implementati	87
6.1	Visione d'insieme	87
6.2	Estensione della struttura CommitValidation	87
6.3	Integrità e ordine verificabile (RS01-RS04)	89
6.4	Autenticità e non ripudio (RS05-RS07)	90
6.5	Gestione delle identità (RS08-RS11)	93
6.6	Sicurezza configurabile (RS12-RS13)	97
6.7	Gestione dei branch (RS14-RS15)	100
6.8	Verificatore di integrità della catena (RS02, RS02, RS06)	102
6.9	Redazione trasparente	103
6.10	Robustezza e fix injection (scenario S5)	106
7	Simulazione scenari di attacco post-implementazione	111
8	Conclusioni	113
	Glossario	115
	Bibliografia	118

Elenco delle Figure

1.1 Logo di Zucchetti S.p.A.	1
3.1 Flusso trasferimento messaggio con uso di chiave pubblica e privata	12
3.2 Flusso firma di un documento con chiave pubblica e privata	12
3.3 Comparazione dimensione tra tipologie di chiavi	13
3.4 Esempio di un file cifrato con AGE	15
3.5 Controllo versioni centralizzato vs distribuito	16
3.6 Struttura del file .sig e la catena degli hash cumulativi	19
3.7 Struttura dei moduli principali di RVC	20

Elenco delle Tabelle

1.1 Obiettivi dello stage.	3
1.2 Pianificazione del periodo di stage.	4
1.3 Analisi preventiva dei rischi.	5
3.1 Confronto tra Git e RVC.	17
4.1 Requisiti di integrità e ordine verificabile.	24
4.2 Requisiti di autenticità e non ripudio.	25
4.3 Requisiti di gestione delle identità.	26
4.4 Requisiti di sicurezza configurabile.	26
4.5 Requisiti di gestione dei branch.	27
4.6 Confronto tra i livelli di sicurezza configurabili.	41
4.7 Autorizzazioni per operazioni sui branch.	51
4.8 Analisi del divario — integrità e ordine verificabile.	57
4.9 Analisi del divario — autenticità e non ripudio.	57
4.10 Analisi del divario — gestione delle identità.	58
4.11 Analisi del divario — sicurezza configurabile.	59
4.12 Analisi del divario — gestione dei branch e incidenti.	59

4.13	Sintesi dell'analisi del divario.	60
5.1	Struttura della repository di test allo stato iniziale.	61
5.2	Sintesi dei risultati — Scenario 1.	70
5.3	Sintesi dei risultati — Scenario 2.	74
5.4	Sintesi dei risultati — Scenario 3.	80
5.5	Sintesi dei risultati — Scenario 4.	83
5.6	Sintesi complessiva degli scenari di attacco.	83
5.7	Corrispondenza tra vulnerabilità osservate e requisiti del modello.	86
6.1	Requisiti di integrità	89
6.2	Requisiti di autenticità	90
6.3	Requisiti di gestione identità	93
6.4	Requisiti di sicurezza configurabile	97
6.5	Livelli di sicurezza	97
6.6	Requisiti di gestione branch	100
6.7	Pattern di crash e correzioni	108

Elenco dei Codici Sorgente

0.1	Esempio di funzione CPL estratta dal sorgente di <u>RVC</u> _G	vii
4.2	Esempio di firma <u>SSH</u> _G	36
6.3	Campi di CommitValidation	88
6.4	Formato nome file ZIP	89
6.5	Generazione ID content-dependent	90
6.6	Sintassi rvc init	91
6.7	SaveAndSignCommitSignature	92
6.8	Gestione configurazione locale	92
6.9	Formato allowed_Dipendenti	93
6.10	CheckValidity — controllo autore e file speciali	94
6.11	Correzione ricerca file speciali	95
6.12	Commit su _rvc_root con chiave master	96
6.13	Contenuto di .rvc_policy per livello 4	97
6.14	Cifratura age per livello 4	99

6.15	Controllo branchStatus in CheckValidity	101
6.16	VerifyIntegrity — verifica per commit	103
6.17	Redact — sostituzione e ricalcolo catena	104
6.18	Funzioni di validazione input	106
6.19	Vulnerabilità CWE-78 su author	107
6.20	Correzione CWE-78 — doppia difesa	107
6.21	Guard injection su path di configurazione	108
6.22	Guard parametri obbligatori — NewProject	109

Capitolo 1

Introduzione

In questo capitolo viene descritta l'azienda ospitante, introdotto il progetto e illustrate le motivazioni che hanno portato alla scelta di questo stage.

1.1 L'azienda

Zucchetti S.p.A. è un'azienda italiana con sede principale a Lodi che produce soluzioni software, hardware e servizi per aziende, professionisti e associazioni di categoria. Le origini risalgono al 1977, quando lo studio del commercialista Domenico «Mino» Zucchetti realizzò il primo software in Italia per l'elaborazione automatica delle dichiarazioni dei redditi. L'anno successivo, nel 1978, fu fondata formalmente l'azienda.

Oggi il gruppo conta più di 8.000 dipendenti, di cui 2.000 dedicati alla Ricerca e Sviluppo, e serve oltre 700.000 clienti tramite sedi distribuite in tutta Italia e più di 1.650 partner. A livello internazionale è presente con proprie società in diversi paesi europei e oltreoceano, con una rete di oltre 350 partner in 50 paesi. Zucchetti si posiziona oggi come la prima *software house* in Italia per fatturato.

Lo stage è stato svolto presso la sede di Padova e di Noventa Padovana (PD), sotto la supervisione del tutor aziendale Gregorio Piccoli.



Logo di Zucchetti S.p.A.

1.2 Il progetto

Il progetto ha riguardato l'analisi della sicurezza di *RVC_G*, un sistema di versionamento distribuito sviluppato internamente da Zucchetti S.p.A. come alternativa a *Git_G*. A differenza dei sistemi tradizionali, *RVC_G* non richiede un server centrale: i *commit* vengono distribuiti come archivi firmati, navigabili direttamente tramite filesystem. L'integrità dei contenuti è garantita attraverso la verifica crittografica degli *hash_G* di ogni *commit*, con l'obiettivo di permettere a qualsiasi utente di accertare autonomamente l'autenticità della *repository* ricevuta, indipendentemente dalla fonte di distribuzione.

L'autenticazione e la firma dei *commit* avvengono tramite chiavi *SSH_G*, rendendo ogni modifica crittograficamente attribuibile al suo autore. Questo approccio distingue *RVC_G* non solo per l'architettura distribuita, ma anche per le garanzie di autenticità che offre rispetto ai sistemi di versionamento convenzionali.

La versione di *RVC_G* fornita per lo stage è una versione di sviluppo deliberatamente priva di alcuni meccanismi di sicurezza, con l'obiettivo di permettere uno studio autonomo delle vulnerabilità e la progettazione di soluzioni originali. Questo approccio ha consentito di affrontare il problema della sicurezza senza vincoli architetturali predefiniti, producendo analisi e implementazioni indipendenti.

1.3 Scelta del progetto

Ho scelto questo progetto per l'interesse verso la sicurezza informatica e la crittografia applicata, temi che avevo già incontrato durante il percorso universitario ma che non avevo mai avuto l'occasione di approfondire in un contesto reale. L'analisi di un sistema in uso aziendale, con l'obiettivo di individuare vulnerabilità concrete e proporre miglioramenti, rappresenta un'opportunità difficilmente replicabile in ambito accademico.

La natura del lavoro — che combina studio teorico delle tecnologie crittografiche, progettazione di modelli di sicurezza e simulazione pratica di scenari di attacco — mi ha convinto che fosse il progetto più adatto per concludere il percorso triennale con un contributo originale e tangibile.

1.4 Obiettivi dello stage

Gli obiettivi dello stage sono stati definiti in accordo con il tutor aziendale e classificati secondo tre livelli di priorità:

- **Obbligatorî** (O): obiettivi il cui soddisfacimento è vincolante per la riuscita del progetto;
- **Desiderabili** (D): obiettivi non strettamente necessari ma dal riconoscibile valore aggiunto;
- **Facoltativi** (F): obiettivi che rappresentano un ulteriore contributo non competitivo.

Codice	Descrizione
O01	Studio delle tecnologie <i>SSH_G</i> , <i>AGE_G</i> e <i>RVC_G</i>
O02	Analisi del sistema <i>RVC_G</i> : architettura, formato dei commit e individuazione delle vulnerabilità
O03	Definizione di un modello di sicurezza formale per sistemi di versionamento distribuito, con requisiti classificati per priorità e confronto con lo stato iniziale di <i>RVC_G</i>
O04	Valutazione del grado di soddisfacimento del modello di sicurezza proposto a seguito degli interventi migliorativi implementati
O05	Produzione della documentazione tecnica e della relazione finale
D01	Progettazione della gerarchia di fiducia e dei livelli di sicurezza configurabili per <i>repository_G</i> distribuite
D02	Simulazione di scenari di attacco con diversi livelli di accesso e analisi delle vulnerabilità individuate
D03	Implementazione dei miglioramenti prioritari nel codice sorgente <i>CPL_G</i> di <i>RVC_G</i>
F01	Progettazione del meccanismo di Redazione-trasparente per la gestione di contenuto illegale o sensibile nella <i>repository_G</i>
F02	Analisi delle possibilità di adozione di una struttura monorepo o polirepo in <i>RVC_G</i>

Obiettivi dello stage.

1.5 Pianificazione

Il lavoro è stato organizzato su otto settimane per un totale di 304 ore, suddivise tra studio delle tecnologie, analisi della sicurezza, sviluppo e documentazione.

Settimana	Ore	Attività
1	32	Studio di <u>SSH_G</u> , crittografia-asimmetrica, firma-digitale e <u>AGE_G</u>
2	40	Studio di <u>RVC_G</u> : architettura, formato dei commit
3	40	Simulazione scenari di attacco senza credenziali e con chiave compromessa
4	40	Stesura documentazione e analisi dei possibili miglioramenti
5	40	Implementazione dei miglioramenti prioritari
6	32	Testing e validazione, implementazione di miglioramenti secondari
7	40	Testing, simulazione scenari di attacco post-miglioramenti, stesura documentazione tecnica
8	40	Completamento e revisione della relazione finale

Pianificazione del periodo di stage.

1.6 Analisi dei rischi

Prima dell'avvio del progetto è stata condotta un'analisi preventiva dei rischi, al fine di individuare le possibili criticità e predisporre le opportune contromisure.

Descrizione	Contromisura	Probabilità Impatto
Codice sorgente di <i>RVC_G</i> non disponibile nella fase iniziale, con impossibilità di analisi <i>white-box_G</i>	Analisi <i>black-box_G</i> tramite osservazione del comportamento esterno e dei file prodotti	Alta Medio
Linguaggio <i>CPL_G</i> proprietario senza documentazione pubblica, con curva di apprendimento elevata	Studio della documentazione interna fornita dall'azienda e confronto diretto col tutor	Alta Medio
Comportamenti silenziosi del sistema in caso di errore, che rendono difficile il debug	Aggiunta sistematica di istruzioni di debug nel codice sorgente durante l'analisi	Media Alto
Vulnerabilità individuate già risolte nella versione interna, rendendo il lavoro ridondante	Verifica periodica col tutor aziendale sull'allineamento tra la versione di test e quella interna	Bassa Medio

Analisi preventiva dei rischi.

Capitolo 2

Descrizione dello stage

In questo capitolo viene descritta l'organizzazione del lavoro durante il tirocinio, l'ambiente di sviluppo utilizzato, gli strumenti adottati e l'approccio metodologico seguito.

2.1 Organizzazione del lavoro

Lo stage si è svolto in presenza presso le sedi di Zucchetti S.p.A., con rare eccezioni in modalità *smart working* nei periodi in cui l'azienda era impegnata in un trasferimento di sede. La prima settimana si è tenuta presso gli uffici di via Giovanni Cittadella a Padova; le settimane successive presso la sede principale di Noventa Padovana (PD), dove ho trascorso la maggior parte del tirocinio.

L'orario di lavoro era strutturato su due fasce giornaliere: dalle 9:00 alle 13:00 e dalle 14:00 alle 18:00, per un totale di otto ore al giorno. Questa organizzazione ha permesso di alternare sessioni di studio teorico al mattino con attività pratiche nel pomeriggio, sfruttando la pausa di mezzogiorno per consolidare i concetti appresi.

2.1.1 Rapporto con i tutor

Il confronto con i tutor ha avuto modalità diverse a seconda del ruolo.

Il **tutor aziendale**, Gregorio Piccoli, era disponibile quasi quotidianamente per chiarimenti, revisioni del lavoro svolto e indicazioni sui passi successivi. Le riunioni non seguivano una cadenza fissa ma avvenivano su richiesta, ogni volta che si presentava un problema tecnico rilevante o si raggiungeva un risultato degno di discussione. Questo approccio ha favorito un dialogo continuo e ha permesso di adattare il percorso in modo flessibile all'avanzamento del lavoro.

Il **tutor accademico**, il Professor Prof. Tullio Vardanega, è stato invece coinvolto con cadenza settimanale tramite scambio di email. Le comunicazioni riguardavano principalmente l'avanzamento dello stage rispetto alle attese previste e la verifica dell'allineamento con gli obiettivi didattici dello stage.

2.2 Ambiente di sviluppo

Per tutta la durata dello stage ho utilizzato un portatile fornito dall'azienda con sistema operativo Windows. La scelta degli strumenti da installare è stata lasciata alla mia discrezione, senza vincoli imposti dall'azienda, il che ha permesso di configurare un ambiente di lavoro ottimale per le esigenze del progetto.

L'accesso ai sistemi aziendali includeva anche i modelli linguistici (*LLM*) disponibili localmente nell'infrastruttura di Zucchetti, utilizzati come supporto durante le fasi di studio e sviluppo.

2.2.1 Strumenti utilizzati

Gli strumenti principali adottati durante lo stage sono stati:

Visual Studio Code come editor di testo principale, utilizzato sia per la scrittura e modifica del codice sorgente *CPL_G* di *RVC_G*, sia per la stesura della documentazione in Typst. L'editor è stato configurato con estensioni per la sintassi *CPL_G* e per la compilazione live dei documenti Typst.

GitHub per il versionamento della tesi e dei file di lavoro personali. Il *repository_G* della relazione finale è ospitato su GitHub con pubblicazione automatica del PDF tramite *GitHub Actions* e *GitHub Pages*.

Typst come sistema di composizione tipografica per la stesura della relazione finale. Typst è un'alternativa moderna a LaTeX che permette di produrre documenti PDF di qualità professionale con una sintassi più accessibile.

Terminale Windows (cmd e PowerShell) per l'esecuzione dei comandi *RVC_G*, la gestione dei file e le operazioni di debug. La maggior parte delle operazioni con *RVC_G* avviene tramite interfaccia a riga di comando.

PuTTY come client *SSH_G* per Windows, utilizzato per la gestione delle chiavi *SSH_G* tramite il componente `puttygen`.

Ssh-keygen (OpenSSH per Windows) per la generazione delle chiavi *Ed25519_G*, la firma crittografica dei file e la verifica delle firme *SSH_G*. Questo strumento è centrale nel meccanismo di sicurezza implementato in *RVC_G*.

AGE_G come strumento di cifratura moderno, studiato durante lo stage in preparazione alla fase di progettazione delle *repository* cifrate prevista negli obiettivi desiderabili.

2.3 Approccio metodologico

Il percorso di lavoro ha seguito un andamento alternato tra fasi teoriche e fasi pratiche, con transizioni determinate dall'avanzamento della comprensione del sistema e dalla disponibilità del materiale.

Nella **fase iniziale** l'attenzione era rivolta allo studio delle tecnologie crittografiche — chiavi SSH_G, firma-digitale, AGE_G — attraverso documentazione ufficiale, esempi pratici al terminale e confronto con il tutor aziendale. Parallelamente ho iniziato a esplorare RVC_G dall'esterno, analizzandone il comportamento tramite i comandi disponibili e studiando il formato dei file prodotti.

Una volta ottenuto accesso ai **sorgenti** CPL_G, l'approccio è cambiato: dall'analisi esterna (*black-box*) si è passati all'analisi interna (*white-box*), con lettura sistematica del codice, identificazione delle vulnerabilità e progettazione degli interventi migliorativi. Questa fase ha richiesto anche lo studio del linguaggio CPL_G, per il quale non esiste documentazione pubblica — la comprensione è avvenuta tramite lettura del codice esistente e consultazione della documentazione interna fornita dall'azienda.

La **fase di implementazione** ha proceduto in parallelo con l'analisi: man mano che venivano individuate vulnerabilità o carenze, venivano progettate e implementate le relative soluzioni, testandole su una *repository* di simulazione appositamente creata.

2.3.1 Ambiente di simulazione

Per riprodurre scenari di attacco in modo controllato e reversibile, ho configurato un ambiente di test dedicato composto da:

- Una **cartella di lavoro** (`simulazione/`) contenente i file di un progetto fittizio su cui eseguire le operazioni RVC_G.
- Una **repository_G locale** (`repo/`) dove vengono archiviati i *commit*, separata dalla cartella di lavoro.
- Un file `.git/repository.info` che collega la cartella di lavoro alla repository_G, seguendo la convenzione di RVC_G.
- Un file `allowed_signers` nel formato OpenSSH, contenente le chiavi pubbliche degli autori autorizzati, utilizzato manualmente per i test di verifica delle firme tramite `ssh-keygen`.

Questo ambiente ha permesso di simulare scenari realistici — inclusi la manomissione dei file di *commit*, la compromissione delle chiavi SSH_G e la verifica della propagazione degli errori nella catena degli hash_G — senza rischiare di danneggiare dati reali.

Capitolo 3

Tecnologie e fondamenti teorici

In questo capitolo sono illustrate le tecnologie e i concetti teorici alla base del progetto. Partendo dalla crittografia-asimmetrica, vengono descritti SSH_G, AGE_G e i sistemi di controllo versione, fino ad arrivare all'architettura di RVC_G.

3.1 Crittografia asimmetrica e firma digitale

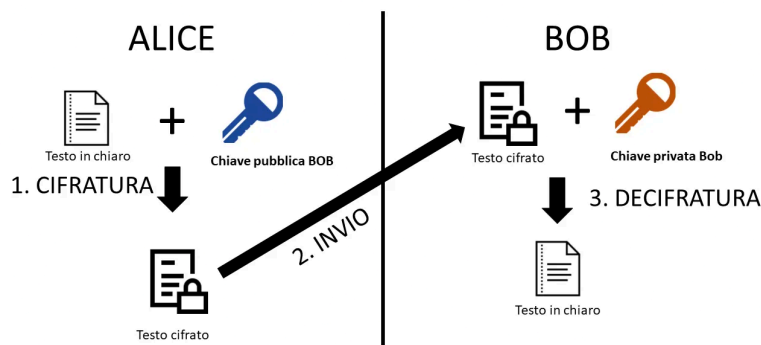
La crittografia è la disciplina che studia le tecniche per proteggere le informazioni rendendole illeggibili a chi non è autorizzato. Nella sua forma classica, detta *crittografia simmetrica*, mittente e destinatario condividono una stessa chiave segreta per cifrare e decifrare i messaggi. Questo approccio presenta un problema fondamentale: come si trasmette la chiave in modo sicuro prima ancora di poter comunicare in modo sicuro?

La risposta a questo problema arrivò negli anni “70 con la *crittografia asimmetrica*, detta anche *crittografia a chiave pubblica*. Ogni utente genera una coppia di chiavi matematicamente collegate tra loro. La **chiave-pubblica** può essere distribuita liberamente a chiunque. La **chiave-privata** deve rimanere segreta e non lasciare mai il dispositivo del proprietario. Le due chiavi sono collegate da una relazione matematica tale per cui ciò che viene cifrato con una può essere decifrato solo con l'altra, ma è computazionalmente impossibile ricavare la chiave-privata a partire da quella pubblica.

Questo meccanismo permette due operazioni fondamentali:

- **Cifratura:** chiunque può cifrare un messaggio usando la chiave-pubblica del destinatario. Solo il destinatario, possedendo la chiave-privata corrispondente, potrà decifrarlo.
- **Firma-digitale:** il mittente cifra un messaggio con la propria chiave-privata. Chiunque, usando la chiave-pubblica del mittente, può verificare che il messaggio provenga effettivamente da lui e non sia stato alterato.

CRITTOGRAFIA CHIAVE ASIMMETRICA



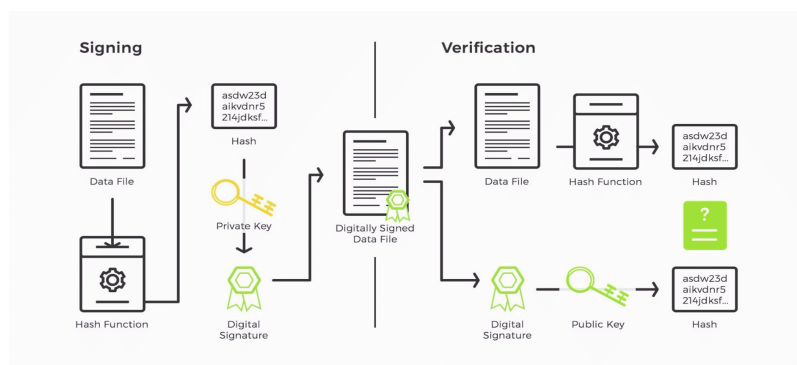
Flusso trasferimento messaggio con uso di chiave pubblica e privata

3.1.1 La firma digitale in dettaglio

La firma-digitale non cifra il messaggio intero — sarebbe inefficiente. Funziona invece in questo modo:

1. Il mittente calcola l'*hash* del messaggio, ovvero una sua impronta digitale di dimensione fissa prodotta da una funzione matematica non invertibile.
2. Il mittente cifra quell'*hash* con la propria chiave-privata. Il risultato è la firma-digitale.
3. Il destinatario riceve messaggio e firma. Decifra la firma usando la chiave-pubblica del mittente, ottenendo l'*hash* originale.
4. Il destinatario calcola autonomamente l'*hash* del messaggio ricevuto e confronta i due valori. Se coincidono, la firma è valida: il messaggio proviene dal mittente dichiarato e non è stato modificato.

La sicurezza di questo meccanismo si basa su due proprietà: la resistenza alle collisioni delle funzioni di *hash_G* (è praticamente impossibile trovare due messaggi diversi con lo stesso *hash_G*) e l'impossibilità computazionale di produrre una firma valida senza possedere la chiave-privata.



Flusso firma di un documento con chiave pubblica e privata

3.1.2 Curve ellittiche: Ed25519

Gli algoritmi crittografici moderni si basano su problemi matematici computazionalmente difficili. RSA, il primo algoritmo a chiave-pubblica diffuso, si basa sulla difficoltà di fattorizzare numeri molto grandi. Gli algoritmi moderni basati su *curve ellittiche* offrono lo stesso livello di sicurezza con chiavi molto più corte, risultando più veloci ed efficienti.

Ed25519 è un algoritmo di firma-digitale basato sulla curva ellittica Curve25519 [1]. Produce chiavi di 256 bit e firme di 512 bit, con un livello di sicurezza equivalente a RSA con chiavi da 3000 bit. È l'algoritmo raccomandato oggi per la firma **SSH** [2] ed è quello utilizzato in questo progetto.

Symmetric Key Size (bits)	RSA and Diffie-Hellman Key Size (bits)	Elliptic Curve Key Size (bits)
80	1024	160
112	2048	224
128	3072	256
192	7680	384
256	15360	521

Comparazione dimensione tra tipologie di chiavi

3.2 SSH e ssh-keygen

SSH è un protocollo di rete che permette di stabilire connessioni sicure tra due macchine attraverso una rete non sicura. Nato come sostituto sicuro di Telnet e rsh, **SSH** cifra tutto il traffico e autentica sia il server che il client, impedendo attacchi di tipo *man-in-the-middle*.

3.2.1 Autenticazione con chiavi

SSH supporta due modalità principali di autenticazione: tramite password e tramite coppia di chiavi. L'autenticazione con chiavi è considerata più sicura e viene raccomandata in tutti i contesti professionali. Il funzionamento è il seguente:

1. L'utente genera una coppia di chiavi con `ssh-keygen`.
2. La chiave-pubblica viene copiata sul server remoto, nella cartella `/.ssh/authorized_keys`.
3. Al momento della connessione, il server invia una sfida cifrata con la chiave-pubblica dell'utente.
4. Il client dimostra di possedere la chiave-privata corrispondente risolvendo la sfida.
5. La connessione viene stabilita senza che la chiave-privata abbia mai lasciato il client.

3.2.2 ssh-keygen per la firma di file

Oltre alla gestione delle chiavi *SSH_G*, *ssh-keygen* offre una funzionalità meno nota ma molto utile: la firma e verifica di file arbitrari [3]. Questo è il meccanismo che *RVC_G* utilizza per firmare crittograficamente i *commit*.

Il comando per firmare un file è:

```
ssh-keygen -Y sign -n <namespace> -f <chiave_privata> <file>
```

Il parametro `-n` specifica il *namespace*, una stringa che identifica il contesto d'uso della firma e previene attacchi di riutilizzo — una firma prodotta con *namespace_G git* non può essere spacciata per una firma con *namespace_G file*.

Il comando per verificare una firma è:

```
ssh-keygen -Y verify -n <namespace> -f <allowed_signers> -I <identità> -s  
<firma> < <file>
```

Il file `allowed_signers` è un elenco di identità autorizzate con le rispettive chiavi pubbliche, nel formato:

```
identita@esempio.com ssh-ed25519 AAAA...chiave...
```

Se la firma è valida e l'identità è presente nel file, il comando restituisce `Good "file" signature for <identità>`.

3.3 AGE

AGE_G (*Actually Good Encryption*) è uno strumento moderno per la cifratura di file, progettato con l'obiettivo di essere semplice, sicuro e componibile [4]. A differenza di PGP, che nel corso degli anni ha accumulato una complessità notevole, *AGE_G* offre un'interfaccia minimale con poche opzioni ben definite.

AGE_G supporta tre modalità di cifratura:

- **Chiave-pubblica:** il file viene cifrato con la chiave-pubblica del destinatario e può essere decifrato solo con la corrispondente chiave-privata.
- **Chiave *SSH_G*:** *AGE_G* può utilizzare le stesse chiavi *SSH_G* già esistenti (incluse le chiavi *Ed25519_G*) come chiavi di cifratura, senza bisogno di generare nuove chiavi dedicate.
- ***Passphrase_G*:** il file viene cifrato con una *passphrase_G*, usando la funzione di derivazione `scrypt` per proteggersi da attacchi a dizionario.

Un file cifrato con *AGE_G* contiene nell'intestazione le informazioni necessarie per la decifratura (il tipo di chiave usata e la chiave di sessione cifrata), seguite dal contenuto

cifrato con ChaCha20-Poly1305, un algoritmo moderno che garantisce sia riservatezza che integrità.

Nel contesto di questo progetto, *AGE* è stato studiato come tecnologia di riferimento per la fase successiva del lavoro, che prevede la progettazione di *progetti cifrati* con distribuzione dei permessi di accesso. In questa fase *AGE* permette di cifrare il contenuto di un *progetto* in modo che solo gli utenti autorizzati — identificati dalle loro chiavi pubbliche — possano decifrarla, senza dover condividere nessuna chiave segreta.

```
age-encryption.org/v1
-> ssh-ed25519 Yu405A MJ134qS7ezoxXy9FQj15TVe1BLjr21hYEpxVxaLqFy8
bs84Hjf93u1AyipJZ50oe2NxxvEeZk1EHkY1/uDZW5pI
--- RxYyA9D3tPSdSdv1cGsV0p/0o0T9XMWCJ1ffjI7mEcs
#0000f,<.'>0^êôx90(Nîÿ
0P$5ø+UüË0pμ[0=ti,...xi'»@I0pÛ#èÇÜt1F,,ô0%×00|J3]Fb“°Juž{§bž#0Àg0é.{00ÍT-00·0F%9*Ã0
```

Esempio di un file cifrato con AGE

3.4 Sistemi di controllo versione

3.4.1 Storia e evoluzione

I sistemi di controllo versione (*Version Control System*, *VCS*) nascono dalla necessità pratica di tenere traccia delle modifiche al codice sorgente nel tempo. Prima della loro diffusione, era comune affidare il versionamento a convenzioni manuali: copie di file con date nel nome, cartelle numerate, archivi compressi. Questo approccio era fragile, difficile da gestire in team e privo di qualsiasi garanzia di integrità.

Il primo sistema formale di controllo versione ampiamente adottato fu **SCCS** (*Source Code Control System*), sviluppato da Bell Labs nel 1972. SCCS introduceva il concetto di *delta*: invece di salvare ogni versione completa del file, memorizzava solo le differenze rispetto alla versione precedente, riducendo significativamente lo spazio occupato.

RCS (*Revision Control System*), sviluppato nel 1982, migliorò SCCS rendendolo più accessibile e introducendo il concetto di *branching*, ossia la possibilità di sviluppare linee di codice parallele a partire da un punto comune.

Entrambi questi sistemi operavano però a livello di singolo file e su singola macchina. Il passo successivo fu l'introduzione dei sistemi *centralizzati*.

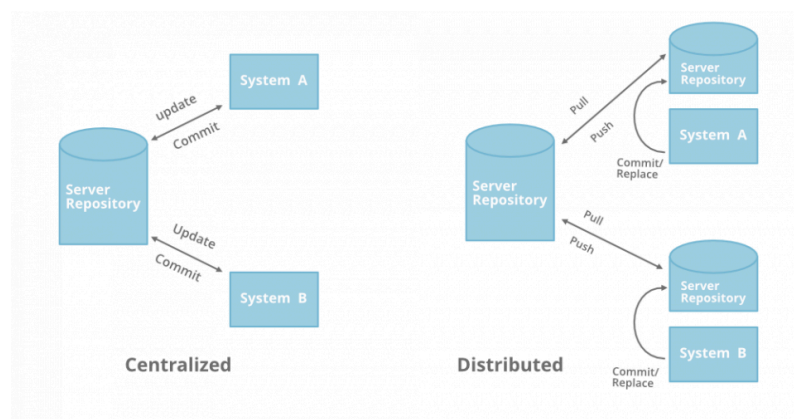
3.4.1.1 Sistemi centralizzati

CVS (*Concurrent Versions System*), negli anni ‘90, portò il controllo versione in rete. Per la prima volta più sviluppatori potevano lavorare contemporaneamente sullo stesso

progetto, con un server centrale che coordinava le modifiche. Il modello era semplice: il server custodisce l'intera storia del progetto; i client effettuano *checkout* (scaricano una versione) e *commit* (inviano le modifiche).

Subversion (SVN), rilasciato nel 2000, nacque esplicitamente come sostituto migliorato di CVS, correggendo molte delle sue limitazioni tecniche. SVN trattava l'intera struttura del progetto come un'unità atomica: un commit poteva riguardare più file contemporaneamente, con la garanzia che o tutte le modifiche venivano salvate o nessuna.

Il limite fondamentale dei sistemi centralizzati era però strutturale: la presenza di un singolo punto di fallimento. Se il server non era raggiungibile, nessuno poteva fare commit. Se il server veniva perso, si perdeva l'intera storia del progetto.



Controllo versioni centralizzato vs distribuito

3.4.1.2 Sistemi distribuiti

Git [5], [6], sviluppato da Linus Torvalds nel 2005 per gestire lo sviluppo del kernel Linux, rivoluzionò il campo introducendo un modello completamente distribuito. In **Git** non esiste un server centrale: ogni sviluppatore possiede una copia completa dell'intera storia del progetto. I commit avvengono localmente e possono essere sincronizzati con altri *repository* in un secondo momento.

Questa architettura offre vantaggi significativi: si può lavorare offline, la storia del progetto è replicata su ogni macchina riducendo il rischio di perdita dei dati, e il *branching* è diventato un'operazione economica e centrale nel flusso di lavoro.

Mercurial, rilasciato nello stesso anno di **Git**, adottò un approccio simile ma con un'interfaccia considerata più accessibile. Oggi **Git** domina il mercato, ma l'ecosistema dei **VCS** distribuiti rimane vivo con strumenti come Fossil e Pijul.

3.4.2 Git in dettaglio

In *Git* ogni oggetto — *blob* (contenuto di un file), *tree* (struttura di una directory), *commit*, *tag* — è identificato da un *hash_G* SHA del suo contenuto. Questo significa che l'identità di ogni oggetto è determinata dal suo contenuto: due oggetti con lo stesso contenuto hanno lo stesso *hash_G*, e qualsiasi modifica produce un *hash_G* diverso.

Un *commit* *Git* contiene: il riferimento all'albero dei file in quello stato, il riferimento al commit precedente (*parent*), i metadati dell'autore e del committer, il messaggio di commit. Questa struttura crea una catena crittograficamente collegata: modificare un commit invalida tutti i commit successivi, poiché i loro *hash_G* dipendono dal *hash_G* del precedente.

Git supporta la firma crittografica dei commit tramite GPG o *SSH_G*. Tuttavia questa funzionalità è opzionale e deve essere abilitata esplicitamente — non fa parte del flusso di lavoro standard.

3.4.3 RVC a confronto con Git

RVC_G condivide con *Git* il modello distribuito ma si differenzia in aspetti fondamentali di architettura e sicurezza.

Caratteristica	<i>Git</i>	<i>RVC_G</i>
Struttura	<i>Repository</i> con oggetti indicizzati	File ZIP navigabili su file-system
Identificazione commit	<i>Hash_G</i> SHA dell'oggetto commit	Timestamp codificato in <i>base36_G</i>
Firma commit	Opzionale (GPG o <i>SSH_G</i>)	Integrata nell'architettura
Server centrale	Non richiesto ma comune	Non richiesto per design
<i>Repository_G</i> multiple	Un remote alla volta tipicamente	Più <i>repository_G</i> sincronizzate nativamente
Linguaggio	C	<i>CPL_G</i>

Confronto tra Git e RVC.

La differenza più significativa riguarda la sicurezza: mentre in *Git* la firma è un'opzione che il singolo sviluppatore può scegliere di abilitare o meno, in *RVC_G* è parte del modello

stesso. Ogni commit produce un file `.sig` che contiene gli *hash_G* crittografici del contenuto e della catena precedente, costruendo una struttura analoga a una *blockchain*: modificare un commit invalida tutti quelli successivi perché l'hash cumulativo non corrisponde più. Il modello di sicurezza proposto nella Sezione 4 estende questa struttura con campi aggiuntivi per supportare la gerarchia di fiducia, i livelli di sicurezza configurabili e la gestione degli incidenti.

3.5 RVC: architettura e funzionamento

3.5.1 Struttura della repository

Una *repository* *RVC_G* è una semplice cartella sul filesystem, senza strutture dati complesse o indici da mantenere. Ogni *commit* è rappresentato da due file:

- Un archivio **ZIP** contenente il *commit* del progetto nella versione corrispondente, incluso il file `.FileManifest` che descrive lo stato di tutti i file tracciati.
- Un file `.sig` contenente i metadati del commit e la firma *SSH_G*, la cui apposizione è opzionale nella versione iniziale del sistema.

I file seguono una convenzione di denominazione che codifica la struttura della storia:

```
<progetto>.<id>.<idPrecedente>.{autore}+tag.zip  
<progetto>.<id>.sig
```

L'id è un timestamp codificato in *base36_G* (cifre 0-9 e lettere A-Z), che permette l'ordinamento cronologico dei commit semplicemente confrontando i nomi dei file. Il riferimento al commit precedente è incorporato nel nome del file ZIP, rendendo la struttura della storia navigabile senza alcun indice aggiuntivo.

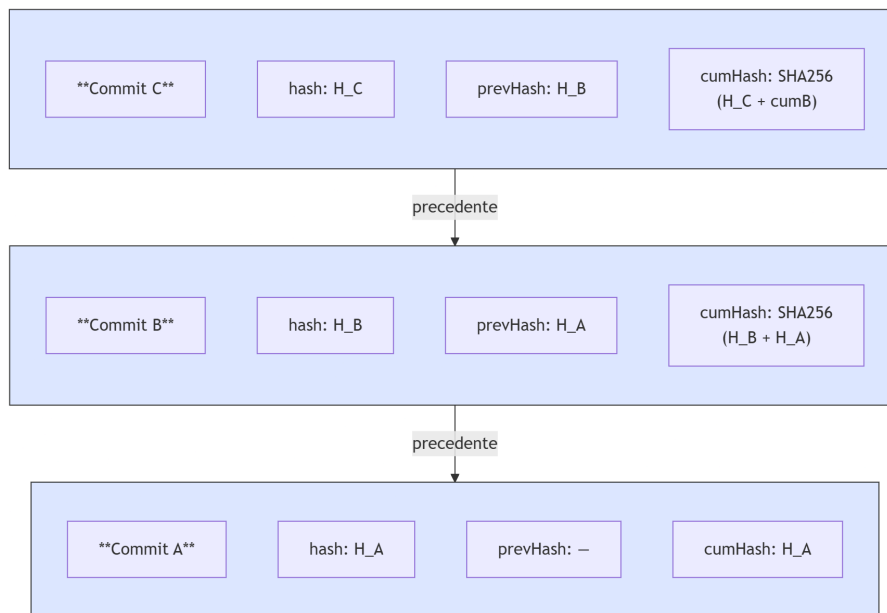
3.5.2 Il file `.sig` e la blockchain degli hash

Il file `.sig` è il cuore del sistema di sicurezza di *RVC_G*. Contiene in formato binario proprietario i seguenti campi:

- `author` : il nome dell'autore del commit
- `comment` : il messaggio del commit
- `fn` : il nome del file ZIP corrispondente
- `id` : l'identificativo del commit
- `prevId` : l'identificativo del commit precedente
- `hash` : lo SHA256 del file ZIP di questo commit
- `prevHash` : lo SHA256 del file ZIP del commit precedente

- `cumulativeHash`: lo SHA256 della concatenazione dell'hash attuale con il `cumulativeHash` del commit precedente

Il `cumulativeHash` è la chiave della sicurezza: ogni commit incorpora crittograficamente l'intera storia precedente. Verificare che il `cumulativeHash` di un commit sia corretto significa verificare implicitamente che tutti i commit precedenti siano integri.



Struttura del file .sig e la catena degli hash cumulativi

Dopo i metadati, il file .sig contiene una firma *SSH_G* nel formato standard:

```

-----BEGIN SSH SIGNATURE-----
...
-----END SSH SIGNATURE-----
  
```

Questa firma attesta che l'autore dichiarato ha effettivamente prodotto il commit, rendendo ogni modifica crittograficamente attribuibile.

Questi sono i campi presenti nella versione attuale di *RVC_G*. Il modello di sicurezza proposto nella Sezione 4 estende questa struttura con campi aggiuntivi — tra cui `security_level`, `allowed_signers`, `branch_status`, `recipients` e `redacted` — necessari per supportare la gerarchia di fiducia e i livelli di sicurezza configurabili.

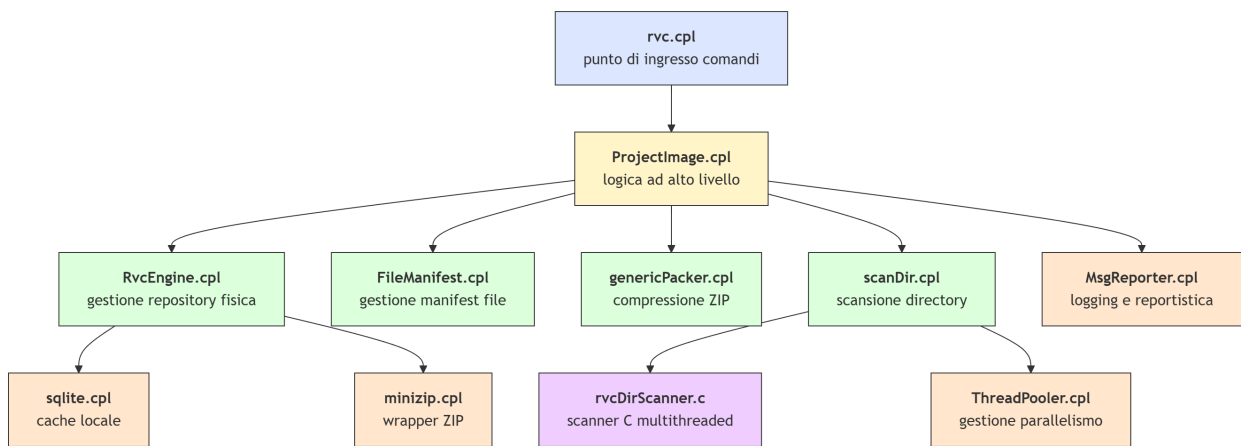
3.5.3 Il linguaggio CPL

RVC_G è scritto in *CPL_G* (*CodePainter Language*), un linguaggio proprietario sviluppato da Zucchetti S.p.A. *CPL_G* è un linguaggio interpretato tipizzato staticamente, con sup-

porto a classi, moduli e gestione dei file. Viene eseguito tramite un interprete (`cpl.exe`) che supporta sia interpretazione diretta che compilazione *JIT_G*.

Le caratteristiche principali che distinguono *CPL_G* dai linguaggi comuni includono la sintassi di assegnazione con `:=`, l'assenza dell'istruzione `return` esplicita (si usa invece la variabile implicita `result`), la dichiarazione obbligatoria di tutte le variabili in cima alla funzione prima di qualsiasi blocco di codice, e la distinzione tra `func` (funzione con valore di ritorno) e `proc` (procedura senza valore di ritorno).

Il codice sorgente di *RVC_G* è organizzato in diversi moduli *CPL_G*, ciascuno con responsabilità ben definite: `ProjectImage.cpl` contiene la logica ad alto livello, `RvcEngine.cpl` gestisce la *repository_G* fisica, `FileManifest.cpl` gestisce il *manifest_G* dei file tracciati.



Struttura dei moduli principali di RVC

3.5.4 Flusso di un commit

Quando un utente esegue `rvc commit`, il sistema esegue i seguenti passi:

1. Legge il file `.FileManifest` nella cartella di lavoro per conoscere lo stato corrente del progetto.
2. Scansiona la directory e calcola le differenze rispetto allo stato precedente.
3. Crea un archivio ZIP con i file modificati e il nuovo `.FileManifest`.
4. Calcola lo SHA256 dell'archivio ZIP.
5. Recupera l'hash e il cumulativeHash del commit precedente dal suo file `.sig`.
6. Calcola il nuovo cumulativeHash come SHA256 della concatenazione dell'hash attuale con il cumulativeHash precedente.
7. Crea il file `.sig` con tutti i metadati.
8. Esegue `ssh-keygen -Y sign` per firmare il file `.sig` e accoda la firma al file.
9. Copia i due file nella *repository_G*.

Capitolo 4

Requisiti e modello di sicurezza

*In questo capitolo viene definito il modello di sicurezza che un sistema di versionamento distribuito moderno dovrebbe soddisfare. Partendo dalle proprietà fondamentali richieste, vengono analizzate le scelte architetturali, i trade-off e le motivazioni che portano alla definizione di un sistema gerarchico di fiducia, di livelli di sicurezza configurabili e di requisiti formali. Il capitolo si conclude con un'analisi del divario che confronta il modello ideale con lo stato iniziale di *RVC_G*, identificando per ciascun requisito il grado di soddisfacimento nella versione del sistema fornita dall'azienda all'avvio dello stage, prima di qualsiasi intervento migliorativo. Data la centralità di questi aspetti all'interno del progetto, il capitolo risulta volutamente più esteso rispetto agli altri, così da consentire un'analisi dettagliata ed esaustiva delle problematiche considerate e delle soluzioni adottate.*

4.1 Sicurezza strutturale nei sistemi di versionamento distribuito

Un sistema di versionamento distribuito è considerato sicuro quando garantisce, per ogni operazione e indipendentemente dal canale di distribuzione, le seguenti proprietà fondamentali.

- **Integrità** — il contenuto di ogni commit non può essere alterato senza che la modifica sia rilevabile. Questa proprietà è garantita da funzioni di *hash_G* crittografiche: dato un archivio ZIP, la sua impronta SHA256 è univoca e deterministica. Qualsiasi modifica — anche di un singolo byte — produce un *hash_G* completamente diverso. Un sistema che garantisce l'integrità permette a chiunque di verificare che il contenuto ricevuto sia identico a quello prodotto dall'autore, senza dover contattare nessuna fonte autoritativa.
- **Autenticità** — ogni commit è crittograficamente attribuibile all'autore che lo ha prodotto. L'autenticità è garantita dalla firma-digitale: l'autore firma il commit con la propria chiave-privata e chiunque può verificare la firma usando la corrispondente chiave-pubblica. Senza autenticità, un sistema può garantire che i dati non siano stati modificati durante il trasferimento, ma non può garantire chi li abbia prodotti originariamente.

- **Non ripudio** — un autore non può negare di aver prodotto un commit firmato con la propria chiave-privata. Il non ripudio è una conseguenza diretta della firma-digitale: poiché solo il possessore della chiave-privata può produrre una firma valida, la presenza di una firma valida è prova crittografica della paternità. Questa proprietà è rilevante in contesti contrattuali e legali — se un fornitore di software distribuisce codice firmato, non può successivamente sostenere di non averlo prodotto.
- **Ordine verificabile** — la sequenza temporale delle modifiche è verificabile e non manipolabile retroattivamente. Questa proprietà è la più complessa da garantire in un sistema distribuito. Non è sufficiente che ogni commit sia integro e autentico — è necessario anche che la loro sequenza sia crittograficamente vincolata, in modo che inserire, rimuovere o riordinare commit sia rilevabile. Questa proprietà è garantita dalla struttura a catena degli *hash*: ogni commit incorpora l'hash del precedente, rendendo impossibile modificare un elemento della catena senza invalidare tutti quelli successivi — con l'unica eccezione del meccanismo di Redazione Trasparente descritto nella Sezione 4.7, riservato esclusivamente alla chiave master.

4.1.1 Confronto con i sistemi esistenti

In *Git* l'integrità è garantita dalla catena di *hash* SHA256 — modificare un commit invalida tutti quelli successivi. Autenticità e non ripudio sono invece opzionali: la firma crittografica tramite *SSH* o GPG deve essere abilitata esplicitamente e non fa parte del flusso di lavoro standard. L'ordine temporale non è verificabile in senso assoluto — *Git* non verifica i timestamp e permette la creazione di commit con date arbitrarie:

```
GIT_AUTHOR_DATE="2020-01-01T00:00:00" git commit -m "commit retrodatato"
```

Il risultato è un commit inserito nella storia con una data arbitraria, indistinguibile da un commit legittimo. Questa non è una vulnerabilità ma una scelta progettuale documentata — *Git* garantisce l'ordine relativo tramite la catena di *hash*, non l'ordine temporale assoluto.

Disponibilità e correttezza non sono la stessa proprietà: un sistema può essere raggiungibile e simultaneamente produrre risultati errati. Un sistema distribuito che incorpora le garanzie crittografiche descritte sopra non dipende dalla correttezza del server per verificare l'integrità dei propri dati. La distinzione tra disponibilità e correttezza di un sistema centralizzato è emersa concretamente il 23 aprile 2026, quando un bug nella funzionalità di merge queue di GitHub ha causato la generazione silenziosa di commit errati per 2.092 pull request in 658 *repository*. Le modifiche precedentemente unite sono state ripristinate dai merge successivi senza alcun avviso. La piattaforma era tecnicamente

operativa durante tutto l'incidente — gli sviluppatori potevano ancora fare push, aprire pull request e fare merge. Il fatto che il merge stesse corrompendo silenziosamente il codice non risultava come incidente nella dashboard di stato.

4.1.2 Applicazione in RVC

RVC_G garantisce l'integrità tramite il cumulativeHash — ogni commit incorpora criticamente l'intera storia precedente. La firma *SSH_G* garantisce autenticità e non ripudio. L'ordine temporale presenta la stessa limitazione di *Git_G* — gli identificativi sono timestamp codificati in *base36_G* basati sull'orologio della macchina. Il cumulativeHash garantisce però l'ordine **relativo**: anche con timestamp errati o manipolati, la sequenza crittografica è verificabile e non manipolabile senza invalidare l'intera catena. Questa limitazione viene documentata come scelta consapevole — per il contesto d'uso di *RVC_G* la garanzia di ordine relativo è sufficiente e la complessità aggiuntiva di un sistema di timestamping certificato non è giustificata.

Per mitigare parzialmente questa limitazione, il modello proposto prevede che l'identificativo di ogni commit sia composto da due componenti: il timestamp codificato in *base36_G* per mantenere l'ordinamento cronologico visivo, e una porzione dell'hash del contenuto per garantire l'unicità crittografica. Un identificativo nella forma `00Q6JTD7XVZ_A3F2B1C4` — dove la prima parte è il timestamp e la seconda sono i primi otto caratteri dell'hash SHA256 dell'archivio ZIP — rende praticamente impossibile la collisione intenzionale mantenendo la leggibilità e l'ordinamento per nome file. Questa modifica è identificata come requisito nel modello ideale e discussa nell'analisi del divario nella Sezione 4.8.

4.2 Requisiti di sicurezza formali

A partire dalle proprietà definite nella sezione precedente, è possibile derivare un insieme di requisiti formali che un sistema di versionamento distribuito sicuro deve soddisfare. Ogni requisito è classificato per priorità obbligatorio (O) o desiderabile (D) e verrà ripreso nell'analisi del divario per valutare lo stato iniziale di *RVC_G*.

4.2.1 Integrità e ordine verificabile

L'integrità è la proprietà fondamentale di qualsiasi sistema di versionamento — senza di essa non è possibile fidarsi del contenuto ricevuto. In un sistema distribuito questa garanzia non può dipendere dalla fiducia nel canale di distribuzione ma deve essere incorporata nei dati stessi. Ogni commit deve portare con sé la prova crittografica della propria integrità, e la struttura della catena deve rendere rilevabile qualsiasi tentativo

di modifica retroattiva. Il problema dell'ordine temporale assoluto, come discusso nella sezione precedente, viene trattato come limitazione documentata piuttosto che come requisito da risolvere completamente — il sistema deve però garantire l'ordine relativo e rendere non manipolabili gli identificativi dei commit.

Codice	Descrizione	Priorità
RS01	Ogni commit deve essere verificabile tramite <i>hash_G</i> crittografico del proprio contenuto	O
RS02	La catena degli <i>hash_G</i> deve essere verificabile in modo che qualsiasi modifica a un commit invalidi tutti i successivi	O
RS03	Gli identificativi dei commit devono essere univoci e non manipolabili tramite timestamp arbitrari	O
RS04	Il sistema deve documentare esplicitamente le proprie limitazioni in termini di ordine temporale assoluto	O

Requisiti di integrità e ordine verificabile.

4.2.2 Autenticità e non ripudio

Garantire l'integrità del contenuto non è sufficiente se non è possibile stabilire chi lo ha prodotto. L'autenticità richiede che ogni commit sia crittograficamente attribuibile al suo autore tramite firma-digitale. Il non ripudio è una conseguenza diretta di questa scelta — un autore non può negare di aver prodotto un commit firmato con la propria chiave-privata. La radice di fiducia dell'intera *repository_G* è il primo commit, firmato dall'amministratore con la propria chiave-privata. Poiché l'autenticità di tutti i commit successivi dipende dalla verificabilità di questa firma, il primo commit è un requisito di autenticità prima ancora che di integrità — senza di esso non è possibile stabilire da chi provenga la catena di autorizzazioni che legittima ogni singola firma successiva.

Codice	Descrizione	Priorità
RS05	Il primo commit di ogni <i>repository</i> deve costituire una radice di fiducia verificabile autonomamente	O
RS06	Il sistema deve supportare e imporre la firma-digitale tramite chiave <i>SSH_G</i> per ogni commit nei progetti configurati con livello di sicurezza maggiore o uguale a 1	O
RS07	Ogni commit di progetto deve referenziare criticamente lo stato di <code>_rvc_root</code> valido al momento della firma, garantendo la verificabilità storica delle autorizzazioni	O

Requisiti di autenticità e non ripudio.

4.2.3 Gestione delle identità

In un sistema multi-utente la gestione delle identità è il meccanismo che traduce le garanzie crittografiche in un modello organizzativo concreto. Non è sufficiente che le firme siano tecnicamente valide — è necessario che il sistema definisca chi è autorizzato a firmare, come vengono gestiti i cambi di personale e chi ha il potere di modificare questi elenchi. La gerarchia a tre livelli — amministratore, responsabile e dipendente — riflette la struttura organizzativa tipica di un'azienda software e permette di delegare la gestione dei permessi mantenendo un controllo centralizzato sulla radice di fiducia. La revoca deve essere immediata e non richiedere operazioni straordinarie — è sufficiente aggiornare il file di autorizzazione nel commit successivo.

Codice	Descrizione	Priorità
RS08	Il sistema deve supportare una gerarchia di fiducia a tre livelli: amministratore, responsabile e dipendente	O
RS09	I permessi di scrittura devono essere configurabili per progetto tramite un file di autorizzazione versionato	O
RS10	La revoca di un'identità deve essere efficace dal commit successivo alla modifica del file di autorizzazione	O
RS11	La successione di un responsabile deve essere gestita esclusivamente dall'amministratore del sistema	D

Requisiti di gestione delle identità.

4.2.4 Sicurezza configurabile

Non tutti i progetti richiedono lo stesso livello di protezione. Un prototipo interno ha esigenze diverse da un modulo che gestisce dati sensibili di un cliente. Imporre lo stesso livello di sicurezza a tutti i progetti sarebbe eccessivamente restrittivo per alcuni e insufficiente per altri. Il modello proposto prevede livelli di sicurezza configurabili per progetto, con il vincolo che il livello non possa essere abbassato nel tempo — una scelta che previene attacchi che cercano di degradare le garanzie di sicurezza di un progetto già avviato. Per i progetti che richiedono la massima riservatezza, il contenuto dei commit può essere cifrato con *AGE_G*, rendendo il codice leggibile solo agli utenti autorizzati.

Codice	Descrizione	Priorità
RS12	Il sistema deve supportare livelli di sicurezza configurabili per progetto, non abbassabili nel tempo	D
RS13	Il contenuto dei commit deve poter essere cifrato con <i>AGE_G</i> per progetti riservati	D

Requisiti di sicurezza configurabile.

4.2.5 Gestione dei branch

I *branch_G* sono uno strumento fondamentale nello sviluppo software parallelo, ma introducono scenari di sicurezza che vanno gestiti esplicitamente. Un *branch_G* può diventare

inutile al termine di una funzionalità, oppure può risultare compromesso a seguito di un commit fraudolento o non autorizzato. In entrambi i casi il sistema deve fornire un meccanismo formale per dichiarare lo stato del *branch*, senza cancellare la storia — che rimane immutabile e verificabile, salvo il meccanismo di Redazione Trasparente descritto nella Sezione 4.7 — ma aggiungendo un commit firmato che ne attesti la chiusura o la compromissione. Questo approccio mantiene la tracciabilità completa degli eventi, inclusa la prova della compromissione stessa.

Codice	Descrizione	Priorità
RS14	I <i>branch</i> compromessi devono poter essere chiusi con un commit firmato che ne attesti la compromissione	D
RS15	Il sistema deve supportare liste di autorizzati differenziate per <i>branch</i>	D

Requisiti di gestione dei branch.

I requisiti obbligatori definiscono le proprietà minime senza le quali il sistema non può essere considerato sicuro per il contesto d'uso descritto. I requisiti desiderabili estendono il modello con funzionalità che aumentano significativamente il livello di sicurezza, ma la cui assenza non compromette le garanzie fondamentali.

I requisiti RS01, RS02 e RS03 corrispondono alla proprietà di integrità e alla gestione dell'ordine verificabile. RS05 e RS06 garantiscono le proprietà di autenticità e non ripudio. RS07 garantisce la verificabilità storica delle autorizzazioni ancorando ogni commit allo stato di `_rvc_root` valido al momento della firma. RS08, RS09, RS10 e RS11 definiscono il modello di gestione delle identità e dei permessi. RS12 e RS13 estendono il modello con funzionalità di sicurezza configurabile. RS04 e RS14 affrontano rispettivamente la documentazione delle limitazioni note e la gestione degli incidenti sui *branch*. RS15 estende il modello con permessi configurabili per *branch* e una regola formale per le merge.

4.3 Gerarchia di fiducia

In un sistema di versionamento distribuito la fiducia non può essere delegata a un server centrale — deve essere incorporata nella struttura stessa dei dati. Il modello proposto definisce una gerarchia a tre livelli operativi più un livello esterno di sola lettura, ciascuno con responsabilità e permessi ben definiti. La gerarchia è asimmetrica: ogni livello superiore può esercitare i poteri del livello inferiore, ma non viceversa.

4.3.1 Commit ordinari e commit amministrativi

Il modello distingue due categorie di commit in base al contenuto dello ZIP.

Commit ordinario — crea o modifica esclusivamente file del progetto, senza toccare nessun file speciale. Può essere firmato da qualsiasi dipendente presente in `allowed_signers`.

Commit amministrativo — crea, modifica o elimina almeno uno dei seguenti file speciali: `allowed_Dipendenti`, `.rvc_policy` o `.rvc_branch_status`. Tutti i commit del progetto `_rvc_root` sono amministrativi per definizione.

La verifica dei commit amministrativi avviene prima che il commit venga prodotto, ed è il punto cruciale per la segregazione dei permessi tra progetti diversi. Il motore confronta il contenuto dello ZIP con quello del commit precedente e, se rileva modifiche ai file speciali, richiede che la firma appartenga all'amministratore o al responsabile del progetto. Per verificare che il firmatario sia il legittimo responsabile di quel progetto, il motore esegue un controllo congiunto: verifica che la chiave del firmatario appartenga a un responsabile (ovvero sia presente nel file `allowed_Responsabili` di `_rvc_root`) e contemporaneamente che sia già presente all'interno del file `allowed_Dipendenti` del progetto stesso. Questo doppio vincolo impedisce a un responsabile di alterare le policy o i permessi di un progetto assegnato a un altro responsabile. L'amministratore (identificato invece dalla presenza della sua chiave nel file `allowed_Dipendenti` di `_rvc_root`) è esente dal controllo locale e ha facoltà di produrre commit amministrativi su qualsiasi progetto. Se la verifica fallisce, il commit viene rifiutato prima ancora della generazione del file `.sig`.

Nel caso specifico del primo commit assoluto di un nuovo progetto, non esistendo uno stato precedente, il motore applica un'eccezione logica: accetta la creazione dei file speciali verificando che il firmatario sia un responsabile in `_rvc_root` e che si sia auto-incluso nel file `allowed_Dipendenti` appena creato.

Il primo commit di qualsiasi progetto è sempre amministrativo — crea `allowed_Dipendenti` e `.rvc_policy` per la prima volta. Di conseguenza solo il responsabile o l'amministratore può inizializzare un progetto.

Questa verifica preventiva ha una conseguenza diretta sulla catena di fiducia: se un commit esiste ed è crittograficamente valido, chiunque lo verifichi può assumere che il firmatario avesse i permessi necessari al momento della produzione. Non è quindi necessario accedere al contenuto dello ZIP per verificare la legittimità di una modifica ai file speciali — è sufficiente verificare che il commit sia crittograficamente valido e che il firmatario fosse autorizzato secondo `_rvc_root`. Questa proprietà vale per tutti i livelli di sicurezza incluso il livello 4, dove la verifica avviene prima della cifratura sul contenuto in chiaro.

I commit amministrativi richiedono sempre la verifica dell'identità del firmatario, indipendentemente dal livello di sicurezza del progetto. Ai livelli 0 e 1 non esiste il file `allowed_Dipendenti` e quindi non esiste la figura del responsabile di progetto — l'unico soggetto autorizzato a produrre commit amministrativi è l'amministratore, che firma con la propria chiave operativa. Questa regola garantisce che i file speciali siano sempre protetti da una firma verificabile anche nei progetti con il livello di sicurezza più basso, dove i commit ordinari non richiedono firma o non verificano l'identità. Il progetto `_rvc_root` segue sempre le stesse regole indipendentemente dal livello — tutti i suoi commit sono amministrativi e devono essere firmati dall'amministratore.

4.3.2 Amministratore

L'amministratore è la radice assoluta di fiducia dell'intera *repository*. Questo ruolo può essere ricoperto da un singolo individuo o da un gruppo direttivo (ad esempio i fondatori o i direttori tecnici). A livello crittografico, il sistema si basa su una netta separazione: esiste un'unica chiave master conservata offline su un dispositivo *air-gapped* o in una cassaforte fisica, e una o più chiavi operative (una per ciascun amministratore autorizzato) utilizzate per le operazioni quotidiane su macchine connesse. La separazione tra la chiave master e le chiavi operative limita la finestra di rischio in caso di compromissione: se una chiave operativa viene rubata o esposta, la chiave master interviene per revocarla e nominarne una nuova senza invalidare le altre chiavi operative o perdere il controllo della *repository*. La prima operazione alla creazione di una *repository* è produrre il file `allowed_Responsabili` — l'elenco delle chiavi pubbliche dei responsabili autorizzati — e firmarlo con la chiave-privata operativa. Questo file e la sua firma costituiscono il primo commit della *repository* e la radice di fiducia da cui deriva tutta la catena di verifica successiva. L'amministratore ha accesso in lettura e scrittura a tutti i progetti della *repository* a qualsiasi livello di sicurezza, incluso il livello 4 con contenuto cifrato — la sua chiave-pubblica è sempre inclusa tra i destinatari autorizzati.

4.3.3 Responsabile di progetto

Il responsabile gestisce uno o più progetti all'interno della *repository*. Il ruolo viene assegnato dall'amministratore tramite inclusione nel file `allowed_Responsabili` — non può essere auto-assegnato né delegato a un altro responsabile. Per ogni progetto gestito, il responsabile mantiene il file `allowed_Dipendenti`, che elenca le chiavi pubbliche dei dipendenti autorizzati a committare. Questo file è versionato all'interno dello ZIP di ogni commit del progetto, fa parte del contenuto hashato e firmato, e la sua storia è completamente tracciabile.

Il responsabile aggiunge o rimuove dipendenti dal proprio progetto in autonomia tramite commit amministrativi — modificando il file `allowed_Dipendenti` e firmando i commit con la propria chiave-privata. L'amministratore può sovrascrivere il file `allowed_Dipendenti` di qualsiasi progetto in qualsiasi momento — il suo potere non è vincolato dalla struttura gerarchica. Se un responsabile lascia l'azienda o viene rimosso dal ruolo, l'amministratore nomina un sostituto aggiornando il file `allowed_Responsabili`. Fino alla nomina del sostituto il progetto entra in stato di attesa: i dipendenti esistenti possono continuare a committare, ma non è possibile aggiungere nuovi dipendenti né modificare i permessi esistenti.

Il responsabile può alzare il livello di sicurezza del proprio progetto in qualsiasi momento producendo un commit firmato che aggiorna il file `.rvc_policy`. Il livello non può essere abbassato — questa operazione viene rifiutata dal motore indipendentemente dall'identità del firmatario. L'amministratore ha lo stesso potere su qualsiasi progetto della *repositoryG*.

4.3.4 Dipendente

Il dipendente è autorizzato a produrre commit ordinari su un progetto se e solo se la propria chiave-pubblica è presente nel campo `allowed_signers` del commit più recente di quel progetto. I permessi sono definiti per progetto — un dipendente autorizzato su ProgettoA non ha accesso a ProgettoB, anche se entrambi appartengono alla stessa *repositoryG*. I permessi sono definiti per progetto e, opzionalmente, per *branchG*: per impostazione predefinita un dipendente autorizzato su un progetto può committare su qualsiasi *branchG* di quel progetto, ma il responsabile può creare *branchG* con liste di autorizzati ristrette, come descritto nella sezione dedicata ai permessi per *branchG*.

I dipendenti non possono produrre commit amministrativi — qualsiasi tentativo di creare, modificare o eliminare un file speciale (`allowed_Dipendenti`, `.rvc_policy`, `.rvc_branch_status`) viene rifiutato dal motore indipendentemente dalla presenza della firma. Questa restrizione vale per tutti i livelli di sicurezza.

La revoca è operativa dal commit successivo alla modifica del file `allowed_Dipendenti`: il dipendente rimosso non può produrre commit validi sul progetto. I commit prodotti prima della revoca rimangono validi in quanto firmati da un'identità che era autorizzata al momento della firma — la storia del progetto è immutabile e ogni modifica ai permessi è tracciata nella catena.

4.3.5 Cliente o Guest

Il cliente o guest riceve la *repository_G* e verifica autonomamente autenticità e integrità del contenuto, senza dipendere dall'infrastruttura del produttore. La verifica parte dalla chiave-pubblica operativa dell'amministratore, ottenuta tramite un canale indipendente dalla *repository_G* stessa — ad esempio il sito ufficiale del produttore distribuito tramite HTTPS. Con questa chiave il cliente verifica la firma sul file `allowed_Responsabili` del primo commit e da lì risale crittograficamente all'intera catena di autorizzazioni e commit. Il cliente opera in sola lettura e non ha permessi di scrittura sulla *repository_G*. Quando riceve un aggiornamento, verifica che i nuovi commit si colleghino correttamente alla catena già in suo possesso. Nei progetti di livello 4 la chiave-pubblica *AGE_G* del cliente deve essere registrata tra i destinatari autorizzati (recipients) per poter decifrare il contenuto. Questo meccanismo sfrutta la separazione architetturale dei permessi: il cliente è autorizzato alla lettura tramite *AGE_G*, ma la sua assenza dal file `allowed_Dipendenti` gli impedisce crittograficamente di produrre commit validi, proteggendo l'integrità dello sviluppo. La verifica della catena e delle firme rimane comunque possibile anche per i non autorizzati senza decifrare, poiché l'hash nel file `.sig` è calcolato sul contenuto cifrato.

4.3.6 Inizializzazione della repository

La creazione di una nuova *repository_G* segue questa procedura:

1. L'amministratore genera la singola coppia di chiavi master con `ssh-keygen -t ed25519` e conserva la chiave-privata master su un dispositivo offline.
2. Gli amministratori generano le proprie coppie di chiavi operative sui rispettivi computer di lavoro.
3. Viene creato il primo commit del progetto `_rvc_root`. Questo commit è fondamentale perché inizializza lo stato del motore e deve contenere:
 - Il file `master.pub` (la chiave-pubblica master in chiaro).
 - Il file di certificato `.sig`.
 - Il file `allowed_Dipendenti` contenente l'elenco di tutte le chiavi pubbliche operative.
 - Il file `allowed_Responsabili` (inizialmente vuoto o con i primi nominati).
4. Questo primo commit viene firmato con la chiave master stessa, stabilendo l'ancora di fiducia interna al sistema.
5. Infine, la chiave-pubblica master viene pubblicata su un canale indipendente (es. sito web HTTPS). Il cliente verifica che la `master.pub` dentro la *repository_G* coincida con quella sul sito web, validando a cascata l'intera catena.

4.3.7 Compromissione di una chiave operativa

La compromissione di una chiave operativa è lo scenario critico del modello. Si utilizza la chiave master — conservata offline — per revocare esclusivamente la chiave operativa compromessa, lasciando intatte le eventuali altre chiavi operative valide. La procedura è la seguente:

1. Viene recuperato il dispositivo offline contenente la chiave-privata master.
2. Se necessario, il soggetto compromesso genera una nuova coppia di chiavi operativa.
3. Viene prodotto uno speciale commit amministrativo su `_rvc_root` che aggiorna il file `allowed_Dipendenti` (inserendo la nuova chiave e/o rimuovendo la vecchia compromessa) e aggiorna i certificati.
4. Questo commit di revoca viene firmato eccezionalmente con la **chiave-privata master**.
5. Il motore di *RVC_G* riceve il commit. Poiché la chiave master non è elencata in `allowed_Dipendenti`, il motore procederebbe a rifiutarlo. Tuttavia, prima di emettere il rifiuto definitivo, il motore verifica la firma del commit contro il file `master.pub` registrato in modo immutabile nel commit iniziale di `_rvc_root`. Se la firma combacia, il motore riconosce l'autorità suprema della chiave master e accetta il commit; altrimenti lo rifiuta.

4.3.8 Inizializzazione di un progetto

La creazione di un nuovo progetto all'interno di una *repository_G* esistente segue percorsi diversi a seconda del livello di sicurezza scelto. Il primo commit di qualsiasi progetto è sempre un commit amministrativo, ma i file da generare e i soggetti autorizzati cambiano in base al contesto.

Per i **progetti ai livelli di sicurezza 2, 3 e 4**, l'inizializzazione è gestita dal responsabile nominato, senza necessità di intervento dell'amministratore. La procedura è la seguente:

1. Il responsabile crea il file `allowed_Dipendenti` con le chiavi pubbliche dei dipendenti autorizzati. In questa fase inaugurale, il motore richiede tassativamente che il responsabile includa la propria chiave-pubblica nel file, in modo da stabilire il vincolo di appartenenza al progetto discusso in precedenza.
2. Il responsabile crea il file `.rvc_policy` con il livello di sicurezza scelto (da 2 a 4) e, per il livello 4, la lista iniziale dei destinatari autorizzati alla decifratura.
3. Il responsabile produce il primo commit del progetto, firmato con la propria chiave-privata. Il motore accetta il commit verificando che il firmatario sia in `_rvc_root` e si sia auto-incluso nel nuovo `allowed_Dipendenti`.

Per i **progetti ai livelli di sicurezza 0 e 1**, non esistendo il file `allowed_Dipendenti` né la figura del responsabile, l'inizializzazione può essere eseguita esclusivamente dall'amministratore. L'amministratore produce un primo commit contenente unicamente il file `.rvc_policy` (che dichiara il livello 0 o 1) e la firma con la propria chiave operativa. Qualsiasi tentativo da parte di un responsabile o di un dipendente di inizializzare un progetto a questi livelli viene rifiutato dal motore.

Il livello di sicurezza definito nel primo commit non può essere abbassato — può essere alzato in qualsiasi momento tramite un commit amministrativo firmato dal responsabile o dall'amministratore. Questa scelta elimina la possibilità di degradare le garanzie di sicurezza di un progetto già avviato.

4.3.9 File amministrativi della repository

In *RVC_G* ogni commit appartiene a un progetto — non esiste il concetto di commit globale della *repository_G*. I file amministrativi `allowed_Responsabili` e la sua firma devono però risiedere nella *repository_G* in modo verificabile e versionato, indipendentemente da qualsiasi progetto specifico.

Il modello proposto risolve questo problema definendo un progetto riservato con nome convenzionale `_rvc_root`, dedicato esclusivamente all'amministrazione della *repository_G*. Al suo interno risiede il file `allowed_Responsabili` e la sua firma. Anche `_rvc_root` contiene al suo interno un proprio file `allowed_Dipendenti`. In questo file è presente unicamente la chiave-pubblica operativa dell'amministratore. Solo l'amministratore è quindi autorizzato a committare su questo progetto, seguendo la medesima struttura logica di tutti gli altri.

Il nome `_rvc_root` è riservato per convenzione del modello. Per prevenire conflitti, il motore verifica all'inizializzazione che questo nome non sia già in uso e lo riserva automaticamente — qualsiasi tentativo di creare un progetto con questo nome da parte di un responsabile o dipendente viene rifiutato.

Questa scelta è preferita all'alternativa di file speciali nella radice della *repository_G* perché non richiede modifiche architetturali al motore e mantiene la coerenza del modello — la verifica della radice di fiducia usa esattamente la stessa logica della verifica di qualsiasi altro progetto.

Il progetto `_rvc_root` opera al livello di sicurezza 3 — ogni commit deve essere firmato dall'amministratore e la firma viene verificata contro il campo `allowed_signers` del `.sig`, che contiene esclusivamente la chiave-pubblica operativa dell'amministratore. Il livello 3 garantisce la verifica dell'identità del firmatario senza richiedere la cifratura del contenuto

— `__rvc_root` deve rimanere leggibile da qualsiasi soggetto che voglia verificare la catena di fiducia.

4.3.10 Il file `.rvc_policy`

Il file `.rvc_policy` definisce le proprietà di sicurezza di un progetto ed è collocato nella radice dello ZIP di ogni commit. È un file speciale — la sua creazione e modifica sono operazioni amministrative riservate al responsabile o all'amministratore. I campi che il file deve contenere sono i seguenti:

- `security_level`: valore intero da 0 a 4 che definisce il livello di sicurezza del progetto. Questo valore viene estratto dal motore e riportato nel campo `security_level` del `.sig` ad ogni commit, in modo che il livello sia verificabile senza accedere allo ZIP. Il livello non può essere abbassato nei commit successivi.
- `recipients`: lista delle chiavi pubbliche dei destinatari autorizzati alla decifratura. Presente solo nei progetti a livello 4. Definisce la lista iniziale dei destinatari al momento della creazione del progetto e include sia i soggetti autorizzati alla scrittura sia eventuali «Guest» in sola lettura (clienti, auditor). Le modifiche successive avvengono tramite commit amministrativi che aggiornano sia questo campo che l'header `AGEG` dello ZIP cifrato.

Il file `.rvc_policy` non contiene informazioni sulle identità dei dipendenti — quelle risiedono in `allowed_Dipendenti`. La separazione tra policy di sicurezza e lista delle identità permette di aggiornare i due aspetti indipendentemente, mantenendo in entrambi i casi la tracciabilità completa nella catena dei commit.

4.3.11 Struttura del file `.sig` nel modello proposto

Il file `.sig` è il punto di contatto tra il contenuto crittografico e il modello di sicurezza. Nel modello proposto la sua struttura estende quella attuale di `RVCG` con i campi necessari per supportare la gerarchia di fiducia, i livelli di sicurezza configurabili e la gestione dei `branchG`. Il `.sig` è firmato crittograficamente nella sua interezza — qualsiasi modifica a uno qualsiasi dei suoi campi invalida la firma e quindi il commit.

I campi del `.sig` nel modello proposto sono i seguenti:

- `author`: identificativo dell'autore del commit, nella forma definita dal file `allowed_Dipendenti` del progetto. Il formato — email, nome opaco o qualsiasi altra convenzione — è una scelta dell'organizzazione che gestisce la `repositoryG`.
- `comment`: messaggio descrittivo del commit.
- `fn`: nome del file ZIP corrispondente.

- `id` : identificativo del commit, composto da timestamp in *base36* e *hash* parziale del contenuto — ad esempio `0Q6JTD7XVZ_A3F2B1C4`.
- `prevId` : identificativo del commit precedente.
- `hash` : SHA256 del file ZIP di questo commit.
- `prevHash` : SHA256 del file ZIP del commit precedente.
- `cumulativeHash` : SHA256 della concatenazione dell'hash attuale con il `cumulativeHash` del commit precedente.
- `rvc_root` : identificativo (ID) dello stato di `_rvc_root` valido al momento della creazione del commit — ovvero l'ultimo commit disponibile di `_rvc_root` oppure un commit precedente se nel frattempo non c'è stato alcun avanzamento. Questo campo ancora il commit di progetto a uno stato specifico della gerarchia di fiducia, garantendo che la verifica avvenga contro la lista dei responsabili e degli amministratori valida in quel preciso istante. Il motore impone che questo valore sia **cronologicamente non precedente** a quello del commit genitore dello stesso progetto — formalmente:

$$C_n.rvc_root \geq C_{n-1}.rvc_root$$

, dove l'uguaglianza corrisponde al caso in cui `_rvc_root` non sia avanzato fra i due commit.

- `security_level` : livello di sicurezza del progetto — da 0 a 4. Estratto dal file `.rvc_policy` dello ZIP e riportato in chiaro nel `.sig` per permettere al motore di applicare le regole corrette senza dover decifrare il contenuto. Questo è necessario in particolare per i progetti a livello 4 — il motore deve sapere che il contenuto è cifrato prima ancora di tentare di leggerlo. La conseguenza è che il livello di sicurezza di un progetto è visibile a chiunque possa leggere il `.sig`, incluso il fatto che un progetto sia riservato. Questo è considerato accettabile perché l'esistenza di un progetto è già visibile dalla struttura dei file nella *repository*.
- `allowed_signers` : elenco delle chiavi pubbliche *SSH* degli identificativi autorizzati a committare al momento di questo commit. Presente solo nei progetti a livello 2, 3 e 4 — estratto dal file `allowed_Dipendenti` dello ZIP prima di qualsiasi cifratura e riportato in chiaro nel `.sig`. Questo garantisce che il campo sia sempre leggibile indipendentemente dal livello di sicurezza del progetto: anche al livello 4, dove lo ZIP viene cifrato dopo l'estrazione, `allowed_signers` rimane in chiaro nel `.sig` e permette la verifica delle firme senza dover decifrare il contenuto. Per il progetto `_rvc_root` contiene esclusivamente la chiave-pubblica operativa dell'amministratore. Nei commit ordinari ai livelli 0 e 1 questo campo è assente. Nei commit amministrativi ai livelli 0 e 1, dove solo l'amministratore può operare, il campo è presente e contiene esclusivamente la chiave-pubblica operativa dell'amministratore.

- `branch_status`: stato corrente del `branchG` — active, archived o compromised. È presente in ogni commit e riflette il contenuto del file `.rvc_branch_status` dentro lo ZIP. Il motore legge sempre questo campo direttamente dal `.sig` — senza dover accedere allo ZIP — indipendentemente dal livello di sicurezza del progetto. Questo garantisce che la gestione dei `branchG` funzioni correttamente anche per i progetti a livello 4 dove lo ZIP è cifrato. Il file `.rvc_branch_status` dentro lo ZIP rimane la fonte di verità completa e può contenere informazioni aggiuntive — motivazione, riferimenti, note — accessibili a chi ha i permessi di lettura.
- `mergeFrom`: nome del `branchG` sorgente, presente nel `.sig` solo se il commit è stato generato da un'operazione di merge.
- `recipients`: elenco delle identità complete dei destinatari autorizzati alla decifrazione del contenuto ZIP. Presente solo nei progetti a livello 4. Contiene le chiavi pubbliche `AGEG` dei destinatari in chiaro — chiunque possa leggere il `.sig` può determinare chi ha accesso al contenuto cifrato. Questa scelta è deliberata: la complessità di meccanismi di oscuramento parziale introduce buchi nella verificabilità senza offrire garanzie di riservatezza robuste.

Dopo questi campi il file `.sig` contiene la firma `SSHG` dell'autore nel formato standard, assente al livello 0:

```
-----BEGIN OPENSSH SIGNATURE-----
<contenuto della firma>
-----END OPENSSH SIGNATURE-----
```

Esempio di firma `SSHG`

La presenza di `allowed_signers` nel `.sig` in chiaro risolve il problema della verificabilità per qualsiasi livello di sicurezza: il motore di `RVCG` e qualsiasi terzo possono verificare la firma del commit e la legittimità del firmatario leggendo esclusivamente il `.sig`, senza dover decifrare il contenuto dello ZIP. Il file `allowed_Dipendenti` dentro lo ZIP rimane la fonte di verità completa — contiene le chiavi pubbliche degli autorizzati con le relative informazioni ed eventuali dati aggiuntivi ad uso interno — ma non è necessario per la verifica crittografica.

4.3.12 Catena di fiducia tra progetti

A differenza dei sistemi tradizionali, il modello proposto collega crittograficamente i progetti alla radice tramite il campo `rvc_root` presente nel `.sig`. Mentre ogni progetto mantiene la propria catena di contenuti, la validità di ogni firma è legata a uno “snapshot” della radice di fiducia. Questo risolve il problema della revoca storica: anche se un

amministratore viene rimosso oggi, i suoi commit passati rimangono verificabili perché referenziano un ID di `_rvc_root` in cui la sua chiave era ancora autorizzata.

La verifica completa di un commit di un qualsiasi progetto segue questa catena:

1. La firma del commit viene verificata crittograficamente contro le chiavi pubbliche presenti nel campo `allowed_signers` del `.sig` di quel commit. Il campo `allowed_signers` è fidato perché il verificatore controlla che solo il responsabile o l'amministratore possano produrre i commit amministrativi che lo modificano — qualsiasi altra modifica viene rifiutata. Di conseguenza, se un commit esiste ed è crittograficamente valido, il suo campo `allowed_signers` riflette una lista di autorizzati approvata da chi ne aveva il potere.
2. La firma del commit di `_rvc_root` che ha prodotto la versione corrente di `allowed_Responsabili` viene verificata contro la chiave-pubblica operativa dell'amministratore, presente nel campo `allowed_signers` del `.sig` di `_rvc_root`. A differenza degli altri progetti, il campo `allowed_signers` di `_rvc_root` contiene esclusivamente le chiavi-pubbliche operative degli amministratori — i responsabili sono il contenuto di `_rvc_root`, non i suoi firmatari.
3. La legittimità della chiave-pubblica operativa viene verificata tramite il certificato firmato con la chiave master, presente nel primo commit di `_rvc_root`.
4. La chiave-pubblica master viene verificata tramite il canale indipendente dalla *repository*.

Questa catena implica un requisito operativo: la verifica completa di qualsiasi commit richiede la presenza di `_rvc_root` nella *repository*. Chi riceve la *repository* riceve automaticamente tutti i progetti incluso `_rvc_root` — ma un sistema che distribuisce solo i file di un singolo progetto non permette la verifica completa della catena di fiducia.

Questa catena di verifica si applica ai progetti a livello 2 o superiore, dove il campo `allowed_signers` è presente nel `.sig`. Per i progetti a livello 0 e 1 la verifica delle identità attraverso la catena non è applicabile — è una conseguenza diretta del livello di sicurezza scelto, che non prevede né l'autorizzazione esplicita né la lista degli autorizzati. In questi progetti l'unica garanzia verificabile è l'integrità della catena degli *hash*. Questa limitazione è documentata come scelta consapevole: i livelli 0 e 1 sono destinati a contesti dove la tracciabilità formale delle identità non è un requisito.

4.3.13 Implicazioni di sicurezza della radice pubblica

Il progetto `_rvc_root` non può essere cifrato — deve essere leggibile da qualsiasi soggetto che voglia verificare la catena di fiducia, incluso il cliente. Questa necessità introduce una tensione strutturale tra verificabilità pubblica e riservatezza organizzativa.

Il contenuto di `__rvc__root` espone le chiavi pubbliche dei responsabili e, implicitamente, la struttura organizzativa dell'azienda. Le chiavi pubbliche non sono segrete per definizione, ma la lista dei responsabili è informazione sensibile — rivela chi ha potere decisionale sulla *repository* e rende questi soggetti bersagli privilegiati per attacchi di *social engineering* e *spear phishing*. Analogamente, i nomi dei file ZIP nella *repository* rivelano i nomi dei progetti anche quando il contenuto è cifrato a livello 4.

Il modello propone due mitigazioni parziali. La prima riguarda le identità: invece di utilizzare indirizzi email o nomi reali nel file `allowed_Responsabili`, si possono adottare identificativi opachi — ad esempio `resp-001` — riducendo la leggibilità immediata senza eliminare la tracciabilità per chi ha accesso alle informazioni di mappatura. La seconda riguarda i nomi dei progetti: l'uso di identificativi non descrittivi — ad esempio `PRJ-4A2F` invece di `ModuloPagamenti` — impedisce la mappatura immediata del contenuto della *repository* a partire dalla struttura dei file.

Queste mitigazioni riducono la superficie di esposizione ma non la eliminano. Il trade-off tra verificabilità pubblica e riservatezza organizzativa è una limitazione strutturale del modello — qualsiasi sistema che permette la verifica autonoma della catena di fiducia deve necessariamente rendere pubblica almeno la radice di quella catena.

4.4 Livelli di sicurezza configurabili

Un sistema di versionamento distribuito utilizzato in contesti aziendali deve servire esigenze di sicurezza eterogenee. Un prototipo interno in fase esplorativa, un modulo di produzione distribuito a clienti e un componente che gestisce dati finanziari hanno requisiti di protezione radicalmente diversi. Imporre un livello di sicurezza uniforme a tutti i progetti è eccessivamente restrittivo per alcuni e insufficiente per altri.

Il modello proposto introduce livelli di sicurezza configurabili per progetto, definiti alla creazione del progetto nel file `.rvc_policy` e non abbassabili nel tempo. Il vincolo di non abbassabilità è una scelta deliberata: un attaccante che compromette l'account di un responsabile non può degradare le garanzie di sicurezza di un progetto già avviato — può solo alzarle. Ogni livello è un sovrainsieme del precedente: un progetto a livello 3 soddisfa tutti i requisiti dei livelli 0, 1 e 2.

4.4.1 Livello 0 — Aperto

Nessuna firma è richiesta per i commit ordinari. I commit amministrativi (come il primo commit di inizializzazione prodotto dall'amministratore) costituiscono un'eccezione architetturale globale: devono sempre essere firmati e includere il campo `allowed_signers` nel `.sig`. Per i commit ordinari, invece, chiunque abbia accesso fisico alla *repository*

può aggiungere commit. Il sistema non impone la verifica automatica dell'identità né dell'integrità, sebbene quest'ultima resti calcolabile matematicamente tramite la catena degli *hash_G*. Il file *.sig* per i commit ordinari al livello 0 contiene solo i campi strutturali — *author*, *comment*, *fn*, *id*, *prevId*, *hash*, *prevHash*, *cumulativeHash*, *security_level* e *branch_status* — ma non il campo *allowed_signers* e non la firma *SSH_G*. L'assenza della firma nei commit ordinari è la caratteristica distintiva del livello 0 e viene rilevata dal motore come indicazione che il progetto opera senza controllo delle identità.

Questo livello è appropriato per prototipi interni in fase esplorativa dove la velocità di sviluppo è prioritaria e la tracciabilità formale non è richiesta. Non fornisce nessuna delle quattro proprietà di sicurezza definite nella sezione precedente — né integrità crittografica delle identità, né autenticità, né non ripudio, né ordine verificabile tramite firme.

4.4.2 Livello 1 — Autenticato

Ogni commit deve contenere una firma *SSH_G* valida nel formato standard. Il motore verifica che la firma sia presente e crittograficamente corretta — non verifica l'identità del firmatario né se la chiave appartenga a un soggetto autorizzato. Questo livello garantisce autenticità e non ripudio: ogni commit è attribuibile a chi possiede la chiave-privata corrispondente alla firma, e la presenza della firma è prova crittografica della paternità. Non garantisce autorizzazione — chiunque possieda una chiave *SSH_G* può committare. È appropriato per *repository_G* interne dove tutti i partecipanti sono implicitamente fidati ma si vuole mantenere la tracciabilità delle modifiche.

4.4.3 Livello 2 — Autorizzato

Ogni commit deve essere firmato da una chiave presente nel file *allowed_Dipendenti* del progetto. Il sistema verifica sia la validità crittografica della firma sia l'appartenenza dell'identità alla lista degli autorizzati. Questo livello garantisce autenticità, non ripudio e autorizzazione — solo i soggetti esplicitamente nominati dal responsabile possono produrre commit validi. È il livello base raccomandato per qualsiasi progetto in produzione.

4.4.4 Livello 3 — Verificato

Come il livello 2, con l'aggiunta della verifica obbligatoria della catena degli *hash_G* a ogni operazione. Nessuna operazione — lettura, aggiornamento, push — può procedere se la catena risulta corrotta o incompleta. Mentre nei livelli precedenti la verifica della catena è un'operazione esplicita eseguita su richiesta, al livello 3 è una preconditione implicita di qualsiasi interazione con il progetto. Questo livello è appropriato per codice distribuito a clienti esterni, dove la garanzia di integrità deve essere continua e non delegabile a verifiche periodiche manuali.

4.4.5 Livello 4 — Riservato

Come il livello 3, con l'aggiunta della cifratura del contenuto degli archivi ZIP tramite *AGE_G*. Solo i soggetti la cui chiave-pubblica è registrata tra i destinatari autorizzati possono decifrare e leggere il contenuto. La verifica della catena e delle firme rimane possibile senza decifrare — l'hash nel file .sig è calcolato sul contenuto cifrato, non su quello in chiaro. Questo livello è appropriato per progetti che contengono codice o dati la cui riservatezza è un requisito contrattuale o legale.

Una proprietà fondamentale del Livello 4 è il disaccoppiamento esplicito tra permessi di lettura e permessi di scrittura. Mentre la capacità di produrre commit è governato dal file `allowed_Dipendenti`, la capacità di leggere i sorgenti è governata dalla lista dei destinatari in `.rvc_policy`. Questa asimmetria permette di definire la figura del «Guest» (ad esempio auditor, tester o clienti): utenti la cui chiave è inclusa tra i destinatari per consentire l'ispezione del codice, ma a cui è inibita la scrittura poiché assenti dall'elenco degli `allowed_Dipendenti`. In un progetto a Livello 4, l'insieme degli utenti autorizzati in scrittura deve essere un sottoinsieme degli utenti autorizzati in lettura.

AGE_G supporta nativamente la cifratura per destinatari multipli: il contenuto è cifrato una volta sola con una chiave di sessione, e la chiave di sessione è cifrata separatamente per ogni destinatario autorizzato. La gestione dei destinatari è descritta in dettaglio nella sezione seguente.

Nei progetti a livello 4 il file `allowed_Dipendenti` risiede all'interno dello ZIP cifrato — è parte del contenuto riservato e non è accessibile a chi non ha i permessi di lettura. La verifica crittografica rimane comunque possibile per qualsiasi osservatore grazie al campo `allowed_signers` presente in chiaro nel .sig: questo campo contiene le chiavi pubbliche degli autorizzati al momento del commit ed è parte del contenuto firmato, quindi la sua integrità è garantita dalla firma stessa. Un osservatore senza permessi di lettura può verificare che il commit sia firmato da una chiave presente negli `allowed_signers` del .sig, risalire alla gerarchia tramite `_rvc_root` e verificare l'intera catena di fiducia — senza mai dover decifrare il contenuto del progetto.

Il file `allowed_Dipendenti` dentro lo ZIP rimane la fonte di verità completa per chi ha i permessi di lettura — contiene le chiavi pubbliche degli autorizzati con le relative informazioni ed eventuali dati aggiuntivi ad uso interno.

4.4.6 Gestione dei destinatari nel livello 4

Nei progetti a livello 4 il campo `recipients` del .sig contiene le identità complete dei destinatari autorizzati alla decifratura — le loro chiavi pubbliche *AGE_G* in chiaro. Chiunque possa leggere il .sig può determinare chi ha accesso al contenuto cifrato.

Questa scelta è deliberata. Meccanismi di oscuramento parziale — come fingerprint delle chiavi o lista nascosta — introducono complessità implementativa e buchi nella verificabilità senza offrire garanzie di riservatezza robuste: un osservatore che conosce le chiavi pubbliche dei candidati può sempre risalire alle identità. La riservatezza reale dei destinatari è garantita dalla scelta organizzativa di non distribuire le chiavi pubbliche dei dipendenti, non da meccanismi tecnici nel .sig.

La gestione dei destinatari segue le stesse regole degli `allowed_signers`: l'amministratore è sempre incluso tra i destinatari di qualsiasi progetto a livello 4. Aggiungere o rimuovere un destinatario richiede un nuovo commit amministrativo firmato dal responsabile o dall'amministratore. Questo commit aggiorna il campo `recipients` nel file `.rvc_policy` e rigenera l'header `AGEG` del file ZIP cifrato per riflettere la nuova lista dei destinatari — il contenuto cifrato non deve essere nuovamente prodotto, poiché `AGEG` separa la cifratura del contenuto dalla cifratura delle chiavi di sessione. Il nuovo commit produce un nuovo file .sig con il campo `recipients` aggiornato — i file .sig delle commit precedenti rimangono immutati e continuano a riflettere la lista dei destinatari valida al momento della loro produzione.

Livello	Firma richiesta	Signers verificati	Verifica catena	Contenuto cifrato
0 — Aperto	No	No	No	No
1 — Autenticato	Sì	No	No	No
2 — Autorizzato	Sì	Sì	No	No
3 — Verificato	Sì	Sì	Sì	No
4 — Riservato	Sì	Sì	Sì	Sì

Confronto tra i livelli di sicurezza configurabili.

Il livello di sicurezza è definito nel primo commit del progetto tramite il file `.rvc_policy` e non può essere abbassato nei commit successivi. Al primo commit il motore accetta il livello dichiarato nel `.rvc_policy` senza confronto con commit precedenti — non esistendone. Per ogni commit successivo il motore verifica che il campo `security_level` del .sig sia maggiore o uguale a quello del commit precedente. Qualsiasi tentativo di abbassare il livello viene rifiutato indipendentemente dall'identità del firmatario.

4.4.7 Sequenza temporale per la creazione e l'aggiornamento di un progetto al Livello 4

Un progetto a Livello 4 passa attraverso una sequenza di operazioni ben definita che garantisce l'integrità crittografica e l'applicazione corretta dei permessi di cifratura. La sequenza è la seguente:

Primo commit (inizializzazione):

1. Il responsabile prepara il contenuto sorgente del progetto e crea il file `.rvc_policy` che specifica:
 - `security_level: 4`
 - `recipients`: lista di identità che includono la chiave-pubblica $\underline{AGE}_G$ del responsabile stesso, e opzionalmente altre identità autorizzate a leggere (guest, auditor, ecc.)
 - Opzionalmente, il file `allowed_Dipendenti` con gli sviluppatori autorizzati a committare (la cui lista può contenere o meno il responsabile stesso)
2. Il responsabile firma il commit con la propria chiave-privata $\underline{SSH}_G$.
3. Il motore riceve la richiesta di commit e esegue le verifiche preventive (prima di qualsiasi cifratura):
 - Verifica che il firmatario sia un responsabile presente in `allowed_Responsabili` di `__rvc_root`
 - Verifica che la firma sia crittograficamente valida
 - Verifica che il responsabile sia incluso nella lista `recipients` specificata nel `.rvc_policy` — altrimenti il responsabile stesso non avrebbe i permessi per decifrare e leggere i propri contenuti in futuro
4. Il motore estrae la lista `recipients` dal file `.rvc_policy` in chiaro — a questo punto il contenuto ZIP è ancora in chiaro.
5. Il motore calcola l'hash SHA256 del file ZIP non cifrato (questo $\underline{hash}_G$ viene memorizzato come riferimento interno ma non è esposto nel `.sig` — il `.sig` contiene l'hash del ZIP cifrato).
6. Il motore cifra l'intero ZIP tramite $\underline{AGE}_G$ utilizzando la lista di destinatari estratta: il contenuto viene cifrato una volta sola con una chiave di sessione, e la chiave di sessione viene cifrata separatamente per ogni destinatario.
7. Il motore calcola l'hash SHA256 del ZIP cifrato e lo memorizza nel campo hash del `.sig`.
8. Il motore genera il file `.sig` contenente:
 - `hash`: SHA256 del ZIP cifrato
 - `cumulativeHash`: calcolato sulla base dello stato vuoto (primo commit)
 - `firma`: firma $\underline{SSH}_G$ sui campi di cui sopra

- `recipients`: copia della lista di destinatari in chiaro (parte del contenuto firmato, quindi la sua integrità è garantita dalla firma)
- `security_level: 4`

9. Il motore salva il file ZIP cifrato e il file `.sig` nella *repository*.

Commit successivi (aggiornamenti):

Per ogni nuovo commit al progetto Livello 4:

1. Il motore riceve il commit e verifica preliminarmente la firma e l'autorizzazione come per gli altri livelli (verificando `allowed_Dipendenti` dal commit precedente).
2. Se il commit è **ordinario** (non modifica file speciali):
 - Il motore procede con le operazioni di hashing e cifratura come descritto sopra
 - La lista dei destinatari rimane quella del commit precedente — il motore legge `recipients` dal `.sig` del commit precedente e la utilizza per la cifratura
3. Se il commit è **amministrativo** (modifica `.rvc_policy`, aggiungendo/rimuovendo destinatari):
 - Il motore estrae la nuova lista `recipients` dal nuovo file `.rvc_policy`
 - Il motore verifica ancora che il responsabile sia incluso nella nuova lista (non può escludere sé stesso)
 - Il contenuto ZIP viene cifrato con la **nuova lista di destinatari**
 - L'header *AGE_G* viene rigenerato per riflettere i nuovi destinatari, ma il contenuto cifrato internamente non deve essere nuovamente decomposto e cifrato — *AGE_G* supporta il re-wrapping della chiave di sessione per una nuova lista di destinatari senza toccare il contenuto
4. Il file `.sig` viene generato con il nuovo valore di `recipients`.
5. Lo ZIP cifrato e il nuovo `.sig` vengono salvati — i `.sig` dei commit precedenti rimangono immutati, continuando a riflettere la lista dei destinatari valida al momento della loro produzione.

Proprietà di consistenza garantite:

Questa sequenza garantisce che:

- Chiunque possieda una chiave-privata *AGE_G* corrispondente a una chiave-pubblica in `recipients` può decifrare gli ZIP di qualsiasi commit in cui compare nella lista.
- La cronologia dei destinatari è completamente tracciabile leggendo sequenzialmente i campi `recipients` nei `.sig` dei commit
- La verifica della catena crittografica rimane possibile per chiunque, indipendentemente dai permessi di lettura, poiché gli *hash_G* e le firme sono sempre in chiaro nel `.sig`

- Se un destinatario viene rimosso dalla lista, perde automaticamente la capacità di decifrare i nuovi commit, ma continua a mantenere la capacità di decifrare i commit precedenti in cui era incluso (la chiave di sessione non viene modificata retroattivamente)

L'innalzamento del livello di sicurezza può essere effettuato in qualsiasi momento tramite un commit firmato dal responsabile del progetto o dall'amministratore. Una volta alzato, il nuovo livello diventa il minimo accettabile per tutti i commit successivi — il sistema non permette di tornare al livello precedente.

4.5 Gestione delle identità e ciclo di vita delle chiavi

La sicurezza di un sistema basato su crittografia-asimmetrica dipende interamente dalla riservatezza delle chiavi private. Una chiave-privata compromessa annulla tutte le garanzie crittografiche — autenticità, non ripudio e autorizzazione diventano prive di significato se un attaccante può produrre firme valide a nome di un utente legittimo. Il modello deve quindi definire procedure esplicite per la gestione ordinaria delle chiavi e per la risposta agli eventi straordinari.

4.5.1 Cambio chiave ordinario

Un dipendente può cambiare la propria coppia di chiavi *SSH_G* in qualsiasi momento — per cambio di dispositivo, per policy aziendale di rotazione periodica o per precauzione in seguito a eventi sospetti. La procedura è la seguente:

1. Il dipendente genera una nuova coppia di chiavi con `ssh-keygen -t ed25519`.
2. Il dipendente comunica la nuova chiave-pubblica al responsabile.
3. Il responsabile aggiorna il file `allowed_Dipendenti` rimuovendo la vecchia chiave-pubblica e aggiungendo la nuova.
4. Il responsabile produce un commit amministrativo firmato con la modifica al file `allowed_Dipendenti` — il motore verifica che la firma appartenga al responsabile o all'amministratore prima di accettarla.
5. Dal commit successivo il dipendente firma con la nuova chiave-privata.

I commit prodotti con la vecchia chiave rimangono validi — erano firmati da un'identità autorizzata al momento della firma e il file `allowed_Dipendenti` di quei commit conteneva la vecchia chiave-pubblica. La storia del progetto è immutabile e ogni cambio di chiave è tracciato nella catena.

4.5.2 Revoca per compromissione

Se una chiave-privata viene compromessa — rubata, esposta accidentalmente o sospettata tale — la revoca deve avvenire nel minor tempo possibile. La procedura è identica al cambio ordinario ma con priorità immediata: il responsabile aggiorna `allowed_Dipendenti` rimuovendo la chiave compromessa e produce un commit amministrativo che documenta l'evento.

Esiste una finestra di rischio tra il momento della compromissione e il commit di revoca: durante questo intervallo un attaccante in possesso della chiave-privata rubata può produrre commit fraudolenti che risultano validi. La dimensione di questa finestra dipende dalla rapidità con cui la compromissione viene rilevata e comunicata al responsabile. Il sistema non può eliminare questa finestra — è una limitazione strutturale di qualsiasi sistema basato su revoca — ma la minimizza richiedendo che la revoca sia operativa dal commit successivo senza procedure straordinarie.

I commit fraudolenti prodotti durante la finestra di rischio rimangono nella storia e risultano validi rispetto al file `allowed_Dipendenti` di quel momento. La loro identificazione richiede un'analisi manuale della storia del progetto nel periodo sospetto.

4.5.3 Revoca offline

In un sistema distribuito non esiste un meccanismo di revoca immediata globale — la revoca è un commit che deve raggiungere tutti i nodi della rete. Se un dipendente revocato tenta di fare push su una *repository* che non ha ancora ricevuto il commit di revoca, il push viene accettato localmente ma rifiutato al momento della sincronizzazione con una *repository* aggiornata.

Questo comportamento è accettabile nel contesto d'uso di *RVC_G*: la sincronizzazione avviene tipicamente tramite push esplicito, e il rifiuto è immediato al primo tentativo di push successivo alla revoca. Se il dipendente revocato ha accesso fisico diretto alla *repository* — ad esempio può copiare file nella cartella della *repository* senza passare per il motore di *RVC_G* — il problema non è più di sicurezza del sistema di versionamento ma di controllo degli accessi fisici all'infrastruttura, che è fuori dallo scope di questo modello.

4.5.4 Verifica della monotonicità della radice

Per garantire la coerenza della storia, il motore di *RVC_G* applica una regola di validazione sulla catena:

$$C_n.\text{rvc_root} \geq C_{n-1}.\text{rvc_root}$$

Questo vincolo impedisce a un utente malintenzionato di produrre un commit referenziando una versione vecchia di `_rvc_root` (magari precedente a una revoca) per tentare

di bypassare i nuovi permessi. La radice di fiducia può solo avanzare o restare stabile, mai tornare indietro.

4.5.5 Successione del responsabile

Se un responsabile lascia l'azienda o viene rimosso dal ruolo, l'amministratore è l'unico soggetto autorizzato a nominare un sostituto. La procedura è la seguente:

1. L'amministratore aggiorna il file `allowed_Responsabili` in `__rvc_root` rimuovendo la chiave del responsabile uscente e aggiungendo quella del nuovo responsabile.
2. L'amministratore firma la modifica con la propria chiave operativa e produce un commit su `__rvc_root`.
3. Il nuovo responsabile acquisisce immediatamente i permessi sul progetto e può modificare `allowed_Dipendenti`.

Fino alla nomina del sostituto il progetto rimane in stato di attesa: i dipendenti esistenti continuano a committare normalmente, ma nessuna modifica ai permessi è possibile. Questo stato non interrompe lo sviluppo — interrompe solo la gestione amministrativa del progetto. Se il responsabile uscente era l'unico soggetto con la conoscenza operativa del progetto, il problema è organizzativo e non tecnico — il sistema garantisce la continuità dei commit esistenti ma non può sostituire la conoscenza umana.

4.5.6 Compromissione della chiave dell'amministratore

La compromissione della chiave operativa dell'amministratore è lo scenario più critico del modello. L'amministratore usa la chiave master — conservata offline — per revocare la chiave operativa compromessa e nominarne una nuova. La procedura è la seguente:

1. L'amministratore recupera il dispositivo offline contenente la chiave-privata master.
2. Genera una nuova coppia di chiavi operativa e ne firma la parte pubblica con la chiave master, creando il nuovo certificato di delega.
3. Produce uno speciale commit amministrativo su `__rvc_root` che aggiorna il file `allowed_Dipendenti` (inserendo la nuova chiave operativa e rimuovendo la vecchia compromessa) e aggiorna il certificato di delega.
4. Questo commit di revoca viene firmato eccezionalmente con la **chiave-privata master**.
5. Il motore di *RVC_G* riceve il commit. Poiché il firmatario non è presente in `allowed_Dipendenti`, il motore — prima di emettere il rifiuto definitivo — verifica la firma contro il file `master.pub` registrato in modo immutabile nel commit iniziale di `__rvc_root`. Se la firma combacia, il motore riconosce l'autorità suprema della chiave master, accetta il commit e rende operativa la nuova delega; altrimenti lo rifiuta.

Chiunque possieda la chiave-pubblica master può verificare la legittimità della nuova chiave operativa e, da lì, ricominciare a verificare la catena di fiducia. I commit prodotti con la vecchia chiave operativa rimangono validi — erano legittimi al momento della firma. I commit prodotti da un attaccante con la chiave compromessa durante la finestra di rischio sono identificabili come fraudolenti tramite analisi della storia nel periodo sospetto. La chiave master non viene mai usata nelle operazioni ordinarie — il suo utilizzo è limitato a questo scenario e alla firma iniziale della chiave operativa. Questa separazione garantisce che la compromissione della chiave operativa, per quanto critica, non comporti la perdita irreversibile del controllo della *repository*.

4.6 Gestione dei branch

I *branch* sono uno strumento fondamentale nello sviluppo software parallelo — permettono di isolare funzionalità, correzioni e sperimentazioni senza interferire con il lavoro principale. In un sistema di versionamento sicuro la gestione dei *branch* introduce scenari che vanno affrontati esplicitamente: un *branch* può diventare obsoleto, può essere abbandonato o può risultare compromesso a seguito di commit non autorizzati.

Il principio fondamentale che governa la gestione dei *branch* nel modello proposto è l'**immutabilità della storia**: nessun commit viene mai cancellato o modificato retroattivamente. Qualsiasi intervento su un *branch* — archiviazione, chiusura o dichiarazione di compromissione — avviene aggiungendo nuovi commit che ne attestano lo stato, non rimuovendo quelli esistenti. Questo principio garantisce che la storia del progetto rimanga sempre verificabile nella sua interezza, inclusa la traccia degli eventi straordinari.

4.6.1 Archiviazione di un branch

Un *branch* di sviluppo che ha concluso il proprio ciclo di vita — perché la funzionalità è stata integrata nel *branch* principale o perché è stata abbandonata — può essere marcato come archiviato tramite un commit amministrativo. Il responsabile produce un commit firmato sul *branch* contenente il file speciale `.rvc_branch_status` dentro lo ZIP, che dichiara lo stato archived e può includere motivazione, riferimenti e note aggiuntive. Lo stato viene estratto da questo file e riportato nel campo `branch_status` del `.sig` — in chiaro e parte del contenuto firmato. Il motore legge esclusivamente il campo `branch_status` del `.sig` per determinare lo stato del *branch*, senza dover accedere allo ZIP — questo garantisce il corretto funzionamento a qualsiasi livello di sicurezza, incluso il livello 4 con contenuto cifrato.

I *branch* archiviati sono trattati dal motore in una speciale modalità protetta: rifiutano tassativamente qualsiasi nuovo commit ordinario, cristallizzando di fatto lo stato del

codice. È possibile leggerne la storia in modo completo, ma i dipendenti non possono aggiungervi modifiche. L’archiviazione è un’operazione reversibile esclusivamente tramite un commit amministrativo con la modifica del file `.rvc_branch_status` per impostarlo ad active, con il conseguente aggiornamento del campo `branch_status` nel `.sig`. Questo commit riporta il `branchG` allo stato operativo normale e ristabilisce i permessi di scrittura standard.

4.6.2 Chiusura di branch compromessi

Un `branchG` compromesso è un `branchG` su cui sono state prodotte uno o più commit fraudolenti o non autorizzati — ad esempio durante la finestra di rischio successiva alla compromissione di una chiave-privata. La gestione di questo scenario segue una procedura in due fasi.

Nella prima fase il `branchG` compromesso viene dichiarato tale tramite un commit amministrativo firmato dal responsabile o dall’amministratore. Il file `.rvc_branch_status` dentro lo ZIP dichiara lo stato compromised, l’identificativo del primo commit sospetto, la motivazione e qualsiasi informazione aggiuntiva utile alla gestione dell’incidente. Lo stato compromised viene estratto e riportato nel campo `branch_status` del `.sig` — il motore legge questo campo direttamente e blocca immediatamente il `branchG` senza dover decifrare il contenuto, indipendentemente dal livello di sicurezza del progetto. I commit fraudolenti rimangono nella storia e sono visibili, ma il `branchG` è marcato come non affidabile. Nei casi in cui la presenza stessa del contenuto fraudolento costituisca un problema legale o di sicurezza, è possibile applicare il meccanismo di Redazione Trasparente descritto nella Sezione 4.7 per rendere inaccessibile il contenuto pur mantenendo la catena intatta.

Nella seconda fase viene creato un nuovo `branchG` pulito a partire dall’ultimo commit verificato come integro prima della compromissione. Lo sviluppo riprende sul nuovo `branchG`. Il `branchG` compromesso rimane nella `repositoryG` come evidenza dell’incidente — la sua storia è verificabile e costituisce la prova crittografica di cosa è accaduto e quando. Se necessario, il contenuto dei commit fraudolenti può essere rimosso tramite Redazione Trasparente (Sezione 4.7) mantenendo comunque intatta la traccia forense delle firme e dei timestamp.

Questa procedura garantisce che la risposta a una compromissione non introduca ambiguità nella storia del progetto. Un approccio alternativo — cancellare i commit fraudolenti rompendo la catena crittografica — renderebbe impossibile distinguere una storia ripulita da una storia alterata da un attaccante. Il meccanismo di Redazione Trasparente (Sezione 4.7) offre una terza via: rendere inaccessibile il contenuto fraudolento senza rompere la catena, con una traccia formale firmata dalla chiave master.

4.6.3 Permessi per branch

Il modello proposto estende il concetto di permessi per progetto introducendo la possibilità di definire liste di autorizzati differenziate per *branch_G*. Per impostazione predefinita ogni *branch_G* eredita il file `allowed_Dipendenti` del progetto — il comportamento è identico al modello base. Il responsabile può però creare *branch_G* con restrizioni specifiche producendo un commit amministrativo che inizializza un file `allowed_Dipendenti` dedicato a quel *branch_G*. Da quel momento il motore verifica i permessi del commit rispetto all'`allowed_Dipendenti` del *branch_G* corrente, non quello globale del progetto.

Nei progetti a livello di sicurezza 0 e 1 non esistono restrizioni sui *branch_G* — chiunque possa committare sul progetto può creare *branch_G* liberamente, coerentemente con la filosofia di massima apertura di questi livelli. Nei progetti a livello 2, 3 e 4 la creazione di un *branch_G* è invece un'operazione amministrativa riservata al responsabile del progetto — ovvero un soggetto presente contemporaneamente in `allowed_Responsabili` di `_rvc_root` e in `allowed_Dipendenti` del progetto stesso.

Il caso d'uso principale è la protezione del *branch_G* principale: il responsabile mantiene in `allowed_Dipendenti` del *branch_G* principale solo i dipendenti autorizzati all'integrazione del codice, mentre i *branch_G* di sviluppo hanno liste più ampie che includono tutti i collaboratori del progetto. Un dipendente presente solo nel *branch_G* di sviluppo non può committare direttamente sul *branch_G* principale — il motore rifiuta il commit prima della generazione del `.sig`.

I file speciali — `allowed_Dipendenti`, `.rvc_policy`, `.rvc_branch_status` — sono sempre esclusi dalla merge, indipendentemente dai permessi dei *branch_G* coinvolti. Ogni *branch_G* mantiene quindi il proprio `allowed_Dipendenti` immutato dopo una merge — i permessi non si contaminano tra *branch_G* diversi e ogni *branch_G* conserva il proprio livello di sicurezza configurato.

4.6.3.1 Identificazione del branch

Il *branch_G* di appartenenza di un commit è codificato direttamente nel nome del file ZIP, nella forma:

```
progetto.idCommit.idPrecedente.[nomeBranch]{autore}.zip
```

Il *branch_G* principale non ha nessuna etichetta tra parentesi quadre — la sua assenza è il segnale che il commit appartiene al *branch_G* principale. Questa convenzione non richiede nessun campo aggiuntivo nel `.sig` — il *branch_G* è sempre identificabile dal nome del file.

4.6.3.2 Creazione di un branch con permessi ristretti

La creazione di un *branch_G* con permessi specifici segue questa procedura:

1. Il responsabile crea il nuovo *branch_G* a partire da un commit esistente.
2. Il primo commit sul nuovo *branch_G* è amministrativo e contiene il file `allowed_Dipendenti` con la lista degli autorizzati per quel *branch_G* specifico. Il responsabile deve includere la propria chiave-pubblica in questa lista.
3. Dal commit successivo il motore verifica i permessi rispetto all'`allowed_Dipendenti` del *branch_G* corrente.

Se il primo commit su un nuovo *branch_G* non è amministrativo, il motore eredita automaticamente l'`allowed_Dipendenti` del *branch_G* di origine — il comportamento predefinito rimane invariato per i *branch_G* senza restrizioni esplicite.

4.6.3.3 Merge tra branch con permessi diversi

La merge tra due *branch_G* con `allowed_Dipendenti` diversi è un commit ordinario — non richiede un commit amministrativo — ma il firmatario deve soddisfare una condizione precisa: deve essere presente nell'`allowed_Dipendenti` del *branch_G* di destinazione, ovvero il *branch_G* su cui la merge viene prodotta.

Questa regola ha due conseguenze dirette. La prima è che un dipendente presente solo nel *branch_G* di sviluppo non può fare merge sul *branch_G* principale — non è nell'`allowed_Dipendenti` di destinazione. La seconda è che il responsabile può sempre fare merge su qualsiasi *branch_G* del proprio progetto, indipendentemente dalle liste — è il meccanismo standard dei commit amministrativi che si applica anche alle merge.

Esiste una terza figura che può fare merge: un dipendente presente nell'`allowed_Dipendenti` di entrambi i *branch_G* coinvolti. Questo permette al responsabile di delegare esplicitamente la capacità di merge a un dipendente fidato includendolo nella lista ristretta del *branch_G* di destinazione. Il contenuto della merge — ovvero i file non speciali — viene integrato normalmente. I file speciali del *branch_G* di destinazione rimangono immutati: non vengono mai sovrascritti dal contenuto del *branch_G* sorgente.

Il risultato è un modello flessibile e coerente: i permessi per *branch_G* sono una naturale estensione del modello esistente, usano gli stessi meccanismi di verifica già definiti e non introducono nuove regole architetturali. La complessità aggiuntiva è interamente gestita dal motore — per i team che non usano questa funzionalità il comportamento è identico al modello base.

Operazione	Dipendente in lista	Responsabile	Amministratore
Commit su branch esistente	Sì	Sì	Sì
Creazione branch (li- vello 0-1)	Sì	Sì	Sì
Creazione branch (li- vello 2-4)	No	Sì	Sì
Merge su branch di destinazione	Sì (se in lista dest.)	Sì	Sì

Autorizzazioni per operazioni sui branch.

4.7 Redazione Trasparente

In un sistema di versionamento distribuito basato sul principio di immutabilità della storia, emerge una tensione strutturale con i requisiti legali e organizzativi che possono richiedere la rimozione di contenuto specifico dalla *repository_G*. Un dipendente infedele o un attaccante che compromette le credenziali di un dipendente può inserire nella *repository_G* contenuto illegale, segreti industriali altrui o dati personali non autorizzati — il cosiddetto *poisoning* della *repository_G*. In un sistema centralizzato l'amministratore può riscrivere la storia sul server, ma in un sistema distribuito questa operazione rompe la catena crittografica e lascia tutti i client con una storia divergente senza nessuna traccia formale di cosa è successo e perché.

Il modello proposto introduce un meccanismo denominato **Redazione Trasparente** che permette di rendere inaccessibile il contenuto di uno o più commit senza rompere la catena crittografica, mantenendo la piena verificabilità dei commit precedenti e successivi, e lasciando una traccia formale firmata dall'autorità più alta del sistema.

4.7.1 Principio matematico

La catena degli *hash_G* in *RVC_G* segue questa struttura:

$$\text{cumulativeHash}(N) = \text{SHA256}(\text{hash}(\text{ZIP}_N) + \text{cumulativeHash}(N - 1))$$

La verifica normale controlla che l'hash del file ZIP corrisponda al campo hash nel .sig e che il cumulativeHash sia calcolato correttamente. La Redazione Trasparente introduce

una regola di eccezione nel motore: se il `.sig` di un commit contiene il campo `redacted`: `true` firmato dalla chiave master, il motore salta la verifica di hash e `cumulativeHash` per quel nodo e riprende normalmente dal commit successivo.

Il punto matematicamente cruciale è che il `cumulativeHash` dei commit successivi è calcolato sul `cumulativeHash` dichiarato nel `.sig` del commit redatto — che non cambia. Il `.sig` redatto aggiunge campi nuovi ma non modifica i campi crittografici originali. Di conseguenza i commit successivi al commit redatto rimangono validi senza nessuna modifica — la loro catena è intatta.

4.7.2 Struttura del commit redatto

Un commit redatto mantiene tutti i campi originali del `.sig` invariati e aggiunge i seguenti campi firmati dalla chiave master:

- `redacted`: valore booleano `true` che segnala al motore di applicare la regola di eccezione.
- `redaction_zip_hash`: SHA256 del nuovo file ZIP che sostituisce quello originale. Permette a chiunque di verificare l'integrità del nuovo ZIP senza dover fidarsi del suo contenuto.
- `redaction_authority`: impronta della chiave master che ha autorizzato la redazione.
- `redaction_timestamp`: timestamp della redazione.
- `redaction_legal_ref`: riferimento al procedimento legale o alla motivazione organizzativa che ha giustificato la redazione. Campo libero.
- `redaction_content`: dichiarazione del tipo di contenuto nel nuovo ZIP — `none`, `sanitized`, `encrypted_master` o `encrypted_authority`.
- `redaction_signature`: firma della chiave master su tutti i campi del `.sig` inclusi quelli di redazione. Questa è la firma aggiuntiva che si affianca alla firma originale del dipendente, che rimane presente e verificabile.
- `redaction_count`: contatore intero che parte da 1 alla prima redazione e si incrementa ad ogni redazione successiva dello stesso commit. Permette a chiunque di verificare quante volte un commit è stato redatto senza dover analizzare la storia completa della *repository*. Una ri-redazione non cancella la traccia della redazione precedente — il `REDACTION_NOTICE.json` nel nuovo ZIP documenta la storia completa delle redazioni applicate al commit.

La firma originale del dipendente sul commit non viene rimossa — è prova forense di chi ha prodotto il contenuto originale e quando. La `redaction_signature` della chiave master certifica che l'autorità più alta del sistema ha autorizzato la modifica.

4.7.3 Opzioni per il contenuto del nuovo ZIP

L'amministratore sceglie cosa inserire nel nuovo ZIP in base alla gravità del caso e ai requisiti legali. In tutti i casi il nuovo ZIP contiene sempre il file REDACTION_NOTICE.json con i campi: identificativo del commit originale, *hash_G* originale, data della redazione, riferimento legale, tipo di contenuto sostitutivo e contatto per informazioni.

Le opzioni disponibili sono le seguenti.

Nessun contenuto (`redaction_content: none`) — il nuovo ZIP contiene solo il file REDACTION_NOTICE.json. Tutto il contenuto originale viene rimosso dalla *repository_G*. Questa opzione soddisfa i requisiti legali che richiedono la distruzione del dato — il file originale non esiste più nella *repository_G*, e il limite residuo è quello strutturale di qualsiasi sistema distribuito: le copie già scaricate sui dispositivi locali prima della redazione non possono essere raggiunte dal motore.

Contenuto bonificato (`redaction_content: sanitized`) — il nuovo ZIP contiene il contenuto originale con i file problematici rimossi o sostituiti e tutti gli altri file mantenuti. Utile quando il problema è localizzato a un singolo file in un commit che contiene anche lavoro legittimo che si vuole preservare.

Cifrato per l'amministratore (`redaction_content: encrypted_master`) — il contenuto originale viene cifrato con *AGE_G* usando esclusivamente la chiave master. Solo l'amministratore può recuperarlo accedendo al dispositivo offline. Utile per preservare il contenuto per uso interno o per future indagini mantenendolo inaccessibile a tutti gli altri.

Cifrato per l'autorità (`redaction_content: encrypted_authority`) — il contenuto originale viene cifrato con *AGE_G* usando la chiave-pubblica dell'autorità giudiziaria o regolatoria competente, oltre alla chiave master. L'autorità può accedere al contenuto originale per le sue indagini tramite la propria chiave-privata. Utile nei casi in cui l'autorità ha bisogno di accedere al contenuto come prova.

4.7.4 Redazione massiva e automazione

Quando il dato problematico è distribuito su più commit — ad esempio un file rimasto nella *repository_G* per diversi commit consecutivi — la redazione deve essere applicata a tutti i commit che lo contengono. Il motore supporta tre modalità operative.

Redazione singola — applicata a un singolo commit identificato dal suo identificativo.

Redazione su range — applicata a tutti i commit di un *branch_G* compresi tra due identificativi specificati.

Redazione di *branch_G* intero — applicata a tutti i commit di un *branch_G* dall'inizio alla fine, con marcatura automatica del *branch_G* come compromised. Questa modalità è

la risposta al caso più grave: un *branch_G* il cui contenuto è problematico fin dalla prima commit. Il *branch_G* rimane nella *repository_G* come evidenza formale — la sua catena è verificabile, le firme originali dei dipendenti sono leggibili, i timestamp sono certificati — ma nessun contenuto è accessibile. Anche in questo caso la traccia forense è preservata: si sa chi ha lavorato su cosa e quando, anche se il cosa non è più leggibile.

In tutti e tre i casi il motore produce automaticamente il file REDACTION_NOTICE.json per ogni commit redatto, verifica che la firma della chiave master sia presente e valida prima di procedere, e aggiorna il branch_status del *branch_G* a compromised se non lo è già.

4.7.5 Sincronizzazione con i client esistenti

Quando un client che aveva già sincronizzato la *repository_G* si aggiorna, riceve il nuovo .sig con redacted: true per il commit interessato. Il motore riconosce questo campo, verifica la redaction_signature contro la chiave master e, se la firma è valida, avvia immediatamente la procedura di propagazione: elimina fisicamente il vecchio file ZIP dal dispositivo locale e lo sostituisce con il nuovo ZIP redatto ricevuto dall'aggiornamento. Il .sig locale viene sovrascritto con quello aggiornato. Questa operazione avviene prima di qualsiasi altra operazione successiva alla sincronizzazione — la priorità è garantire che il contenuto problematico non rimanga disponibile localmente più a lungo del necessario.

Se il client non aveva ancora scaricato il file ZIP originale al momento della redazione, non è necessaria nessuna operazione di cancellazione — il client riceve direttamente il nuovo ZIP redatto come parte normale della sincronizzazione.

Nel caso di una ri-redazione — ovvero una seconda o successiva redazione dello stesso commit già redatto in precedenza — il meccanismo è identico. Il motore riceve un nuovo .sig con redaction_count incrementato, verifica la redaction_signature contro la chiave master e sostituisce il ZIP locale con quello aggiornato. Non è necessario nessun trattamento speciale per le ri-redazioni — il motore tratta ogni aggiornamento del .sig con `redacted: true` allo stesso modo, indipendentemente da quante redazioni precedenti siano già state applicate.

Al termine di ogni sincronizzazione in cui è stata applicata almeno una redazione automatica, il motore produce un avviso esplicito all'utente:

Sincronizzazione completata.

Avviso: N commit sono stati redatti dalla chiave master durante questa sincronizzazione.

Per i dettagli consultare i file REDACTION_NOTICE.json corrispondenti.

Il messaggio indica il numero di redazioni applicate e rimanda ai file REDACTION_NOTICE.json per i dettagli. Non elenca i commit redatti nel testo del messaggio — l'utente trova tutte le informazioni nel REDACTION_NOTICE.json di ogni commit interessato. Questa scelta è deliberata: il motore non espone nel log standard più informazioni del necessario sulla natura del contenuto rimosso.

Il sistema non può garantire la cancellazione fisica del contenuto sui dispositivi che avevano già scaricato il ZIP originale prima della sincronizzazione — copie manuali, backup personali o file estratti dalla *repository* e conservati altrove sono fuori dallo scope del modello. La propagazione automatica garantisce che la *repository* distribuita sia pulita dopo la sincronizzazione, non che ogni copia del contenuto esistente nel mondo sia stata eliminata. Questa è una limitazione strutturale di qualsiasi sistema distribuito e non è specifica di *RVCg*.

4.7.6 Garanzie e limitazioni

La Redazione Trasparente offre le seguenti garanzie.

La catena crittografica non si rompe mai — i commit precedenti e successivi al commit redatto rimangono verificabili senza modifiche. Nessun nuovo client che riceve la *repository* dopo la redazione può accedere al contenuto rimosso. La redazione è visibile a tutti — non esiste nessuna storia nascosta, solo una storia dichiarata come modificata con la firma della massima autorità. Esiste una traccia forense completa: firma originale del dipendente, timestamp originale, firma della chiave master, riferimento legale. L'abuso della funzione è rilevabile — ogni redazione è visibile nella *repository* e non può essere nascosta, e ogni uso della chiave master lascia una traccia nel progetto `_rvc_root`.

La propagazione della redazione è automatica e immediata — qualsiasi client che si sincronizza dopo una redazione riceve il nuovo ZIP redatto e il vecchio viene eliminato localmente senza necessità di intervento manuale. In caso di ri-redazione dello stesso commit, il meccanismo si applica nuovamente con le stesse garanzie — il `redaction_count` nel `.sig` documenta quante redazioni sono state applicate.

Le limitazioni residue sono le seguenti.

I file già scaricati sui client locali prima della redazione non possono essere rimossi dal motore — questa è la limitazione strutturale del modello distribuito già discussa. La funzione richiede la chiave master — un attaccante che compromette la chiave master può produrre redazioni fraudolente. La mitigazione è che ogni redazione è pubblica e rilevabile, e che la chiave master è conservata offline.

La propagazione automatica non elimina le copie del contenuto già estratte dalla *repository* e conservate al di fuori di essa — backup personali, file copiati manualmente o

contenuto già letto e salvato dall'utente prima della sincronizzazione sono fuori dallo scope del modello. La responsabilità della *repository_G* termina al confine dei propri file.

4.8 Analisi del divario

Questa sezione confronta il modello di sicurezza definito nelle sezioni precedenti con lo stato iniziale di *RVC_G*, identificando per ciascun requisito il grado di soddisfacimento nella versione del sistema disponibile durante lo stage. I requisiti vengono classificati in tre stati: **soddisfatto** (il comportamento nella versione iniziale corrisponde al requisito), **parziale** (esiste una base implementativa ma il requisito non è completamente soddisfatto) e **assente** (il comportamento non è implementato).

Nella versione iniziale di *RVC_G* nessun requisito risulta completamente soddisfatto — i migliori risultati sono parziali, il che riflette la natura deliberatamente ridotta della versione fornita per lo stage.

4.8.1 Integrità e ordine verificabile

L'integrità strutturale è la proprietà meglio supportata dalla versione iniziale di *RVC_G*. Il calcolo dell'hash SHA256 del file ZIP e del cumulativeHash sono già presenti nel flusso di commit — ogni commit produce un .sig con i valori corretti. Tuttavia la verifica di questi valori non è ancora accessibile tramite interfaccia a riga di comando: esistono funzioni helper nel codice sorgente predisposte per la verifica, ma non sono ancora esposte come comandi utilizzabili. Il sistema calcola ma non verifica — la garanzia di integrità è quindi presente nella struttura dati ma non è ancora azionabile dall'utente.

L'identificativo del commit è attualmente composto dal solo timestamp codificato in *base36_G*, senza la componente *hash_G* del contenuto prevista dal modello. Questo lo rende vulnerabile a collisioni intenzionali basate sulla manipolazione del timestamp. Infine, la limitazione dell'ordine temporale assoluto non è documentata esplicitamente in nessun documento tecnico al di fuori di questa relazione.

Codice	Descrizione	Stato
RS01	Verificabilità tramite <i>hash_G</i> crittografico del contenuto	Parziale
RS02	Verifica della catena degli <i>hash_G</i> a ogni operazione	Parziale
RS03	Identificativi univoci e non manipolabili tramite timestamp	Assente
RS04	Documentazione esplicita delle limitazioni sull'ordine temporale	Assente

Analisi del divario — integrità e ordine verificabile.

RS01 è parziale perché l'hash viene calcolato e memorizzato ma non verificato automaticamente. RS02 è parziale perché il cumulativeHash esiste nella struttura dati ma la sua verifica non è esposta all'utente. RS03 è assente perché l'identificativo è il solo timestamp. RS04 è assente perché la limitazione non era documentata prima di questa relazione.

4.8.2 Autenticità e non ripudio

La firma *SSH_G* è il punto di maggiore maturità della versione iniziale. Il meccanismo è implementato e funzionante — ogni commit può essere firmato con una chiave *SSH_G* e la firma viene apposta al file .sig. Tuttavia la firma è opzionale: deve essere abilitata esplicitamente al momento del commit tramite un parametro della riga di comando. Il modello proposto la rende obbligatoria dal livello di sicurezza 1 in poi.

Non esiste invece nessun concetto di radice di fiducia o prima commit privilegiata. Tutti i commit sono trattati allo stesso modo dal motore — non c'è distinzione tra il commit iniziale che dovrebbe stabilire l'ancora di fiducia e i commit successivi. Il progetto `_rvc_root` e l'intera gerarchia di fiducia sono assenti.

Codice	Descrizione	Stato
RS05	Primo commit come radice di fiducia verificabile autonomamente	Assente
RS06	Firma-digitale <i>SSH_G</i> supportata e imposta per i livelli di sicurezza maggiori o uguali a 1	Parziale
RS07	Riferimento crittografico a <code>_rvc_root</code> in ogni commit	Assente

Analisi del divario — autenticità e non ripudio.

RS05 è assente perché non esiste il concetto di radice di fiducia né di commit privilegiata. RS06 è parziale perché la firma è implementata e funzionante ma opzionale — il modello richiede che sia imposta automaticamente in base al livello di sicurezza del progetto. RS07 è assente perché il campo `rvc_root` nel `.sig` non esiste nella versione iniziale — ogni commit è completamente scollegato dalla radice di fiducia e non esiste nessun meccanismo per verificare le autorizzazioni storiche.

4.8.3 Gestione delle identità

La gestione delle identità è l'area con il divario più ampio tra il modello proposto e la versione iniziale. Non esiste nessuna distinzione tra utenti — amministratore, responsabile e dipendente sono concetti assenti dal motore. Non esiste un file `allowed_Dipendenti` né nessun altro meccanismo per limitare chi può produrre commit validi su un progetto. Di conseguenza non esiste nemmeno il concetto di revoca — non c'è nulla da revocare se non c'è nessuna lista di autorizzati.

La versione fornita per lo stage era deliberatamente sprovvista di questi meccanismi, con l'obiettivo di permettere uno studio autonomo delle vulnerabilità e la progettazione di soluzioni originali. L'assenza di questi controlli è il punto di partenza dell'intero modello proposto in questo capitolo.

Codice	Descrizione	Stato
RS08	Gerarchia di fiducia a tre livelli: amministratore, responsabile e dipendente	Assente
RS09	Permessi di scrittura configurabili per progetto tramite file di autorizzazione versionato	Assente
RS10	Revoca efficace dal commit successivo alla modifica del file di autorizzazione	Assente
RS11	Successione del responsabile gestita esclusivamente dall'amministratore	Assente

Analisi del divario — gestione delle identità.

Tutti i requisiti di questa categoria sono assenti per la ragione strutturale già descritta: senza una lista di autorizzati non è possibile implementare nessuno dei meccanismi che ne dipendono.

4.8.4 Sicurezza configurabile

Non esiste nella versione iniziale nessun concetto di livello di sicurezza per progetto. Tutti i progetti sono trattati in modo identico dal motore — non c'è nessun file `.rvc_policy` né nessun altro meccanismo per configurare il comportamento del sistema per singolo progetto. La cifratura del contenuto tramite *AGE_G* è completamente assente: la versione fornita per lo stage non includeva nessuna implementazione di cifratura degli archivi ZIP.

Codice	Descrizione	Stato
RS12	Livelli di sicurezza configurabili per progetto, non abbassabili nel tempo	Assente
RS13	Cifratura dei commit con <i>AGE_G</i> per progetti riservati	Assente

Analisi del divario — sicurezza configurabile.

4.8.5 Gestione dei branch e incidenti

Non esiste nella versione iniziale nessun meccanismo formale per dichiarare lo stato di un *branch_G*. I *branch_G* sono sequenze di commit senza nessuna metainformazione sullo stato — non esiste il concetto di *branch_G* archiviato, compromesso o bloccato. Il file `.rvc_branch_status` e il campo `branch_status` nel `.sig` sono proposte del modello ideale, assenti nell'implementazione corrente. Il meccanismo di Redazione Trasparente, che permette di rendere inaccessibile il contenuto di commit problematici senza rompere la catena crittografica, è anch'esso una proposta originale di questa relazione e non ha nessuna corrispondenza nella versione iniziale.

Codice	Descrizione	Stato
RS14	<i>Branch_G</i> compromessi chiudibili con commit firmato che ne attesti la compromissione	Assente
RS15	Permessi per <i>branch_G</i> e regola formale per le merge	Assente

Analisi del divario — gestione dei branch e incidenti.

4.8.6 Sintesi

Il confronto tra il modello proposto e lo stato iniziale di *RVC_G* evidenzia un sistema con solide basi crittografiche — la struttura degli *hash_G* cumulativi e il meccanismo di

firma SSH_G sono già presenti e funzionanti — ma privo dei meccanismi organizzativi e di controllo degli accessi necessari per un uso in contesti aziendali con requisiti di sicurezza non banali.

Codice	Requisito	Stato
RS01	Verificabilità tramite <u>hash_G</u> crittografico	Parziale
RS02	Verifica della catena degli <u>hash_G</u>	Parziale
RS03	Identificativi univoci e non manipolabili	Assente
RS04	Documentazione limitazioni ordine temporale	Assente
RS05	Radice di fiducia verificabile autonomamente	Assente
RS06	Firma-digitale <u>SSH_G</u> imposta per livello ≥ 1	Parziale
RS08	Gerarchia di fiducia a tre livelli	Assente
RS09	Permessi configurabili per progetto	Assente
RS10	Revoca efficace dal commit successivo	Assente
RS11	Successione del responsabile	Assente
RS12	Livelli di sicurezza configurabili	Assente
RS13	Cifratura <u>AGE_G</u> per progetti riservati	Assente
RS14	Gestione <u>branch_G</u> compromessi	Assente
RS15	Permessi configurabili per <u>branch_G</u>	Assente
RS07	Riferimento crittografico a <code>_rvc_root</code> in ogni commit	Assente

Sintesi dell'analisi del divario.

I requisiti RS01, RS02 e RS06 sono parzialmente soddisfatti — la base implementativa esiste ma non è ancora completa o accessibile all'utente. Tutti gli altri requisiti sono assenti. Questo divario non è una critica al sistema esistente — RVC_G è stato progettato con obiettivi diversi e la versione fornita per lo stage era deliberatamente ridotta — ma definisce con precisione l'area di intervento che i capitoli successivi affrontano con la simulazione degli scenari di attacco e l'implementazione dei miglioramenti prioritari.

Capitolo 5

Simulazione scenari di attacco

In questo capitolo vengono documentati gli scenari di attacco simulati in ambiente controllato sulla versione iniziale di RVC_G. Per ciascuno scenario viene descritto il contesto dell'attaccante, le tecniche impiegate e i risultati osservati, distinguendo ciò che il motore rileva preventivamente da ciò che emerge solo tramite verifica esplicita. Il capitolo si conclude con una sintesi trasversale che mette in relazione i risultati osservati con i requisiti definiti nella Sezione 4.

5.1 Ambiente di simulazione e metodo

I test sono stati condotti su una repository_G locale composta da cinque commit legittimi firmati con chiave SSH_G, ripristinata allo stato iniziale prima di ogni tecnica tramite una copia di backup. La struttura dei commit dello stato iniziale è la seguente:

Identificativo	Operazione	File coinvolto
0Q6PHT7QCI	Aggiunta	file.txt
0Q6PHUA0SV	Modifica	file.txt
0Q6PHV1YTU	Aggiunta	fileNuovo.txt
0Q6PHW0IJW	Modifica	fileNuovo.txt
0Q6PHWPMPQ	Aggiunta	fileSuperPrivato.txt

Struttura della repository di test allo stato iniziale.

Il verificatore di integrità sviluppato durante lo stage espone il comando `rvc integrity` con il parametro `-signers` che accetta il percorso di un file `allowed_signers` nel formato OpenSSH. Per ogni commit della repository_G il verificatore controlla tre proprietà indipendenti: l'hash SHA256 del file ZIP, la catena dei cumulativeHash e la validità della firma SSH_G rispetto alla lista degli autorizzati. L'output classifica ogni commit con tre

stati — [OK], [WARN] e [ERR] — e produce un contatore finale degli errori critici e degli avvisi.

Lo stato iniziale pulito produce il seguente output del verificatore:

```
> rvc integrity -signers="C:\Users\stemic\stage\allowed_signers"
```

```
Verifica integrita repository: tutti i progetti
allowed_signers: C:\Users\stemic\stage\allowed_signers
analyzing repository C:\Users\stemic\stage\repo\ ...
[OK] 0Q6PHT7QCI hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHUA0SV hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHV1YTU hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHW0IJW hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHWPMPQ hash:OK catena:OK firma:OK (Michele)
Risultato: 0/5 commit con problemi.
Risultato: 0/5 commit con warning.
```

Per ogni tecnica sono stati documentati i comandi eseguiti, l'output completo del verificatore e il comportamento del motore durante le operazioni normali successive all'attacco.

5.2 Scenario 1 — Attaccante esterno senza credenziali

L'attaccante è un soggetto esterno che ha ottenuto accesso fisico o di rete alla cartella della *repository* ma non possiede nessuna chiave *SSH_G* valida. Conosce il formato dei file *RVC_G* dall'osservazione della struttura della *repository*. L'obiettivo è inserire modifiche non autorizzate oppure alterare la storia esistente senza che sia rilevabile dal motore.

5.2.1 T1 — Commit senza firma

L'attaccante produce un commit tramite il motore *RVC_G* senza fornire una chiave *SSH_G*. Nella versione iniziale la firma è opzionale e il motore non la richiede.

```
> echo Aggiunto testo malevolo >> fileSuperPrivato.txt
```

```
> rvc commit -project=simulazione -tag=Produzione -note=MoltoImportante
```

```
Reading manifest, path:.\
packing simulazione.0Q6PI7VZWV.0Q6PHWPMPQ.+Produzione.zip
copying simulazione.0Q6PI7VZWV.0Q6PHWPMPQ.+Produzione.zip to repo\ ...
copying simulazione.0Q6PI7VZWV.sig to repo\ ...
tot. exec time: 0.23 sec
```

```
> rvc integrity -signers="C:\Users\stemic\stage\allowed_signers"
```

```
Verifica integrita repository: tutti i progetti  
[OK] 0Q6PHWPMPQ hash:OK catena:OK firma:OK (Michele)  
[WARN] 0Q6PI7VZWV hash:OK catena:OK firma:ASSENTE (?)  
Risultato: 0/6 commit con problemi.  
Risultato: 1/6 commit con warning.
```

Risultato. Il motore accetta il commit senza nessun avviso. Il verificatore segnala `[WARN] firma:ASSENTE` ma non incrementa il contatore degli errori critici. Tutte le operazioni successive del motore procedono normalmente.

Impatto: Alto. L'attaccante può inserire qualsiasi modifica al codice sorgente senza che il motore lo rilevi. Il commit fraudolento è visibile nella history con un autore vuoto ma non produce nessun segnale automatico di anomalia — è rilevabile solo tramite esecuzione esplicita del verificatore.

Requisiti violati: RS06 — autenticità e non ripudio.

5.2.2 T2 — Commit con identità falsa

L'attaccante produce un commit senza firma dichiarando nel campo author il nome di un autore autorizzato. Poiché non esiste una firma crittografica, il campo author non è verificabile e può contenere qualsiasi valore.

```
> rvc commit -project=simulazione -tag=Produzione -note=MoltoImportante -author=Michele
```

```
Reading manifest, path:.\  
packing simulazione.0Q6PK4VQQR.0Q6PHWPMPQ.{Michele}+Produzione.zip  
ERROR: chiave-privata non trovata ne in rvc.config ne in identities\  
Il commit e stato salvato senza firma Ssh.  
copying simulazione.0Q6PK4VQQR.sig to repo\ ...  
tot. exec time: 0.14 sec
```

```
> rvc integrity -signers="C:\Users\stemic\stage\allowed_signers"
```

```
[WARN] 0Q6PK4VQQR hash:OK catena:OK firma:ASSENTE (Michele)  
Risultato: 0/6 commit con problemi.  
Risultato: 1/6 commit con warning.
```

Risultato. Il motore segnala che la chiave-privata non è stata trovata ma accetta comunque il commit senza firma. Il verificatore produce lo stesso risultato di T1 —

`[WARN] firma:ASSENTE` con il nome dell'autore dichiarato. Un osservatore che legge la history vede il commit attribuito a Michele senza nessun indicatore di anomalia.

Va notato che se l'attaccante dichiara un autore non presente nel file `allowed_signers`, il verificatore segnala `[ERR] firma:FALLITA` anziché `[WARN] firma:ASSENTE` — non è quindi possibile impersonare un autore completamente arbitrario senza essere rilevati.

Impatto: Alto. L'attaccante può impersonare qualsiasi autore autorizzato producendo un commit che nella history risulta indistinguibile da uno legittimo. Il rischio aumenta significativamente in contesti dove la firma non è considerata obbligatoria nella pratica operativa.

Requisiti violati: RS06 — autenticità e non ripudio.

5.2.3 T3 — Modifica del contenuto di uno ZIP esistente

L'attaccante modifica il contenuto di un file ZIP già presente nella *repository* senza aggiornare il .sig corrispondente.

```
> mkdir temp_estrazione
```

```
> tar -xf simulazione.0Q6PHV1YTU.0Q6PHUA0SV.{Michele}+documentazione.zip -C  
temp_estrazione
```

```
> echo aggiunta di codice malevolo >> temp_estrazione\fileNuovo.txt
```

```
> tar -a -c -f simulazione.0Q6PHV1YTU.0Q6PHUA0SV.{Michele}+documentazione.zip -  
C temp_estrazione .
```

```
> rmdir /s /q temp_estrazione
```

```
> rvc integrity -signers="C:\Users\stemic\stage\allowed_signers"
```

```
[OK] 0Q6PHUA0SV hash:OK catena:OK firma:OK (Michele)
[ERR] 0Q6PHV1YTU hash:FALLITO catena:NON VERIFICABILE firma:OK (Michele)
      ERROR: Hash ZIP non corrisponde
[ERR] 0Q6PHW0IJW hash:OK catena:FALLITO firma:OK (Michele)
      ERROR: CumulativeHash non corrisponde
[ERR] 0Q6PHWPMPQ hash:OK catena:FALLITO firma:OK (Michele)
      ERROR: CumulativeHash non corrisponde
Risultato: 3/5 commit con problemi.
Risultato: 0/5 commit con warning.
```

Risultato. Il verificatore rileva immediatamente la discrepanza tra l'hash del file ZIP modificato e quello dichiarato nel .sig. Il commit alterato produce errori a cascata su tutti i successivi. Il motore continua a funzionare normalmente senza rilevare l'alterazione.

Impatto: Basso. La catena degli *hash*_G è già una garanzia operativa nella versione iniziale — la modifica è rilevabile in modo inequivocabile. La protezione è però solo reattiva: il motore non blocca l'alterazione, la rileva solo il verificatore su richiesta esplicita.

Requisiti violati: RS01, RS02 — integrità e ordine verificabile.

5.2.4 T4 — Sostituzione completa di ZIP e .sig

L'attaccante sostituisce sia il file ZIP che il .sig di un commit esistente con valori ricalcolati su contenuto alterato, nel tentativo di produrre un commit che superi la verifica dell'hash.

```
> python -c "import hashlib; print(hashlib.sha256(open('simulazio-
ne.0Q6PHV1YTU...zip', 'rb').read()).hexdigest().upper())"

35EFF8161A9E587248B4595AE1921FE3389663C4E1EFFCA44990918DB3853980
```

```
> python -c "import hashlib; h1='35EFF8...'; h2='7047DC...';
print(hashlib.sha256((h1+h2).encode('utf-16-le')).hexdigest().upper())"

2D2239E1A71520B807315AA212A765C00CDB6508F6A2C939A5B6BC7D7EF513A7
```

```
> rvc integrity -signers="C:\Users\stemic\stage\allowed_signers"

[WARN] 0Q6PHV1YTU hash:OK catena:OK firma:ASSENTE (Michele)
[ERR] 0Q6PHW0IJW hash:OK catena:FALLITO firma:OK (Michele)
      ERROR: CumulativeHash non corrisponde
[ERR] 0Q6PHWPMPQ hash:OK catena:FALLITO firma:OK (Michele)
      ERROR: CumulativeHash non corrisponde
Risultato: 2/5 commit con problemi.
Risultato: 1/5 commit con warning.
```

Risultato. Il commit alterato supera la verifica dell'hash — il ricalcolo corretto inganna il verificatore sul nodo modificato. La catena si rompe nel commit successivo. Applicando il ricalcolo in cascata su tutti i commit successivi è possibile eliminare gli errori critici riducendo l'output a soli warning.

Impatto: Alto. Questa tecnica è il vettore di attacco più significativo dello scenario 1. In assenza di firme *SSH*_G obbligatorie, un attaccante sufficientemente determinato può riscrivere silenziosamente la storia di un progetto producendo solo warning. La firma *SSH*_G

è l'unica protezione che impedisce questo scenario: se presente, il .sig modificato produce `[ERR] firma:FALLITA` e l'attacco è immediatamente rilevabile.

Requisiti violati: RS01, RS02 — integrità e ordine verificabile. RS06 come contromisura necessaria.

5.2.5 T5 — Inserimento di un commit in mezzo alla catena

L'attaccante tenta di inserire un commit tra due commit esistenti modificando i riferimenti `prevId` e `prevHash` e ricalcolando la catena degli `hashG` in cascata. Il risultato è analogo a T4 — i commit modificati producono warning per la firma assente e i commit successivi producono errori di catena fino al completamento del ricalcolo in cascata.

Impatto: Alto. Le implicazioni sono identiche a T4 — senza firme obbligatorie è possibile inserire commit arbitrari in qualsiasi punto della storia riducendo l'output a soli warning. La firma `SSHG` è la contromisura necessaria: qualsiasi modifica al .sig originale invalida la firma e produce un errore critico.

Requisiti violati: RS02, RS03 — integrità e ordine verificabile.

5.2.6 T6 — Replay di un commit legittimo

L'attaccante copia un commit legittimo esistente e lo reintroduce nella `repositoryG` con un nuovo nome file.

```
> copy "simulazione.0Q6PHWPMPQ.0Q6PHW0IJW.{Michele}+documentazione.zip" "simulazione.0Q6PHWPZZZ.0Q6PHWPMPQ.{Michele}+documentazione.zip"
```

```
1 file copiati.
```

```
> copy simulazione.0Q6PHWPMPQ.sig simulazione.0Q6PHWPZZZ.sig
```

```
1 file copiati.
```

```
> rvc integrity -signers="C:\Users\stemic\stage\allowed_signers"
```

```
[OK] 0Q6PHWPMPQ hash:OK catena:OK firma:OK (Michele)
[ERR] 0Q6PHWPZZZ hash:OK catena:FALLITO firma:OK (Michele)
      ERRORE: CumulativeHash non corrisponde
Risultato: 1/6 commit con problemi.
Risultato: 0/6 commit con warning.
```

Risultato. Il verificatore rileva immediatamente l'errore di catena. La history mostra il commit reintrodotta come un `branchG` separato. Il ricalcolo del cumulativeHash è possibile

ma non permette di alterare il contenuto dello ZIP — la firma originale diventerebbe invalida.

Impatto: Basso. L'attaccante non può inserire codice arbitrario — può solo duplicare commit esistenti creando *branch*_G con lo stesso nome, sporcando la history senza modificare il contenuto.

Requisiti violati: RS02, RS03 — ordine verificabile.

5.2.7 T7 — Modifica del .sig senza modificare lo ZIP

L'attaccante modifica solo il file .sig di un commit firmato lasciando lo ZIP intatto e la firma originale invariata.

```
> rvc integrity -signers="C:\Users\stemic\stage\allowed_signers"

[OK] 0Q6PHT7QCI hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHUA0SV hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHV1YTU hash:OK catena:OK firma:OK (Michele)
[ERR] 0Q6PHW0IJW hash:OK catena:OK firma:FALLITA (Michele)
      ERRORE: Firma Ssh non valida
[OK] 0Q6PHWPMPQ hash:OK catena:OK firma:OK (Michele)
Risultato: 1/5 commit con problemi.
Risultato: 0/5 commit con warning.
```

Risultato. Qualsiasi modifica a qualsiasi campo del .sig invalida la firma *SSH*_G, che copre il file nella sua interezza. L'errore è localizzato al solo commit modificato — i successivi rimangono validi perché il cumulativeHash non è stato alterato.

Impatto: Nullo. La firma *SSH*_G è una protezione già operativa nella versione iniziale — qualsiasi tentativo di alterare i metadati di un commit firmato è immediatamente rilevabile.

Requisiti soddisfatti: RS06 — la firma *SSH*_G protegge l'integrità del .sig nella sua interezza.

5.2.8 T8 — Troncamento della catena

L'attaccante elimina fisicamente uno o più commit dalla *repository*_G cancellando i file ZIP e .sig corrispondenti.

```
> del "simulazione.0Q6PHV1YTU.0Q6PHUA0SV.{Michele}+documentazione.zip"
```

```
> del simulazione.0Q6PHV1YTU.sig
```

```
> rvc integrity -signers="C:\Users\stemic\stage\allowed_signers"

[OK] 0Q6PHT7QCI hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHUA0SV hash:OK catena:OK firma:OK (Michele)
[ERR] 0Q6PHW0IJW hash:OK catena:FALLITO firma:OK (Michele)
      ERRORRE: CumulativeHash non corrisponde
[ERR] 0Q6PHWPMPQ hash:OK catena:FALLITO firma:OK (Michele)
      ERRORRE: CumulativeHash non corrisponde
Risultato: 2/4 commit con problemi.
Risultato: 0/4 commit con warning.
```

Risultato. Il verificatore rileva la discontinuità nella catena. La history mostra la catena spezzata come due *branch_G* separati senza connessione.

Impatto: Nullo. Il troncamento è immediatamente rilevabile e non produce nessun vantaggio per l'attaccante. I commit successivi al punto di eliminazione producono errori critici e la history risulta incoerente.

Requisiti violati: RS02 — ordine verificabile.

5.2.9 T9 — Commit con cumulativeHash falsificato

L'attaccante produce un commit con *hash_G* e cumulativeHash calcolati correttamente sul nuovo ZIP ma con prevHash che punta a un commit inesistente.

```
> rvc integrity -signers="C:\Users\stemic\stage\allowed_signers"

[OK] 0Q6PHT7QCI hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHUA0SV hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHV1YTU hash:OK catena:OK firma:OK (Michele)
[ERR] 0Q6PHV1ZZZ hash:OK catena:FALLITO firma:ASSENTE (Michele)
      ERRORRE: CumulativeHash non corrisponde
[OK] 0Q6PHW0IJW hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHWPMPQ hash:OK catena:OK firma:OK (Michele)
Risultato: 1/6 commit con problemi.
Risultato: 0/6 commit con warning.
```

Risultato. Il verificatore rileva l'errore di catena sul commit anomalo lasciando validi tutti gli altri. Il commit risulta isolato e non collegato alla storia legittima.

Impatto: Basso. L'attaccante non riesce a inserire un commit che si agganci alla storia legittima senza produrre errori critici. Il commit falsificato è immediatamente identificabile come anomalo e isolato dalla catena principale.

Requisiti violati: RS02, RS03 — integrità e ordine verificabile.

Nota. I commit successivi risultano OK perchè non sono collegati al commit falsificato, che è isolato come un *branch_G* separato.

5.2.10 T10 — Iniezione di una repository fasulla

L'attaccante crea da zero una *repository_G* completa con una catena di commit senza firma, strutturalmente valida dal punto di vista degli *hash_G*, e la distribuisce come se fosse legittima.

```
> rvc commit -project=simulazione -tag=Documentazione -note=MoltoImportante

packing  simulazione.0Q6R82GADL...+Documentazione.zip  \  copying  simulazio-
ne.0Q6R82GADL.sig to repo\ ...
```

```
> rvc integrity -signers="C:\Users\stemic\stage\allowed_signers"
```

```
Verifica integrita repository: tutti i progetti
[WARN] 0Q6R82GADL hash:OK catena:OK firma:ASSENTE (?)
[WARN] 0Q6R832YWQ hash:OK catena:OK firma:ASSENTE (?)
[WARN] 0Q6R83HVQD hash:OK catena:OK firma:ASSENTE (?)
[WARN] 0Q6R83ULIB hash:OK catena:OK firma:ASSENTE (?)
[WARN] 0Q6R844SKJ hash:OK catena:OK firma:ASSENTE (?)
Risultato: 0/5 commit con problemi.
Risultato: 5/5 commit con warning.
```

Risultato. Il verificatore non rileva errori critici — la catena è strutturalmente valida. La *repository_G* fasulla produce solo warning per la firma assente, identici a quelli di qualsiasi progetto senza firme *SSH_G*.

Impatto: Alto. Un attaccante può distribuire codice arbitrario in una *repository_G* che supera tutti i controlli strutturali del verificatore. Senza una radice di fiducia verificabile non esiste nessun meccanismo automatico per distinguere questa *repository_G* da una legittima. Questa tecnica dimostra direttamente l'assenza di RS05 e la necessità del progetto `__rvc__root` proposto nel modello.

Requisiti violati: RS05 — radice di fiducia verificabile autonomamente.

5.2.11 Sintesi dello scenario 1

Tecnica	Descrizione	Verificatore	Motore	Impatto
T1	Commit senza firma	WARN	No	Alto
T2	Identità falsa	WARN	No	Alto
T3	Modifica ZIP esistente	ERR	No	Basso
T4	Sostituzione ZIP e .sig	WARN+ERR	No	Alto
T5	Inserimento in mezzo	WARN+ERR	No	Alto
T6	Replay commit legittimo	ERR	No	Basso
T7	Modifica .sig senza ZIP	ERR	No	Nulla
T8	Troncamento catena	ERR	No	Nulla
T9	cumulativeHash falsificato	ERR	No	Basso
T10	<i>Repository</i> fasulla	WARN	No	Alto

Sintesi dei risultati — Scenario 1.

Il risultato più significativo dello scenario 1 è la distinzione tra due categorie di tecniche. Le tecniche T3, T6, T7, T8 e T9 producono errori critici rilevabili dal verificatore — la catena degli *hash_G* e la firma *SSH_G* forniscono già protezione efficace, con impatto nullo o basso. Le tecniche T1, T2, T4, T5 e T10 producono solo warning — l'attaccante può agire senza produrre errori critici, con impatto alto in tutti i casi. Il denominatore comune è l'assenza di firma *SSH_G* obbligatoria. T4 e T5 meritano attenzione particolare: con ricalcolo in cascata degli *hash_G* è possibile riscrivere la storia di un progetto producendo solo warning, rendendo l'attacco praticamente invisibile a chi non esegue il verificatore in modo sistematico.

5.3 Scenario 2 — Dipendente con chiave SSH valida ma non autorizzato

Il secondo scenario simula un dipendente dell'organizzazione che possiede una chiave *SSH_G* valida e sa produrre commit firmati correttamente, ma non è autorizzato a operare su uno o più progetti specifici. A differenza dello scenario 1, i commit prodotti sono

crittograficamente validi — la firma è presente e corretta. Il verificatore deve quindi distinguere tra «firma valida» e «firmatario autorizzato», due proprietà che nella versione iniziale non sono entrambe verificate preventivamente dal motore.

5.3.1 T1 — Commit firmato da chiave non autorizzata

Il dipendente firma un commit con la propria chiave *SSH_G* valida su un progetto a cui non dovrebbe avere accesso. Il motore non verifica se la chiave del firmatario sia presente in una lista di autorizzati — nella versione iniziale questa lista non esiste.

```
> rvc commit -project=simulazione -tag=attaccante -author=Luigi -note=Questo-CommitFattoDaLuigi
```

```
Reading manifest, path:.\
packing simulazione.0Q6RCTTRWQ.0Q6PHWPMPQ.{Luigi}+attaccante.zip
copying simulazione.0Q6RCTTRWQ.0Q6PHWPMPQ.{Luigi}+attaccante.zip to repo\ ...
copying simulazione.0Q6RCTTRWQ.sig to repo\ ...
tot. exec time: 0.41 sec
```

```
> rvc integrity -signers="C:\Users\stemic\stage\allowed_signers"
```

```
[OK] 0Q6PHT7QCI hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHUA0SV hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHV1YTU hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHW0IJW hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHWPMPQ hash:OK catena:OK firma:OK (Michele)
[ERR] 0Q6RCTTRWQ hash:OK catena:OK firma:FALLITA (Luigi)
      ERRORE: Firma Ssh non valida
Risultato: 1/6 commit con problemi.
Risultato: 0/6 commit con warning.
```

Risultato. Il motore accetta il commit senza nessun avviso e le operazioni successive procedono normalmente. Il verificatore rileva il problema segnalando `[ERR] firma:FALLITA` — la chiave di Luigi non è presente nel file `allowed_signers`. L'anomalia è rilevabile solo tramite verifica esplicita.

Impatto: Alto. Un dipendente non autorizzato può produrre commit su qualsiasi progetto senza che il motore lo impedisca. Il commit è crittograficamente firmato e appare nella history come un commit normale — la differenza rispetto a uno legittimo emerge solo dal verificatore. In assenza di verifiche periodiche, l'accesso non autorizzato può passare inosservato a lungo.

Requisiti violati: RS08, RS09 — gerarchia di fiducia e permessi configurabili per progetto.

5.3.2 T2 — Commit firmato con author dichiarato diverso dal firmatario

Il dipendente firma il commit con la propria chiave *SSH_C* ma dichiara nel campo author il nome di un collega autorizzato. Il motore accetta il commit perché la firma è crittograficamente valida. Il verificatore rileva la discrepanza tra l'autore dichiarato e la chiave effettivamente usata per la firma.

```
> rvc commit -project=simulazione -tag=attaccante -author=Michele -note=Questo-CommitFattoDaLuigiSottoNomeDiMichele
```

```
Reading manifest, path:.\
packing simulazione.0Q6RJRDEU.0Q6PHWPMPQ.{Michele}+attaccante.zip
copying simulazione.0Q6RJRDEU.0Q6PHWPMPQ.{Michele}+attaccante.zip to repo\ ...
copying simulazione.0Q6RJRDEU.sig to repo\ ...
tot. exec time: 0.50 sec
```

```
> rvc integrity -signers="C:\Users\stemic\stage\allowed_signers"
```

```
[OK] 0Q6PHT7QCI hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHUA0SV hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHV1YTU hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHW0IJW hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHWPMPQ hash:OK catena:OK firma:OK (Michele)
[ERR] 0Q6RJRDEU hash:OK catena:OK firma:FALLITA (Michele)
      ERRORE: Firma Ssh non valida
Risultato: 1/6 commit con problemi.
Risultato: 0/6 commit con warning.
```

Risultato. Il motore non rileva nessuna discrepanza. Il verificatore segnala `[ERR] firma:FALLITA` perché la chiave usata per firmare appartiene a Luigi ma il campo author dichiara Michele — il confronto tra autore dichiarato e chiave-pubblica nel file `allowed_signers` fallisce.

Impatto: Alto. Il rischio principale esiste nei contesti dove la verifica delle firme non è eseguita sistematicamente ma basta la presenza di una qualsiasi firma. In quel caso il commit appare firmato da un autore autorizzato — Michele — e passa inosservato. Solo il verificatore con il file `allowed_signers` corretto rivela che la chiave usata non appartiene all'autore dichiarato.

Requisiti violati: RS08, RS09 — autenticità e autorizzazione.

5.3.3 T3 — Volume di commit non autorizzati senza rilevamento operativo

Il dipendente produce una serie di commit firmati con la propria chiave su un progetto non autorizzato nel corso del tempo. L'obiettivo è dimostrare che il motore non emette nessun segnale operativo durante le operazioni normali, indipendentemente dal numero di commit non autorizzati presenti.

```
> rvc commit -project=simulazione -tag=attaccante -author=Luigi -note=Questo-CommitFattoDaLuigi
```

```
copying simulazione.0Q6RLK7XSL.sig to repo\ ... tot. exec time: 0.47 sec
```

```
> rvc commit -project=simulazione -tag=attaccante -author=Luigi -note=Questo-CommitFattoDaLuigi
```

```
copying simulazione.0Q6RLK9DFH.sig to repo\ ... tot. exec time: 0.41 sec
```

```
> rvc commit -project=simulazione -tag=attaccante -author=Luigi -note=Questo-CommitFattoDaLuigi
```

```
copying simulazione.0Q6RLKAHPC.sig to repo\ ... tot. exec time: 0.42 sec
```

```
> rvc commit -project=simulazione -tag=attaccante -author=Luigi -note=Questo-CommitFattoDaLuigi
```

```
copying simulazione.0Q6RLKCNCY.sig to repo\ ... tot. exec time: 0.44 sec
```

```
> rvc integrity -signers="C:\Users\stemic\stage\allowed_signers"
```

```
[OK] 0Q6PHT7QCI hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHUAOSV hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHV1YTU hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHW0IJW hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHWPMPQ hash:OK catena:OK firma:OK (Michele)
[ERR] 0Q6RLK7XSL hash:OK catena:OK firma:FALLITA (Luigi)
      ERRERE: Firma Ssh non valida
[ERR] 0Q6RLK9DFH hash:OK catena:OK firma:FALLITA (Luigi)
      ERRERE: Firma Ssh non valida
[ERR] 0Q6RLKAHPC hash:OK catena:OK firma:FALLITA (Luigi)
      ERRERE: Firma Ssh non valida
[ERR] 0Q6RLKCNCY hash:OK catena:OK firma:FALLITA (Luigi)
      ERRERE: Firma Ssh non valida
```

Risultato: 4/9 commit con problemi.
Risultato: 0/9 commit con warning.

Risultato. Il motore esegue tutti i commit e tutte le operazioni di history senza segnalare nessuna anomalia. Solo il verificatore, eseguito esplicitamente alla fine, rileva tutti i commit non autorizzati in una singola analisi.

Impatto: Alto. L'assenza di RS08 e RS09 non produce nessun segnale operativo visibile — il sistema non avvisa mai autonomamente che qualcosa non va. La rilevazione dipende interamente dall'esecuzione periodica e sistematica del verificatore. In un contesto operativo reale dove il verificatore non viene eseguito ad ogni commit, un dipendente non autorizzato può operare indisturbato per un periodo prolungato, accumulando modifiche non autorizzate che risultano crittograficamente valide.

Requisiti violati: RS08, RS09 — gerarchia di fiducia e permessi configurabili per progetto.

5.3.4 Sintesi dello scenario 2

Tecnica	Descrizione	Verificatore	Motore	Impatto
T1	Commit da chiave non autorizzata	ERR	No	Alto
T2	Author dichiarato diverso dal firmatario	ERR	No	Alto
T3	Commit multipli senza segnale operativo	ERR	No	Alto

Sintesi dei risultati — Scenario 2.

A differenza dello scenario 1, tutte le tecniche dello scenario 2 producono errori critici nel verificatore — la firma *SSH_G* è presente e il verificatore può confrontarla con il file `allowed_signers`. Tuttavia questo non riduce la gravità: il motore non blocca mai nessuno di questi commit preventivamente. La protezione esiste solo a posteriori, tramite verifica esplicita. Il risultato chiave di questo scenario è che nella versione iniziale di *RVC_G* non esiste distinzione tra un dipendente autorizzato e uno non autorizzato — chiunque possieda una chiave *SSH_G* valida può operare su qualsiasi progetto senza restrizioni operative.

5.4 Scenario 3 — Chiave privata di un dipendente compromessa

Il terzo scenario simula la compromissione della chiave-privata *SSH_G* di un dipendente legittimo. L'attaccante può produrre commit firmati crittograficamente identici a quelli del dipendente reale — la chiave è quella corretta e il firmatario è presente nel file `allowed_signers`. Questo è lo scenario più insidioso: la firma è valida, il firmatario è autorizzato, e non esiste nessun meccanismo automatico per distinguere un commit fraudolento da uno legittimo.

Una nota sul verificatore: il verificatore attuale usa un file `allowed_signers` esterno e statico. Nel modello proposto ogni commit porta il proprio `allowed_Dipendenti` interno al momento della firma — i commit prodotti prima della revoca rimarrebbero validi perché la lista includeva la chiave al momento della firma. Per simulare entrambi i comportamenti vengono usati due file separati: `allowed_signers_con_chiave` che include la chiave compromessa, e `allowed_signers_senza_chiave` che non la include.

5.4.1 T1 — Commit fraudolento indistinguibile dal legittimo

L'attaccante produce un commit firmato con la chiave rubata del dipendente legittimo. Il verificatore eseguito con `allowed_signers_con_chiave` segnala il commit come completamente valido.

```
> rvc commit -project=simulazione -tag=produzione -author=Michele -note=Commit-FattoDaQualcunaltro
```

```
Reading manifest, path:.\
packing simulazione.0Q6WRGCMHR.0Q6PHWPMPQ.{Michele}+produzione.zip
copying simulazione.0Q6WRGCMHR.0Q6PHWPMPQ.{Michele}+produzione.zip to repo\ ...
copying simulazione.0Q6WRGCMHR.sig to repo\ ...
tot. exec time: 0.33 sec
```

```
> rvc integrity -signers="C:\Users\stemic\stage\allowed_signers_con_chiave"
```

```
[OK] 0Q6PHT7QCI hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHUAOSV hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHV1YTU hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHW0IJW hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHWPMPQ hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6WRGCMHR hash:OK catena:OK firma:OK (Michele)
Risultato: 0/6 commit con problemi.
Risultato: 0/6 commit con warning.
```

Risultato. Il commit è accettato dal motore e risulta completamente valido al verificatore. Non esiste nessun indicatore automatico che permetta di distinguerlo da un commit legittimo prodotto dal dipendente reale.

Impatto: Alto. L'attaccante ha piena capacità di committare codice arbitrario che supera tutti i controlli automatici. Non è rilevabile né dal motore né dal verificatore — l'unica contromisura possibile è un controllo manuale del contenuto committato e una rotazione periodica delle chiavi. Questa è una limitazione strutturale di qualsiasi sistema basato su crittografia-asimmetrica: la chiave-privata è l'unico indicatore di identità e la sua compromissione annulla tutte le garanzie crittografiche.

Requisiti coinvolti: RS10 — revoca efficace delle identità.

5.4.2 T2 — Commit fraudolenti durante la finestra di rischio

L'attaccante produce più commit nell'intervallo tra la compromissione della chiave e la sua revoca, dimostrando concretamente la finestra di rischio descritta nel modello.

```
> rvc commit -project=simulazione -tag=produzione -author=Michele -note=Commit-FattoDaQualcunaltro
```

```
copying simulazione.0Q6WRGCMHR.sig to repo\ ... tot. exec time: 0.33 sec
```

```
> rvc commit -project=simulazione -tag=produzione -author=Michele -note=Commit-FattoDaQualcunaltro
```

```
copying simulazione.0Q6WTK5GET.sig to repo\ ... tot. exec time: 0.39 sec
```

```
> rvc commit -project=simulazione -tag=produzione -author=Michele -note=Commit-FattoDaQualcunaltro
```

```
copying simulazione.0Q6WTKCDJL.sig to repo\ ... tot. exec time: 0.31 sec
```

```
> rvc integrity -signers="C:\Users\stemic\stage\allowed_signers_con_chiave"
```

```
[OK] 0Q6PHT7QCI hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHUA0SV hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHV1YTU hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHW0IJW hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHWPMPQ hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6WRGCMHR hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6WTK5GET hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6WTKCDJL hash:OK catena:OK firma:OK (Michele)
```



```
Risultato: 0/8 commit con problemi.  
Risultato: 0/8 commit con warning.
```

Dopo la simulazione della revoca, il verificatore viene rieseguito con `allowed_signers_senza_chiave`:

```
> rvc integrity -signers="C:\Users\stemic\stage\allowed_signers_senza_chiave"  
  
[ERR] 0Q6PHT7QCI hash:OK catena:OK firma:FALLITA (Michele)  
      ERRORRE: Firma Ssh non valida  
[ERR] 0Q6PHUA0SV hash:OK catena:OK firma:FALLITA (Michele)  
      ERRORRE: Firma Ssh non valida  
[ERR] 0Q6PHV1YTU hash:OK catena:OK firma:FALLITA (Michele)  
      ERRORRE: Firma Ssh non valida  
[ERR] 0Q6PHW0IJW hash:OK catena:OK firma:FALLITA (Michele)  
      ERRORRE: Firma Ssh non valida  
[ERR] 0Q6PHWPMPQ hash:OK catena:OK firma:FALLITA (Michele)  
      ERRORRE: Firma Ssh non valida  
[ERR] 0Q6WRGCMHR hash:OK catena:OK firma:FALLITA (Michele)  
      ERRORRE: Firma Ssh non valida  
[ERR] 0Q6WTK5GET hash:OK catena:OK firma:FALLITA (Michele)  
      ERRORRE: Firma Ssh non valida  
[ERR] 0Q6WTKCDJL hash:OK catena:OK firma:FALLITA (Michele)  
      ERRORRE: Firma Ssh non valida  
Risultato: 8/8 commit con problemi.  
Risultato: 0/8 commit con warning.
```

Risultato. Con la chiave presente nell'`allowed_signers` tutti i commit — legittimi e fraudolenti — risultano validi. Dopo la rimozione della chiave, tutti risultano in errore indistintamente. Questo comportamento differisce dal modello proposto: con `allowed_Dipendenti` interno a ogni commit, i commit prodotti prima della revoca rimarrebbero validi perché la lista al momento della firma includeva la chiave. Il verificatore attuale non distingue tra «chiave valida al momento della firma» e «chiave valida ora».

Impatto: Alto. Durante la finestra di rischio l'attaccante può produrre qualsiasi numero di commit fraudolenti che risultano indistinguibili da quelli legittimi. La dimensione di questa finestra dipende interamente dalla rapidità con cui la compromissione viene identificata e comunicata.

Requisiti coinvolti: RS10 — revoca efficace delle identità.

5.4.3 T3 — Propagazione nella catena dopo la revoca

Con la *repository* prodotta in T2, il verificatore viene eseguito con entrambi i file `allowed_signers` per confrontare i comportamenti e identificare i commit sospetti.

```
Con allowed_signers_con_chiave:  
[OK] tutti i commit – legittimi e fraudolenti risultano validi  
  
Con allowed_signers_senza_chiave:  
[ERR] tutti i commit – legittimi e fraudolenti risultano in errore
```

Risultato. Con il verificatore attuale non è possibile distinguere automaticamente i commit fraudolenti da quelli legittimi dopo la revoca — entrambi cambiano stato in blocco al cambio del file `allowed_signers`. La catena degli *hash* risulta intatta in entrambi i casi — i commit fraudolenti non hanno alterato la struttura crittografica della storia.

Impatto: Alto. L'identificazione dei commit fraudolenti richiede un'analisi manuale della history nel periodo sospetto. Nel modello proposto con `allowed_Dipendenti` interno questa distinzione sarebbe automatica — i commit legittimi prodotti prima della revoca rimarrebbero validi mentre quelli fraudolenti prodotti con la stessa chiave sarebbero distinguibili per contenuto. Questa limitazione è la motivazione principale della scelta architetturale dell'`allowed_Dipendenti` versionato nel modello proposto.

Nota sulla finestra di rischio. È importante sottolineare che la revoca minimizza ma **non elimina** la finestra di rischio durante l'intervallo fra la compromissione della chiave e il momento in cui il commit di revoca viene ricevuto e processato. Durante questa finestra, l'attaccante rimane crittograficamente indistinguibile dal proprietario legittimo della chiave — nessun meccanismo automatico può identificare i commit fraudolenti prodotti in questo intervallo. La revoca «dal commit successivo» significa che il sistema operativo riceve il comando di revoca e lo implementa immediatamente, non che il sistema possa retroattivamente distinguere i commit fraudolenti già creati. Questa è una limitazione strutturale di qualsiasi sistema basato su crittografia-asimmetrica e revoca distribuita — la finestra può essere ridotta minimizzando i tempi di sincronizzazione e rilevamento della compromissione, ma non può essere azzerata.

Requisiti coinvolti: RS10 — revoca efficace delle identità.

5.4.4 T4 — Recovery dopo compromissione della chiave

Questa tecnica non documenta un attacco ma la procedura di risposta — come il sistema gestisce il ripristino delle garanzie di sicurezza dopo la compromissione.

```
> ssh-keygen -t ed25519 -f chiave_nuova

Generating public/private ed25519 key pair.
```

Dopo aver aggiornato il file `allowed_signers` rimuovendo la chiave compromessa e aggiungendo quella nuova, il verificatore mostra:

```
> rvc integrity -signers="C:\Users\stemic\stage\allowed_signers"

[ERR] 0Q6PHT7QCI hash:OK catena:OK firma:FALLITA (Michele)
[ERR] 0Q6PHUA0SV hash:OK catena:OK firma:FALLITA (Michele)
[ERR] 0Q6PHV1YTU hash:OK catena:OK firma:FALLITA (Michele)
[ERR] 0Q6PHW0IJW hash:OK catena:OK firma:FALLITA (Michele)
[ERR] 0Q6PHWPMPQ hash:OK catena:OK firma:FALLITA (Michele)
[ERR] 0Q6WWOWTGV hash:OK catena:OK firma:FALLITA (Michele)
[ERR] 0Q6WWP3QDM hash:OK catena:OK firma:FALLITA (Michele)
[ERR] 0Q6WWP4TNH hash:OK catena:OK firma:FALLITA (Michele)
```

Dopo aver prodotto un nuovo commit firmato con la nuova chiave:

```
> rvc integrity -signers="C:\Users\stemic\stage\allowed_signers"

[ERR] 0Q6PHT7QCI hash:OK catena:OK firma:FALLITA (Michele)
[ERR] 0Q6PHUA0SV hash:OK catena:OK firma:FALLITA (Michele)
[ERR] 0Q6PHV1YTU hash:OK catena:OK firma:FALLITA (Michele)
[ERR] 0Q6PHW0IJW hash:OK catena:OK firma:FALLITA (Michele)
[ERR] 0Q6PHWPMPQ hash:OK catena:OK firma:FALLITA (Michele)
[ERR] 0Q6WWOWTGV hash:OK catena:OK firma:FALLITA (Michele)
[ERR] 0Q6WWP3QDM hash:OK catena:OK firma:FALLITA (Michele)
[ERR] 0Q6WWP4TNH hash:OK catena:OK firma:FALLITA (Michele)
[OK] 0Q6WWTIPTA hash:OK catena:OK firma:OK (Michele)
Risultato: 8/9 commit con problemi.
Risultato: 0/9 commit con warning.
```

Risultato. La procedura di recovery è operativa — il nuovo commit firmato con la nuova chiave risulta immediatamente valido. I commit precedenti risultano tutti in errore indistintamente, senza distinzione tra legittimi e fraudolenti — limitazione già documentata in T3.

Impatto. La recovery non richiede interventi straordinari — è sufficiente aggiornare il file `allowed_signers`. La finestra di rischio si chiude dal commit successivo alla revoca. La limitazione principale è che il verificatore attuale non distingue i commit legittimi prodotti prima della revoca da quelli fraudolenti — nel modello proposto con `allowed_Dipendenti` interno questa distinzione sarebbe automatica.

Obiettivo stage soddisfatto: D03 — studio e implementazione delle tecniche di recovery delle credenziali.

5.4.5 Sintesi dello scenario 3

Tecnica	Descrizione	Verificatore	Motore	Impatto
T1	Commit fraudolento indistinguibile	OK (con chiave)	No	Alto
T2	Finestra di rischio	OK→ERR	No	Alto
T3	Propagazione dopo revoca	ERR indistinto	No	Alto
T4	Recovery chiave compromessa	OK (nuova)	No	-

Sintesi dei risultati — Scenario 3.

Lo scenario 3 è il più critico dell'intera simulazione — nessuna delle tecniche è rilevabile automaticamente durante la finestra di rischio. La compromissione di una chiave-privata azzerava tutte le garanzie crittografiche perché la chiave è l'unico indicatore di identità del sistema. Le uniche mitigazioni sono procedurali: rotazione periodica delle chiavi e revoca immediata alla scoperta della compromissione. Il modello proposto con `allowed_Dipendenti` interno a ogni commit migliora la situazione post-revoca permettendo di distinguere automaticamente i commit legittimi da quelli fraudolenti — ma non elimina la finestra di rischio, che rimane una limitazione strutturale di qualsiasi sistema basato su crittografia-asimmetrica.

5.5 Scenario 4 — Chiave operativa dell'amministratore compromessa

Il quarto scenario simula la compromissione della chiave-privata *SSH_G* operativa dell'amministratore — il soggetto con i poteri più ampi sull'intera *repository_G*. Nella versione iniziale di *RVC_G* questo scenario non è distinguibile tecnicamente dagli scenari precedenti — non esistono controlli di permessi differenziati per ruolo e la chiave dell'amministratore non ha nessun trattamento speciale da parte del motore.

Il valore di questo scenario è quindi principalmente analitico: dimostrare che nella versione iniziale la compromissione dell'amministratore non produce nessun segnale aggiuntivo

rispetto alla compromissione di qualsiasi altro utente, e analizzare perché nel modello proposto questo scenario sarebbe invece il più critico in assoluto.

5.5.1 T1 — Compromissione totale e silenziosa

L'attaccante usa la chiave operativa dell'amministratore per produrre commit su più progetti della *repository*. Il motore accetta tutto senza distinzione. Il verificatore segnala tutti i commit come validi perché la chiave è presente nel file `allowed_signers`.

```
> rvc commit -project=simulazione -tag=produzione -author=Michele -note=Commit-FattoDaQualcunAltro
```

```
Reading manifest, path:.\
packing simulazione.0Q6WW0WTGV.0Q6PHWPMPQ.{Michele}+produzione.zip
copying simulazione.0Q6WW0WTGV.sig to repo\ ...
tot. exec time: 0.33 sec
```

```
> rvc commit -project=simulazione -tag=produzione -author=Michele -note=Commit-FattoDaQualcunAltro
```

```
copying simulazione.0Q6WWP3QDM.sig to repo\ ... tot. exec time: 0.31 sec
```

```
> rvc commit -project=simulazione -tag=produzione -author=Michele -note=Commit-FattoDaQualcunAltro
```

```
copying simulazione.0Q6WWP4TNH.sig to repo\ ... tot. exec time: 0.29 sec
```

```
> rvc integrity -signers="C:\Users\stemic\stage\allowed_signers"
```

```
[OK] 0Q6PHT7QCI hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHUA0SV hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHV1YTU hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHW0IJW hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6PHWPMPQ hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6WW0WTGV hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6WWP3QDM hash:OK catena:OK firma:OK (Michele)
[OK] 0Q6WWP4TNH hash:OK catena:OK firma:OK (Michele)
Risultato: 0/8 commit con problemi.
Risultato: 0/8 commit con warning.
```

Risultato. Il motore accetta tutti i commit senza avvisi. Il verificatore li segnala tutti come validi — la chiave è quella corretta e il firmatario è nel file `allowed_signers`. Il

risultato è tecnicamente identico allo scenario 3: la versione iniziale non distingue tra ruoli.

Impatto nella versione iniziale: Alto, identico allo scenario 3. Non è rilevabile né dal motore né dal verificatore in modo automatico. L'unica contromisura è procedurale.

Impatto nel modello proposto: Critico. Nel modello proposto la compromissione della chiave operativa dell'amministratore è lo scenario più grave in assoluto perché l'attaccante acquisisce poteri esclusivi che nessun altro soggetto possiede:

- Può modificare `allowed_Responsabili` in `_rvc_root` — aggiungendo identità false o rimuovendo responsabili legittimi
- Può produrre commit amministrativi su qualsiasi progetto senza nessun vincolo di appartenenza
- Può alterare le policy di sicurezza di qualsiasi progetto modificando il file `.rvc_policy`
- Può revocare l'accesso a dipendenti legittimi su qualsiasi progetto

Queste azioni compromettono non solo la *repository* attuale ma l'intera struttura di fiducia — incluse le autorizzazioni future di tutti i soggetti. A differenza della compromissione di un dipendente, che ha effetti limitati al proprio progetto, la compromissione dell'amministratore può invalidare l'intera catena di fiducia della *repository*.

Contromisure architetturali nel modello proposto. Il modello affronta questo scenario con due meccanismi complementari. Il primo è la separazione tra chiave master e chiave operativa: la chiave master è conservata offline su un dispositivo *air-gapped* e non viene mai usata nelle operazioni ordinarie. La compromissione della chiave operativa non comporta la perdita del controllo della *repository* — la chiave master può revocarla e nominarne una nuova senza perdere la catena di fiducia. Il secondo meccanismo è la revoca con chiave master: il commit di revoca firmato con la chiave master è riconosciuto dal motore come autoritativo anche se la chiave master non è in `allowed_Dipendenti` — è il meccanismo eccezionale descritto nella sezione sulla compromissione della chiave dell'amministratore nel Sezione 4.

Requisiti coinvolti: RS05, RS08, RS09, RS10, RS11 — radice di fiducia, gerarchia, permessi, revoca e successione.

Obiettivo stage soddisfatto: D01 — simulazione di attacchi con chiave del capo progetto compromessa.

5.5.2 Sintesi dello scenario 4

Tecnica	Descrizione	Verificatore	Motore	Impatto
T1	Compromissione totale e silenziosa	OK	No	Critico

Sintesi dei risultati — Scenario 4.

Lo scenario 4 conferma che nella versione iniziale di *RVC_G* non esiste nessuna distinzione tecnica tra la compromissione dell'amministratore e quella di qualsiasi altro utente — il motore tratta tutte le chiavi allo stesso modo. Il valore di questo scenario è quindi prospettico: dimostrare perché il modello proposto introduce una gerarchia di fiducia asimmetrica con la chiave master offline come ultima ancora di sicurezza. Senza questa separazione, la compromissione dell'amministratore comporterebbe la perdita irreversibile del controllo dell'intera *repository_G*.

5.6 Sintesi complessiva

I quattro scenari documentano un quadro coerente delle vulnerabilità della versione iniziale di *RVC_G*. Il sistema dispone di basi crittografiche solide — la catena degli *hash_G* e la firma *SSH_G* sono strumenti efficaci quando presenti — ma manca dei meccanismi organizzativi che trasformano queste garanzie tecniche in un modello di sicurezza praticabile in contesti multi-utente.

Scenario	Contesto	Rilevabile	Impatto massimo
S1	Esterno senza credenziali	Parzialmente	Alto
S2	Dipendente con chiave non autorizzata	Solo verificatore	Alto
S3	Chiave dipendente compromessa	Non rilevabile	Alto
S4	Chiave amministratore compromessa	Non rilevabile	Critico

Sintesi complessiva degli scenari di attacco.

5.6.1 Osservazione 1 — Il motore non costituisce una linea di difesa

In nessuno dei quattro scenari il motore ha bloccato preventivamente un'azione non autorizzata. Ogni tecnica di attacco è stata eseguita con successo dal punto di vista operativo — i commit sono stati accettati, inseriti nella catena e resi disponibili nella history senza nessun avviso. Il verificatore ha rilevato le anomalie solo su esecuzione esplicita e a posteriori.

Questa osservazione ha un'implicazione diretta sul modello di sicurezza operativa: nella versione iniziale di *RVC_G* la sicurezza della *repository_G* dipende interamente dalla disciplina procedurale di chi la gestisce — in particolare dalla frequenza e dalla sistematicità con cui il verificatore viene eseguito. In assenza di questa disciplina, qualsiasi delle tecniche documentate può operare indisturbata per un periodo arbitrariamente lungo.

5.6.2 Osservazione 2 — La firma SSH è una protezione necessaria ma non sufficiente

La firma *SSH_G* è il meccanismo di sicurezza più maturo della versione iniziale. T7 dimostra che qualsiasi modifica al file .sig di un commit firmato produce un errore critico immediatamente rilevabile — la firma copre il file nella sua interezza e non lascia spazio a modifiche parziali. T3 dimostra analogamente che la catena degli *hash_G* rileva qualsiasi alterazione del contenuto ZIP senza ricalcolo.

Tuttavia la firma è opzionale nella versione iniziale, e questa opzionalità vanifica le protezioni che offre. T4 e T5 dimostrano che in assenza di firma è possibile riscrivere la storia di un progetto — modificando ZIP e ricalcolando gli *hash_G* in cascata — riducendo l'output del verificatore a soli warning, senza errori critici. La firma *SSH_G* obbligatoria è quindi una condizione necessaria per la sicurezza del sistema, non una funzionalità opzionale. Questa osservazione corrisponde direttamente al requisito RS06 identificato nel modello e classificato come obbligatorio.

5.6.3 Osservazione 3 — L'assenza di controllo delle identità è la vulnerabilità strutturale principale

Gli scenari 2, 3 e 4 convergono sulla stessa vulnerabilità di fondo: nella versione iniziale non esiste nessuna lista di autorizzati per progetto. Qualsiasi soggetto in possesso di una chiave *SSH_G* — autorizzata o meno, legittima o rubata — può produrre commit validi su qualsiasi progetto. Il verificatore può rilevare le anomalie confrontando le firme con un file *allowed_signers* esterno, ma questo controllo è manuale e non integrato nel flusso operativo del motore.

La conseguenza più grave di questa assenza è documentata nello scenario 3: la compromissione di una chiave-privata non produce nessun segnale automatico rilevabile — il verificatore con la chiave ancora presente nell'`allowed_signers` segnala tutti i commit come validi, inclusi quelli fraudolenti. Solo la rimozione esplicita della chiave dall'`allowed_signers` permette di identificare le anomalie, ma a quel punto non è possibile distinguere automaticamente i commit legittimi prodotti prima della revoca da quelli fraudolenti — entrambi risultano in errore indistintamente. Questa limitazione è la motivazione principale della scelta architetturale dell'`allowed_Dipendenti` versionato proposta nel modello, che risolve il problema mantenendo all'interno di ogni commit la lista degli autorizzati valida al momento della firma.

5.6.4 Osservazione 4 — La gravità cresce con il livello di privilegio dell'attaccante

I quattro scenari sono stati costruiti in ordine crescente di privilegio dell'attaccante — da nessuna credenziale fino alla chiave dell'amministratore. L'analisi mostra che la gravità degli attacchi segue questa progressione in modo non lineare.

Negli scenari 1 e 2 la rilevabilità è almeno parziale — le firme presenti permettono al verificatore di identificare le anomalie, anche se solo a posteriori. Nello scenario 3 la rilevabilità scompare durante la finestra di rischio — un attaccante con la chiave di un dipendente legittimo è indistinguibile dal dipendente stesso per qualsiasi strumento automatico. Nello scenario 4 la gravità diventa critica non per le azioni immediate — identiche allo scenario 3 nella versione iniziale — ma per le implicazioni nel modello proposto: la compromissione dell'amministratore non riguarda un singolo progetto ma l'intera struttura di fiducia della *repository*, inclusa la capacità di nominare e revocare responsabili, modificare le policy di sicurezza e accedere a qualsiasi progetto cifrato.

Questa progressione non lineare della gravità è esattamente la motivazione per cui il modello proposto introduce la separazione tra chiave master e chiave operativa: limitare le conseguenze della compromissione della chiave operativa a uno scenario recuperabile, preservando attraverso la chiave master la possibilità di ristabilire la catena di fiducia senza perdere il controllo della *repository*.

5.6.5 Corrispondenza con i requisiti del modello

I risultati degli scenari confermano empiricamente la classificazione dei requisiti stabilita nella Sezione 4. Ogni vulnerabilità documentata corrisponde a uno o più requisiti assenti nella versione iniziale:

Vulnerabilità osservata	Scenario	Requisiti
Firma opzionale — storia riscrivibile senza errori critici	S1 T4, T5	RS06
Nessuna radice di fiducia — <i>repository_G</i> fasulla indistinguibile	S1 T10	RS05
Nessuna lista autorizzati — accesso non controllato per progetto	S2	RS08, RS09
Nessuna revoca operativa — finestra di rischio non gestita	S3	RS10
Nessuna gerarchia — compromissione admin identica a dipendente	S4	RS08, RS11
Nessuna separazione master/operativa — perdita controllo totale	S4	RS05, RS11

Corrispondenza tra vulnerabilità osservate e requisiti del modello.

Tutti i requisiti classificati come assenti nell'analisi del divario del Sezione 4 trovano una corrispondenza diretta in almeno una delle tecniche documentate in questo capitolo. I requisiti RS01 e RS02, classificati come parziali, sono confermati: la catena degli *hash_G* funziona correttamente quando i file non vengono sostituiti interamente, ma non costituisce una protezione sufficiente in assenza di firma obbligatoria. Questi risultati definiscono con precisione la priorità degli interventi descritti nel capitolo successivo.

Capitolo 6

Miglioramenti implementati

Il capitolo descrive le modifiche apportate al codice sorgente di [RVC_G](#) in risposta alle vulnerabilità e alle carenze individuate nei capitoli precedenti. Per ciascun intervento sono illustrati la motivazione, le scelte progettuali e gli estratti di codice [CPL_G](#) rilevanti.

6.1 Visione d'insieme

La Sezione 4 ha definito quindici requisiti di sicurezza distribuiti in cinque categorie — integrità, autenticità, gestione delle identità, sicurezza configurabile e gestione dei branch — e ne ha quantificato il divario rispetto alla versione iniziale di [RVC_G](#). La Sezione 5 ha verificato empiricamente le conseguenze di tali carenze attraverso quattro scenari di attacco progressivi. L'implementazione ha colmato sistematicamente il divario intervenendo sui moduli `ProjectImage.cpl`, `RvcEngine.cpl`, `FileManifest.cpl` e sul nuovo modulo `Init.cpl`.

Il punto architetturale comune a quasi tutti gli interventi è la classe `CommitValidation`, che rappresenta il contenuto serializzato del file `.sig` associato a ogni commit. Nella versione iniziale la struttura conteneva solo i campi necessari per la catena di hash; la sua estensione con i campi richiesti dal modello di sicurezza ha reso possibile implementare tutti gli altri controlli senza modificare né il formato dei file ZIP né il meccanismo di base del commit.

Le sezioni che seguono descrivono gli interventi nell'ordine delle cinque categorie di requisiti, il meccanismo di Redazione Trasparente e, infine, il lavoro di robustezza e correzione delle vulnerabilità di injection emerse dallo scenario S5.

6.2 Estensione della struttura `CommitValidation`

La classe `CommitValidation` è il nucleo del file `.sig`: ogni campo che il verificatore deve controllare senza aprire lo ZIP deve essere presente in questa struttura. La scelta di includere metadati di sicurezza direttamente nel `.sig` è necessaria perché un verificatore che riceve una [repository](#) da un canale non fidato non può presumere di riuscire ad aprire ogni ZIP — ad esempio perché cifrato al livello 4 — ma deve poter comunque accertare l'integrità e l'autenticità della catena.

La versione iniziale conteneva solo i campi strettamente necessari per la catena crittografica di base. Il modello di sicurezza ha richiesto tre gruppi di campi.

Il primo gruppo, già presente, costituisce la catena crittografica originale: hash dello ZIP corrente, hash dello ZIP precedente e hash cumulativo. Il secondo gruppo permette al verificatore di operare senza aprire lo ZIP: livello di sicurezza, stato del branch, chiavi autorizzate, riferimento all'ultimo commit di `_rvc_root` e nome del branch vengono estratti dallo ZIP al momento del commit e scritti direttamente nel `.sig`. Il terzo gruppo supporta la Redazione Trasparente (sezione 6.9) ed è compilato esclusivamente quando un commit viene redatto dalla chiave master.

```
class CommitValidation
  -- Gruppo 1: catena crittografica originale
  var str author, comment, fn
  var str id, prevId
  var str hash, prevHash, cumulativeHash
  var array[str] merged, mergedHash
  var str authorPublicKey

  -- Gruppo 2: campi del modello di sicurezza
  var str securityLevel    -- 0-4
  var str branchStatus    -- active / archived / compromised
  var str allowedSigners  -- chiavi autorizzate separate da ';'
  var str rvcRootId       -- ID ultimo commit di \_rvc_root
  var str branchName, mergeFrom, recipients

  -- Gruppo 3: Redazione Trasparente
  var bool redacted
  var str redactionZipHash, redactionAuthority
  var str redactionTimestamp, redactionLegalRef
  var str redactionContent, redactionSignature
  var str redactionCount, redactionOriginalSig
end
```

Campi di CommitValidation

La scelta di includere `allowedSigners` nel `.sig` — che è già firmato — garantisce che le chiavi autorizzate usate in fase di verifica siano esattamente quelle vigenti al momento del commit, e che non possano essere alterate successivamente. Questo risolve il problema della verifica storica: la revoca di un dipendente non invalida i commit prodotti prima della revoca, poiché le chiavi registrate nel loro `.sig` riflettono lo stato delle autorizzazioni vigente quando furono prodotti. Il campo `recipients`, analogo, serve alla cifratura al livello 4: elenca le chiavi SSH pubbliche dei destinatari autorizzati, consentendo al verificatore di controllarne la lista senza decifrare il file.

6.3 Integrità e ordine verificabile (RS01-RS04)

I requisiti RS01 e RS02 erano già soddisfatti nella versione iniziale mediante la catena di hash cumulativi. L'intervento in questa categoria ha riguardato RS03 (unicità degli identificativi content-dependent) e RS04 (documentazione delle limitazioni).

Requisito	Descrizione
RS01	Verificabilità hash di ogni commit
RS02	Catena hash verificabile, modifica invalida i successivi
RS03	Identificativi univoci, non manipolabili tramite timestamp
RS04	Limitazioni temporali documentate esplicitamente

Requisiti di integrità

RS03 è stato soddisfatto aggiungendo al nome del file ZIP un suffisso di otto caratteri derivato dall'hash SHA256 del contenuto. RS04 è stato soddisfatto documentando esplicitamente nel Sezione 4 i limiti strutturali della garanzia temporale assoluta, affidata al cumulativeHash e non al timestamp.

6.3.1 Identificativi content-dependent (RS03)

La versione iniziale utilizzava solo un timestamp codificato in *base36c* come identificativo del commit, con due problemi: collisioni tra commit generati nello stesso millisecondo e manipolabilità del contenuto senza modifica del nome file.

La soluzione adottata aggiunge al nome file un suffisso di otto caratteri derivato dall'hash SHA256 del contenuto del commit:

```
mioprogetto.0Q7BK6WQRB_D7000B46.0Q7BK1L0ZY_E168FAB3.{Michele}.zip
           ^timestamp ^hash8 ^prevId
```

Formato nome file ZIP

L'ID effettivo del commit è la coppia `timestamp_hash8`, ad esempio `0Q7BK6WQRB_D7000B46`. La generazione avviene in due fasi: creazione dello ZIP, poi calcolo dello SHA256 e rinomina del file includendo i primi otto caratteri dell'hash. Questo rende il nome *content-dependent*: qualsiasi sostituzione del contenuto modifica il nome e invalida la catena successiva.

```
cv.hash := crypt.SHA256(zipPath)
newId := timestamp + '_' + Left(cv.hash, 8)
os.Rename(zipPath, Strtran(zipPath, oldId, newId))
```

cpl

Generazione ID content-dependent

È importante precisare cosa questa modifica garantisce e cosa no. Il suffisso content-dependent rende la sostituzione silenziosa rilevabile meccanicamente, ma la garanzia dell'ordine temporale assoluto rimane affidata al cumulativeHash: un sistema con orologio alterato può produrre timestamp arbitrari.

6.4 Autenticità e non ripudio (RS05-RS07)

Requisito	Descrizione
RS05	Radice di fiducia verificabile autonomamente
RS06	Firma SSH obbligatoria per livello ≥ 1
RS07	Ogni commit referencia lo stato corrente di <code>__rvc__root</code>

Requisiti di autenticità

RS05 è stato soddisfatto introducendo il progetto speciale `__rvc__root`, il cui primo commit include la chiave pubblica master direttamente nel `.sig` tramite il campo `masterPubBootstrap`. RS06 è stato soddisfatto rendendo la firma SSH obbligatoria per tutti i commit su progetti con livello ≥ 1 , con namespace `-n file` che impedisce il riutilizzo della firma in altri contesti. RS07 è stato soddisfatto aggiungendo il campo `rvcRootId` in `CommitValidation`, che ancora crittograficamente ogni commit allo stato delle autorizzazioni vigente al momento della sua produzione.

6.4.1 Il progetto `__rvc__root` e la chiave master (RS05)

Nella versione iniziale non esisteva alcuna radice di fiducia verificabile autonomamente. Il modello introduce `__rvc__root`, un progetto speciale creato con `rvc init` che funge da ancora crittografica per l'intera *repository*.

L'inizializzazione richiede due chiavi distinte: la chiave pubblica master (`-master-pub=`) e la chiave privata master (`-master-key=`), più la chiave pubblica operativa dell'amministratore (`-op-key=`). La distinzione è intenzionale: la chiave master è desti-

nata a un supporto [air-gapped](#) e usata esclusivamente per operazioni straordinarie, mentre la chiave operativa è quella impiegata nei commit ordinari.

```
rvc init
  -master-pub=<path>      chiave pubblica master      [OBBLIGATORIO]
  -master-key=<path>      chiave privata master      [OBBLIGATORIO]
  [-op-key=<path>]        chiave pubblica operativa admin (singola)
  [-op-keys=<path>]       file multi-admin (formato allowed_signers)
  [-responsabili=<path>]  chiavi responsabili di progetto
  [-author=<nome>]        nome dell'amministratore
  [-dir=<dir>]            directory di lavoro
  [-repo=<path>]          path della repository
```

Sintassi rvc init

```
> rvc init -master-pub=C:chiavimaster.pub -master-key=C:chiavimaster_key
-op-key=C:UsersMichele.sshid_ed25519.pub
-responsabili=C:chiaviresponsabili.pub
-author=Michele -repo=C:stagerepo
analyzing repository C:stagerepo
...
packing _rvc_root.0Q7BJJFHEH.{Michele}.zip
copying _rvc_root.0Q7BJJFHEH.0Q7BJJFHEH_1EEBDB09.sig to C:stagerepo
...
Repository inizializzata. Progetto _rvc_root creato.
```

Il primo commit di `_rvc_root` contiene tre file speciali: `master.pub`, `allowed_Dipendenti` e `allowed_Responsabili`. Il campo `masterPubBootstrap` nel `.sig` contiene la chiave pubblica master in chiaro, rendendola disponibile al verificatore senza dipendenze da infrastrutture esterne. Qualsiasi verifica dell'intera catena di fiducia può partire da questa chiave, distribuibile fuori banda. Tutti i commit su `_rvc_root` devono essere firmati con la chiave master; il motore rifiuta qualsiasi commit che non soddisfi questo vincolo, indipendentemente dall'identità del richiedente.

6.4.2 Firma SSH obbligatoria nel flusso di commit (RS06)

Nella versione iniziale la firma SSH era opzionale. Il Sezione 5 ha dimostrato che questa opzionalità consente di riscrivere la storia di un progetto senza produrre anomalie nel verificatore.

Il modello richiede che la firma sia prodotta per tutti i commit su progetti con livello ≥ 1 e che la sua assenza o invalidità sia rilevata come anomalia. L'intervento ha integrato la firma direttamente nel flusso di commit tramite la procedura `SaveAndSignCommitSignature`, che invoca `ssh-keygen -Y sign` al termine della costru-

zione del .sig. Il namespace `-n file` è scelto per il suo significato nella specifica OpenSSH: le firme prodotte in tale contesto non possono essere riutilizzate per l'autenticazione SSH, prevenendo attacchi di riutilizzo.

```
proc SaveAndSignCommitSignature(str path, str project, CommitValidation cv,
bool sign)
  -- Ricerca della chiave privata
  keyPath := identityDir + cv.author
  if !file.Exist(keyPath) then keyPath := ReadConfig('default_key') end

  -- Firma del file .sig con ssh-keygen -Y sign -n file
  if sign and keyPath <> nil
    os.Exec('cmd /C ssh-keygen -Y sign -n file -f "'
      + keyPath + '" "' + sigPath + '"')
    -- Accodamento della firma al file .sig
    appendAndDelete(sigPath + '.sig', sigPath)
  end
end
```

SaveAndSignCommitSignature

La chiave privata viene cercata prima per nome nel registro delle identità (che consente a sistemi multi-utente di avere ciascuno la propria chiave), poi come chiave di default nella configurazione utente impostabile con:

```
> rvc config -author=Michele -private_key=C:UsersMichele.sshid_ed25519
default_author : Michele
default_key : C:UsersMichele.sshid_ed25519
```

Le tre funzioni che gestiscono la configurazione locale sono:

```
func str ReadConfig(str key)
proc WriteConfig(str key, str value)
proc EnsureConfig()
```

Gestione configurazione locale

6.4.3 Ancoraggio a `__rvc__root` (RS07)

Anche con la firma obbligatoria, un verificatore che riceve una *repository* in un momento successivo non dispone di informazioni sullo stato di `__rvc__root` — e quindi sulle autorizzazioni vigenti — al momento di ogni commit. Senza questa informazione non è possibile distinguere un commit prodotto prima di una revoca da uno prodotto dopo.

La soluzione è il campo `rvcRootId` in `CommitValidation`: al momento di ogni commit il motore recupera l'ID dell'ultimo commit di `__rvc__root` e lo scrive nel `.sig`. Questo crea un legame crittografico tra ogni commit e lo stato delle autorizzazioni vigente in quel momento, rendendo la verifica storica accurata anche dopo cambi di personale o revoche di chiave.

6.5 Gestione delle identità (RS08-RS11)

Requisito	Descrizione
RS08	Gerarchia a tre livelli: amministratore, responsabile, dipendente
RS09	Permessi configurabili per progetto tramite file versionato
RS10	Revoca efficace dal commit successivo alla modifica
RS11	Successione responsabile gestita solo dall'amministratore

Requisiti di gestione identità

RS08 è stato soddisfatto implementando la gerarchia tramite `__rvc__root` — che contiene `allowed_Dipendenti` per l'amministratore e `allowed_Responsabili` per i responsabili — con doppia verifica per qualsiasi commit che modifica file speciali. RS09 è stato soddisfatto versionando `allowed_Dipendenti` dentro ogni commit ZIP: la storia delle autorizzazioni è parte integrante della catena crittografica. RS10 è soddisfatto strutturalmente: la revoca è efficace dal commit successivo alla rimozione della chiave, mentre i commit storici del dipendente revocato restano validi perché il campo `allowedSigners` del loro `.sig` riflette lo stato vigente al momento della firma. RS11 è stato soddisfatto richiedendo la firma con chiave master per qualsiasi commit su `__rvc__root`.

6.5.1 Il file `allowed_Dipendenti` e il controllo preventivo (RS08, RS09)

Il file `allowed_Dipendenti` è il registro delle identità autorizzate a fare commit su un progetto. È un file di testo nel formato OpenSSH `allowed_signers`, dove ogni riga associa un nome identificativo (il *principal*) alla corrispondente chiave pubblica:

```
Michele ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIGieAS3G0opZtqDPs3...
Luigi   ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIF0Upazh3bucxj4lf6...
```

Formato `allowed_Dipendenti`

Il *principal* deve corrispondere al nome autore usato al momento del commit: è il meccanismo con cui `ssh-keygen -Y verify -I <autore>` seleziona quale riga confrontare con la firma. Il file è *versionato*: viene archiviato nello ZIP di ogni commit e la sua modifica richiede un commit firmato dal responsabile di progetto.

La gerarchia a tre livelli è implementata attraverso due file in `_rvc_root`: `allowed_Dipendenti` (chiave amministratore) e `allowed_Responsabili` (chiavi responsabili). Ogni progetto ha il proprio `allowed_Dipendenti` locale. Un commit che modifica file speciali è autorizzato solo se il firmatario è presente sia in `allowed_Responsabili` di `_rvc_root` che in `allowed_Dipendenti` del progetto: questa doppia verifica impedisce l'auto-promozione.

Il controllo preventivo è implementato nella funzione `CheckValidity`, invocata prima di ogni commit su progetti con livello ≥ 2 :

```
func bool CheckValidity(str path, str project, str author,
                        str masterKey, str branch)
-- 1. Legge security_level da .rvc_policy risalendo la storia dei commit
secLevel := getSecLevelFromHistory(project)

-- 2. Per livello >= 2 verifica autore in allowed_Dipendenti
if secLevel >= 2 and masterKey = nil
    pubKey := getAuthorPubKeyContent(author)
    if !Init.isKeyInFile(pubKey, getFileFromHistory(project,
                                                    'allowed_Dipendenti'))
        errmsg := 'Autore "' + author + '" non autorizzato.'
        result := false
    end
end

-- 3. Rileva modifiche a file speciali (allowed_Dipendenti, .rvc_policy)
if specialFileChanged and masterKey = nil
    if !checkResponsabile(author, project) and !checkAdmin(author)
        errmsg := 'Modifica a file speciali richiede Admin o Responsabile.'
        result := false
    end
end
end
```

CheckValidity — controllo autore e file speciali

Se `CheckValidity` restituisce `false`, il commit non viene prodotto e non rimangono file parziali nella *repository*. Il tentativo di commit da parte di un autore non autorizzato produce:

```
> rvc commit -author=Intruso -note=«modifica non autorizzata»
-dir=C:stagetest_work -repo=C:stagerepo
```

```
ERROR:Autore «Intruso» non autorizzato su «TestProj» branch «principale»  
(chiave non in allowed_Dipendenti).
```

Un bug critico scoperto durante lo scenario S5 riguardava questa funzione: il codice originale cercava `allowed_Dipendenti` solo nell'ultimo ZIP con `readFileFromCommit`. Poiché i commit successivi al primo sono commit *delta* — contengono nello ZIP solo i file modificati — se `allowed_Dipendenti` non era stato modificato nell'ultimo commit il risultato era `nil`, e il confronto `nil <> contenuto_su_disco` risultava sempre `true`, causando il rifiuto sistematico di qualsiasi commit successivo al primo. La correzione ha sostituito `readFileFromCommit` con `getFileFromHistory`, che risale la catena fino al commit che contiene il file:

```
-- Prima: cercava il file solo nell'ultimo commit delta  
var str dipContent := Init.readFileFromCommit(project,  
                                              'allowed_Dipendenti',  
                                              repos)  
  
-- Dopo: risale la catena fino al commit che contiene il file  
var str dipContent := getFileFromHistory(project, 'allowed_Dipendenti')
```

Correzione ricerca file speciali

6.5.2 Protezione dei file speciali (RS08)

I file `.rvc_policy`, `allowed_Dipendenti` e `allowed_Responsabili` sono *file speciali* la cui modifica altera le regole di sicurezza del progetto. Il tentativo di modifica di `allowed_Dipendenti` da parte di un dipendente non autorizzato produce:

```
> rvc commit -author=Dipendente -note=«aggiorno i permessi»  
-dir=C:stagetest_work -repo=C:stagerepo  
ERROR:Commit rifiutato: Modifica a file speciali richiede Admin (in _rvc_root/  
allowed_Dipendenti) o Responsabile (in allowed_Responsabili E nel progetto  
allowed_Dipendenti). Autore «Dipendente» non autorizzato.
```

Il progetto `_rvc_root` è protetto ulteriormente: il suo livello di sicurezza è forzato a 3 indipendentemente dal file `.rvc_policy` e qualsiasi commit deve essere firmato con la chiave master. Per i commit legittimi su `_rvc_root` — ad esempio aggiornamento di `allowed_Responsabili` per un cambio di personale — si usa il parametro `-master-key=`:

```
rvc commit -master-key=C:\chiavi\master_key -dir=C:\stage\_rvc_root -  
repo=C:\stage\repo
```

Commit su _rvc_root con chiave master

6.5.3 Revoca e successione (RS10, RS11)

La revoca di un'identità avviene tramite un commit amministrativo che rimuove la riga corrispondente da `allowed_Dipendenti`. A partire dal commit successivo, la chiave rimossa non è più autorizzata. I commit prodotti prima della revoca continuano a risultare validi perché il campo `allowedSigners` del loro `.sig` registra lo stato delle chiavi vigente al momento della firma.

Un commit firmato con una chiave appena rimossa da `allowed_Dipendenti` viene rifiutato preventivamente da `CheckValidity` prima ancora di produrre il file ZIP:

```
> rvc commit -author=ExDipendente -note=«ultima modifica»  
-dir=C:stagework -repo=C:stagerepo  
ERROR:Autore «ExDipendente» non autorizzato: chiave non presente in  
allowed_Dipendenti.  
Commit annullato.
```

I commit prodotti prima della revoca continuano invece a risultare validi in `VerifyIntegrity`: il campo `allowedSigners` nel `.sig` di ciascuno registra lo stato delle chiavi vigente al momento della firma, non quello attuale. La revoca non è retroattiva — è efficace dal commit successivo all'aggiornamento di `allowed_Dipendenti`, come richiesto da RS10.

La successione di un responsabile segue lo stesso meccanismo, con il vincolo aggiuntivo che la modifica di `allowed_Responsabili` in `_rvc_root` deve essere firmata con la chiave master. Il flusso prevede: checkout di `_rvc_root`, modifica di `allowed_Responsabili`, commit con chiave master. Il nuovo responsabile diventa operativo a partire dal commit successivo.

6.6 Sicurezza configurabile (RS12-RS13)

Requisito	Descrizione
RS12	Livelli di sicurezza configurabili per progetto, non abbassabili
RS13	Cifratura dei contenuti con <i>AGE_G</i> per progetti riservati

Requisiti di sicurezza configurabile

RS12 è stato soddisfatto implementando il file `.rvc_policy` versionato con controllo di monotonicità in `CheckValidity`. RS13 è stato soddisfatto integrando *age* nel flusso di commit per il livello 4, sfruttando direttamente le chiavi SSH Ed25519 senza conversioni di formato.

6.6.1 Il file `.rvc_policy` e i livelli di sicurezza (RS12)

Il livello di sicurezza è definito nel file `.rvc_policy`, archiviato come file tracciato nella directory di lavoro e incluso in ogni commit ZIP. Per un progetto al livello 4 il contenuto è:

```
security_level=4
recipients=Michele ssh-ed25519 AAAA...;Luigi ssh-ed25519 AAAA...
```

Contenuto di `.rvc_policy` per livello 4

Livello	Descrizione	Firma	Cifratura
0	Accesso anonimo — nessun controllo di identità	no	no
1	Firma SSH registrata	sì	no
2	Firma verificata — autore deve essere in <code>allowed_Dipendenti</code>	sì	no
3	Integrità pre-commit — catena verificata prima di ogni commit	sì	no
4	Cifratura ZIP con <i>AGE_G</i> per i soli destinatari autorizzati	sì	sì

Livelli di sicurezza

I livelli sono cumulativi. La creazione di un nuovo progetto specifica il livello desiderato:

```
> rvc new-project -name=TestProj2 -level=2
-signers=allowed.txt -author=Michele -repo=C:stagerepo
packing TestProj2.0Q7BK1NYLS.{Michele}.zip
copying TestProj2.0Q7BK1NYLS.0Q7BK1NYLS_9B5749ED.sig to C:stagerepo
...
Progetto TestProj2 creato a livello di sicurezza 2.
```

La monotonicità del livello è implementata in `CheckValidity`: prima di accettare un commit che modifica `.rvc_policy`, il motore confronta il nuovo `security_level` con quello del commit precedente e rifiuta il commit se il valore è inferiore:

```
> rvc commit -author=Michele -note=«porto a livello 0»
-dir=C:stagetest_work -repo=C:stagerepo
ERROR:Impossibile abbassare il livello di sicurezza da 2 a 0
(RS12: livello non riducibile).
```

Questa scelta progettuale previene attacchi che tentano di degradare temporaneamente le garanzie di sicurezza per inserire commit senza firma.

6.6.2 Integrazione di *AGE* per il livello 4 (RS13)

Al livello 4, il file ZIP del commit viene cifrato con *AGE* prima di essere archiviato nella *repository*. Solo i destinatari le cui chiavi pubbliche sono elencate nel campo `recipients` del `.rvc_policy` possono decifrarlo. La lista è scritta anche in `cv.recipients` nel `.sig`, consentendo al verificatore di controllare i destinatari autorizzati senza aprire lo ZIP.

age accetta direttamente chiavi Ed25519 in formato OpenSSH — le stesse usate per la firma — senza bisogno di un keyring separato o di conversioni di formato. Prima di passare le chiavi ad *age*, viene estratta la parte `<keytype> <base64data>` escludendo il *principal*, che *age* non accetterebbe:

```
-- Costruisce il comando age con un parametro -r per ogni destinatario.
-- I destinatari sono separati da ';' nel campo recipients del .rvc_policy.
ageCmd := '' + ageExe + ' --encrypt'
for each rcpKey in split(recipients, ';')
  -- Estrae la parte chiave
  ageRcpKey := Substr(rcpKey, firstOf(rcpKey, 'ssh-', 'ecdsa-', 'sk-'))
  -- Guard: un carattere '"' nel path spezzerebbe le
  -- virgolette del comando.
  if At('"', ageRcpKey) = 0
    ageCmd := ageCmd + ' -r "' + ageRcpKey + '"'
  end
end
os.Exec(ageCmd + ' -o "' + encPath + '" "' + zipPath + '"')
```

Cifratura age per livello 4

Le chiavi di tutti gli amministratori vengono aggiunte automaticamente ai destinatari alla creazione del progetto, garantendo che un progetto non possa diventare illeggibile all'amministratore:

```
> rvc new-project -name=SecureProj -level=4 -author=Michele -repo=C:stagerepo
[INFO] level 4: chiave di «Michele» aggiunta automaticamente a recipients.
packing SecureProj.0Q7BKD2LIZ.{Michele}.zip
copying SecureProj.0Q7BKD2LIZ_45083758.sig to C:stagerepo
...
```

Il checkout richiede la chiave privata SSH dell'utente (la stessa usata per la firma):

```
> rvc checkout -project=SecureProj
-age-key=C:UsersMichele.sshid_ed25519
-dir=C:stagesecure_work -repo=C:stagerepo
checkout project: SecureProj version:0Q7BKD2LIZ_45083758
Checkout version 0Q7BKD2LIZ completato.
```

Se la chiave non corrisponde a nessun destinatario autorizzato:

```
> rvc checkout -project=SecureProj
-age-key=C:UsersIntruso.sshid_ed25519
-dir=C:stagetest -repo=C:stagerepo
ERROR:Chiave non autorizzata o errata per il progetto level 4 «SecureProj».
```

Il verificatore `rvc integrity` può operare su progetti al livello 4 senza decifrazione: l'hash nel .sig è calcolato sul file ZIP cifrato e la firma copre tutti i metadati del commit:

```
> rvc integrity -project=SecureProj -repo=C:stagerepo
Progetto: SecureProj [level=4]
[OK] 0Q7BKD2LIZ_45083758 2026-05-19 07:48 hash:OK catena:OK firma:OK [cifrato]
```

(Michele)

Risultato: 0 commit con problemi.

6.7 Gestione dei branch (RS14-RS15)

Requisito	Descrizione
RS14	Branch compromessi chiudibili con commit firmato da Responsabile
RS15	Autorizzazioni differenziate per branch tramite <code>allowed_Dipendenti</code>

Requisiti di gestione branch

RS14 è stato soddisfatto tramite il campo `branchStatus` nel `.sig` e il file `.rvc_branch_status` nella directory di lavoro: un commit firmato da un Responsabile può marcare il branch come `archived` o `compromised`, impedendo qualsiasi commit ordinario successivo. RS15 è stato soddisfatto estendendo `CheckValidity` al confronto del file `allowed_Dipendenti` specifico per ogni branch, distinto da quello globale del progetto.

6.7.1 Controllo dello stato del branch in `CheckValidity`

Il campo `branchStatus` viene letto direttamente dal `.sig` dell'ultimo commit del branch. `CheckValidity` controlla questo valore prima di qualsiasi altra verifica: se il branch risulta `archived` o `compromised`, il commit viene bloccato immediatamente, indipendentemente dall'identità del firmatario e dal livello di sicurezza del progetto.


```

func bool CheckValidity(str path, str project,
                        str author, str masterKey:=nil, str branch:=nil)
-- ...
-- Recupera l'ultimo commit del branch corrente e ne legge il .sig.
var str lastId := RvcEngine.getLastCommit(project, branch, nil, repos,
rp)
var str lastSig := getSignatureFileNameForExtraction(project, lastId,
repos)
var CommitValidation lastCv := ReadCommitSignature(lastSig)

-- Blocca commit ordinari su branch non attivi.
-- Solo un commit firmato con chiave master puo' riaprire
-- un branch compromised (per documentare la chiusura dell'incidente).
if lastCv.branchStatus = 'archived'
    rp.Error('Branch "' + branch + '" archiviato: nessun commit
consentito.')
    result := false
    return
end
if lastCv.branchStatus = 'compromised'
    rp.Error('Branch "' + branch + '" compromesso: solo la chiave master '
+ 'puo'' produrre commit su questo branch.')
    if masterKey = nil
        result := false
        return
    end
end
end
-- ...

```

Controllo branchStatus in CheckValidity

Un tentativo di commit ordinario su un branch archiviato o compromesso produce un errore esplicito prima ancora che il file ZIP venga creato:

```

> rvc commit -author=Michele -note=«fix»
-dir=C:stagework_feature -repo=C:stagerepo
ERROR:Branch «feature-xyz» archiviato: nessun commit consentito.
Commit annullato.

```

```

> rvc commit -author=Michele -note=«hotfix urgente»
-dir=C:stagework_main -repo=C:stagerepo
ERROR:Branch «main» compromesso: solo la chiave master puo' produrre commit su
questo branch.
Commit annullato.

```

6.7.2 Autorizzazioni differenziate per branch (RS15)

Per i branch con lista ristretta di autorizzati, CheckValidity cerca il file `allowed_Dipendenti` specifico del branch — archiviato con nome `allowed_Dipendenti.<branchName>` — prima di ricorrere a quello globale del progetto.

Se esiste una lista branch-specifica, solo gli autori in quella lista possono fare commit su quel branch, anche se sono presenti nel file globale.

Un dipendente autorizzato sul progetto ma escluso dalla lista ristretta del branch ottiene:

```
> rvc commit -author=Ospite -note=«contributo»  
-dir=C:stagework_main -repo=C:stagerepo  
ERROR:Autore «Ospite» non autorizzato sul branch «main».  
(allowed_Dipendenti.main non contiene questa identita')  
Commit annullato.
```

6.7.3 Protezione dei file speciali durante il merge

Il campo mergeFrom nel .sig registra il nome del branch sorgente nei commit di merge, consentendo al verificatore di controllare che il merge fosse autorizzato. La situazione più delicata riguarda però il contenuto del merge stesso: un utente che controlla il branch sorgente potrebbe costruire un commit che, integrato nel branch di destinazione, sovrascrive allowed_Dipendenti o .rvc_policy con valori a lui favorevoli — un'escalation di permessi mascherata da operazione di merge.

Il motore risolve questo con un'eccezione esplicita nella logica di checkout in modalità merge: i file speciali (allowed_Dipendenti, .rvc_policy, allowed_Responsabili) del branch di destinazione vengono *sempre preservati*, anche se il branch sorgente li ha modificati. Le modifiche ai file speciali nel sorgente vengono scartate silenziosamente durante il merge e devono essere applicate separatamente con un commit amministrativo esplicito, firmato dal Responsabile del branch di destinazione. Questo garantisce che nessun merge possa elevare i propri permessi al di sopra di quelli del branch che lo accoglie.

6.8 Verificatore di integrità della catena (RS02, RS02, RS06)

Il verificatore rvc integrity è il controllo post-hoc che completa il sistema: mentre Check-Validity agisce prima del commit come guardia preventiva, `VerifyIntegrity` può essere eseguito in qualsiasi momento — anche su una *repository* ricevuta da un canale non fidato — per accertare l'integrità della catena dall'inizio alla fine.

Per ogni commit nella catena, il verificatore controlla quattro proprietà in cascata:

1. **Hash ZIP**: ricalcola lo SHA256 del file ZIP e lo confronta con cv.hash.
2. **Hash catena**: verifica che cv.prevHash coincida con cv.hash del commit precedente.
3. **Hash cumulativo**: ricalcola `SHA256(cv.hash + prevCumulativeHash)` e lo confronta con cv.cumulativeHash.

4. **Firma SSH:** invoca `ssh-keygen -Y verify -n file` usando `cv.allowedSigners` estratto dal `.sig`, verificando che l'autore fosse autorizzato secondo le chiavi vigenti al momento del commit.

```
func bool VerifyIntegrity(str project, ...)
    hst := getCommitsHistory(project) -- ordine dal più vecchio al più
    recente
    for each ci in hst
        cv := ReadCommitSignature(sigPathFor(ci.id))
        hashOk := crypt.SHA256(zipPathFor(ci.id)) = cv.hash
        catenaOk := cv.prevHash = prevHash
        cumHashOk := crypt.SHA256(cv.hash + prevCumHash) = cv.cumulativeHash
        firmaOk := VerifySignature(sigPath, cv.allowedSigners)
        -- Continua anche in caso di errore: in contesto forense è necessario
        -- conoscere l'estensione completa della manomissione.
        prevCumHash := cv.cumulativeHash
        rp.Report(formatRow(ci.id, hashOk, catenaOk, cumHashOk,
                           firmaOk, cv.author))
    end
end
```

VerifyIntegrity — verifica per commit

La scelta di continuare la verifica anche dopo il primo errore è deliberata: in un contesto forense è necessario conoscere l'estensione completa della manomissione, non solo il punto di ingresso. L'output su una *repository* integra:

```
> rvc integrity -project=TestProj -repo=C:stagerepo
analyzing repository C:stagerepo
...
Progetto: TestProj [level=0]
[OK] 0Q7BK1L0ZY_E168FAB3 2026-05-19 07:41 hash:OK catena:OK firma:OK autorita:OK
(Michele)
[OK] 0Q7BK6WQRB_D7000B46 2026-05-19 07:44 hash:OK catena:OK firma:OK (Michele)
[OK] 0Q7BK7VQZR_F6794C35 2026-05-19 07:45 hash:OK catena:OK firma:OK (Michele)
Risultato: 0 commit con problemi.
Risultato: 0 commit con warning.
```

6.9 Redazione trasparente

La Redazione Trasparente (F01) consente di rimuovere dalla storia di una *repository* il contenuto di un commit sensibile — ad esempio dati personali caricati per errore in violazione del GDPR o credenziali accidentalmente committate — senza interrompere la catena crittografica.

Il problema è strutturalmente vincolato: la catena di `cumulativeHash` rende impossibile modificare un commit senza invalidare tutti i successivi. Qualsiasi sostituzione del contenuto richiede quindi il ricalcolo dell'intera catena a partire dal commit redatto. `rvc redact` non simula l'eliminazione del commit, ma lo sostituisce con un commit che dichiara esplicitamente la propria natura, firmato dalla chiave master.

6.9.1 Il meccanismo di sostituzione

L'esecuzione di `rvc redact` prevede i seguenti passi:

1. Lettura del `.sig` del commit da redarre
2. Costruzione di un nuovo ZIP contenente solo `REDACTION_NOTICE.json`, con riferimento legale, timestamp e autore originale
3. Aggiornamento di `cv.hash` sul nuovo ZIP; il vecchio hash è registrato in `cv.redactionZipHash`
4. Conservazione della firma originale del dipendente in `cv.redactionOriginalSig` come evidenza forense
5. Ricalcolo di `cumulativeHash` per il commit redatto e per tutti i commit successivi
6. Firma della `CommitValidation` aggiornata con la chiave master, producendo `cv.redactionSignature`

```
func bool Redact(str project, str commitId, str masterKeyPath, str legalRef, ...)
    cv := ReadCommitSignature(sigPathFor(commitId))
    -- Sostituisce lo ZIP con un archivio contenente solo
    REDACTION_NOTICE.json
    newZipPath := buildRedactionZip(legalRef, cv.author, commitId)
    -- Aggiorna i campi di redazione nel .sig
    cv.redactionZipHash      := crypt.SHA256(newZipPath)
    cv.redactionAuthority    := crypt.SHA256File(masterKeyPath + '.pub')
    cv.redactionTimestamp    := RvcEngine.NewUID(6)
    cv.redactionLegalRef     := legalRef
    cv.redactionCount        := Str(IntVal(cv.redactionCount) + 1)
    cv.redactionOriginalSig  := extractSignatureBlock(sigPath)
    cv.redacted              := true
    recomputeCumulativeHashChain(project, commitId, repos)
    Init.signWithKey(sigPath, masterKeyPath)
end
```

Redact — sostituzione e ricalcolo catena

Il `redactionCount` permette di tracciare eventuali ri-redazioni successive dello stesso commit; ogni ri-redazione incrementa il contatore e produce una nuova `redactionSignature`.

6.9.2 Varianti del comando

Il comando `rvc redact` supporta tre modalità. La prima opera su un singolo commit:

```
> rvc redact -project=mioprogetto
-id=0Q7BK6WQRB_D7000B46
-master-key=C:chiavimaster_key
-legal-ref=«GDPR Art.17 - Diritto all'oblio»
-repo=C:stagerepo
Redazione singola: commit 0Q7BK6WQRB_D7000B46 redatto.
Catena aggiornata: N commit successivi ricalcolati.
Firma master applicata.
```

La seconda opera su un intervallo consecutivo di commit, specificato con due ID separati da `..`:

```
> rvc redact -project=mioprogetto
-range=0Q7BK6WQRB_D7000B46..0Q7BK7VQZR_F6794C35
-master-key=C:chiavimaster_key
-repo=C:stagerepo
Redazione range: 3 commit redatti.
Catena aggiornata: M commit successivi ricalcolati.
```

La terza redige tutti i commit di un branch, marcandolo automaticamente come compromised:

```
> rvc redact -project=mioprogetto
-branch=feature-xyz
-master-key=C:chiavimaster_key
-repo=C:stagerepo
Redazione branch: tutti i commit del branch feature-xyz redatti.
Branch marcato come compromised.
```

6.9.3 Proprietà garantite

La redazione trasparente soddisfa quattro proprietà simultaneamente:

- **Contenuto rimosso:** il file ZIP originale è fisicamente sostituito con un ZIP contenente solo REDACTION_NOTICE.json; il contenuto sensibile non è recuperabile dalla *repository*
- **Catena intatta:** il cumulativeHash è ricalcolato sull'intera catena successiva; un verificatore esterno ottiene risultato «integro» dopo la redazione
- **Trasparenza:** il .sig dichiara esplicitamente la natura del commit redatto, con riferimento legale e timestamp; il verificatore lo mostra come [REDACTED] con indicazione della chiave master autorizzante

- **Evidenza forense:** la firma originale del dipendente (redactionOriginalSig) e la firma master (redactionSignature) coesistono nel .sig, permettendo di ricostruire sia la paternità del contenuto originale sia l'autorizzazione alla sua rimozione

6.10 Robustezza e fix injection (scenario S5)

Lo scenario S5 ha testato la resistenza del motore a input malevoli, crash da parametri mancanti e tentativi di injection tramite campi stringa. Sono stati identificati diciassette problemi distinti, raggruppati in tre categorie: vulnerabilità di injection, crash da assenza di guard e collisioni di naming.

6.10.1 Validazione degli input

Prima degli interventi, i parametri ricevuti dalla riga di comando venivano utilizzati direttamente nelle operazioni interne senza validazione preventiva. Il principio adottato per la correzione è la *validazione al confine*: ogni valore proveniente dall'esterno viene validato all'ingresso nel motore, prima di qualsiasi utilizzo.

Sono state introdotte quattro funzioni di validazione nel modulo RvcEngine.cpl:

```
func bool isValidProjectName(str project)
-- Pattern [0-9A-Za-z_]+, max 64 caratteri.
-- Esclude metacaratteri shell e limita la lunghezza del path ZIP entro
MAX_PATH.

func bool isValidAuthorName(str author)
-- Pattern [0-9A-Za-z_\.@]+, max 64 caratteri.
-- Permette formato email (mario.rossi@az.it); esclude &, |, ;, >, <, ",
spazio.

func bool isValidBranchName(str branch)
-- Pattern [-]?[0-9A-Za-z_\.]+, max 64 caratteri.
-- Il trattino iniziale opzionale identifica branch chiusi per
convenzione.

func bool isValidTagName(str tag)
-- Pattern ([0-9A-Za-z_\.]+)((\+[0-9A-Za-z_\.]+)*), max 128 caratteri.
-- Il '+' separa tag multipli (es. 'release+stable').
```

Funzioni di validazione input

I limiti di lunghezza non sono arbitrari: Windows impone un limite di 260 caratteri per i path dei file (MAX_PATH). Il nome del file ZIP include nome progetto, due ID commit (18 caratteri ciascuno), nome autore ed estensione; un nome di progetto di 300 caratteri porterebbe sistematicamente il path oltre tale limite. Limitando progetto e autore a 64 caratteri si garantisce che il path risultante non superi mai il limite di sistema.

6.10.2 Fix injection sul campo author

La vulnerabilità più critica riguardava il campo `cv.author` del file `.sig`, inserito senza quotatura nella costruzione del comando di verifica della firma. Un `.sig` manomesso con un campo `author` contenente caratteri come `"` avrebbe potuto iniettare comandi arbitrari nel processo di verifica tramite `cmd.exe`:

```
-- Prima (vulnerabile): cv.author non quotato nel cmd /c
-- Se cv.author = 'Michele" & calc.exe & echo "' → cmd.exe esegue
calc.exe
os.Exec('cmd /c ssh-keygen -Y verify -n file -f ' + kn
      + ' -I ' + cv.author + ' -s ' + sn + ' < ' + tn)
```

Vulnerabilità CWE-78 su `author`

La correzione agisce su due livelli indipendenti:

```
-- Livello 1: validazione semantica – ammette solo [0-9A-Za-z_\-.@]+ (max
64 char).
-- Un .sig manomesso con author invalido causa firma:FAIL senza eseguire
codice.
if !RvcEngine.IsValidAuthorName(cv.author)
    result := false
else
    -- Livello 2: quotatura del valore anche se supera la validazione.
    -- Difesa in profondità contro errori futuri nella regex.
    os.Exec('cmd /c ssh-keygen -Y verify -n file -f ' + kn
          + ' -I "' + cv.author + '" -s ' + sn + ' < ' + tn)
end
```

Correzione CWE-78 — doppia difesa

I due livelli si difendono da classi di attacchi distinte: la validazione blocca payload con caratteri semanticamente invalidi (e.g. `"`, `&`, `|`, `;`); la quotatura protegge i valori che passano la validazione da interpretazioni shell inattese.

Fix analoghi sono stati applicati ai path di configurazione per prevenire injection da file di configurazione manomessi:

```

-- Guard su keyPath in signWithKey: un '"' spezzerebbe le virgolette in cpl
cmd /C
proc signWithKey(str sigPath, str keyPath)
  if At('"' , keyPath) > 0
    errormsg := 'Il path della chiave non puo'' contenere il carattere ": ' + keyPath
  else
    os.Exec('cmd /C ssh-keygen -Y sign -n file -f "' + keyPath + '" "' + sigPath + '"')
  end
end

-- Guard su rvcHome in syncRvcEnvVars: un '"' spezzerebbe il file PS1
generato
proc syncRvcEnvVars(str rvcHome)
  if At('"' , rvcHome) > 0
    rp.Error('Il path rvc_home non puo'' contenere il carattere ": ' + rvcHome)
  else
    -- genera ed esegue lo script PS1 con $h = "rvcHome"
  end
end

```

Guard injection su path di configurazione

6.10.3 Fix crash da parametri mancanti

Lo scenario T5 aveva identificato quattordici situazioni in cui il motore produceva un traceback CPL interno anziché un messaggio leggibile dall'utente. I crash derivavano da tre pattern ricorrenti:

Causa	Correzione
<code>ArchiverByExt(nil)</code> — path ZIP nil quando versione non trovata	Guard <code>if pkn <> nil</code> in <code>getManifest</code> con messaggio esplicito
Doppia stampa errore: <code>rp.Error()</code> sia in <code>InitRepo</code> sia nell'outer handler	Rimossi tutti i <code>rp.Error()</code> interni; solo <code>errormsg</code> impostato
<code>errormsg</code> sovrascritta con <code>nil</code> da <code>errormsg := fmc.errormsg</code>	Guard <code>if fmc.errormsg <> nil</code> prima della sovrascrittura
Flag <code>-vers</code> interpretato come <code>-ver=s</code>	Rinominato in <code>-allver</code>
Comandi eseguiti senza parametri obbligatori	Guard all'ingresso con messaggi di errore espliciti

Pattern di crash e correzioni

Il pattern di correzione verifica il parametro obbligatorio all'ingresso della funzione, prima di qualsiasi operazione su disco:

```
func bool NewProject(str projectName, int level, ...)
    result := false
    if projectName = nil or projectName.LRtrim() = ''
        errormsg := 'Parametro -name= obbligatorio.'
    elseif level < 0 or level > 4
        errormsg := 'Livello non valido: ' + Str(level)
        + '. Valori ammessi: 0-4.'
    elseif projectName = '_rvc_root'
        errormsg := 'Nome "_rvc_root" riservato al sistema.'
    elseif getLastCommit('_rvc_root', ...) = nil
        errormsg := '_rvc_root non trovato. Eseguire "rvc init" prima.'
    else
        -- procedura di creazione del progetto
    end
end
```

Guard parametri obbligatori — NewProject

L'output risultante è un messaggio leggibile al posto di un traceback interno:

```
> rvc new-project -level=9 -name=TestProj -repo=C:stagerepo
ERROR:Livello di sicurezza non valido: 9. Valori ammessi: 0, 1, 2, 3, 4.
```

```
> rvc info -ver=NONEXISTENT -repo=C:stagerepo -dir=C:stagetest_work
ERROR:Versione non trovata nel repository: NONEXISTENT
```

La rinomina del flag `-vers` in `-allver` merita una nota: il parser CLI di *RVC_G* usa il prefisso come disambiguatore. Il flag `-vers` veniva interpretato come abbreviazione di `-ver` con valore `s`, producendo una ricerca della versione `"s"` nella *repository_G* e un crash per `ArchiverByExt(nil)`. La rinomina in `-allver` elimina l'ambiguità senza alterare la funzionalità.

6.10.4 Sintesi del soddisfacimento dei requisiti

A conclusione degli interventi di miglioramento descritti, è possibile tracciare un quadro riassuntivo dello stato dei requisiti formali definiti nella Sezione 4. L'implementazione e l'integrazione dei controlli preventivi nel motore di *RVC_G*, affiancate dallo sviluppo del verificatore post-hoc e del meccanismo di Redazione Trasparente, hanno permesso di colmare integralmente il divario riscontrato all'avvio dello stage. L'architettura proposta non si limita a fornire una protezione reattiva delegata a verifiche asincrone, ma converte i vincoli crittografici in barriere operative preventive, garantendo la conformità del sistema ai requisiti di sicurezza aziendali senza comprometterne la natura distribuita.

Capitolo 7

Simulazione scenari di attacco post-implementazione

In questo capitolo vengono ripetuti gli scenari di attacco del Sezione 5 sulla versione aggiornata di RVC_G, dopo l'implementazione dei miglioramenti descritti nel Sezione 6. Per ciascuno scenario vengono documentati i comandi eseguiti, l'output del motore e del verificatore, e il confronto con i risultati della versione iniziale, evidenziando quali tecniche sono ora bloccate preventivamente e quali rimangono parzialmente o interamente efficaci.

Capitolo 8

Conclusioni

In questo capitolo tratto le conclusioni sul progetto.

Glossario

AGE – Actually Good Encryption: Strumento moderno per la cifratura di file. Utilizza algoritmi crittografici contemporanei come X25519 e ChaCha20-Poly1305, con un'interfaccia volutamente semplice. [v](#), [3](#), [4](#), [8](#), [9](#), [11](#), [14](#), [15](#), [26](#), [31](#), [34](#), [36](#), [40](#), [41](#), [42](#), [43](#), [53](#), [59](#), [60](#), [97](#), [98](#)

Air-gapped: Isolamento fisico di un dispositivo da qualsiasi rete. Utilizzato per conservare chiavi crittografiche ad alta sensibilità, come la chiave master di un sistema di versionamento: un dispositivo air-gapped non può essere compromesso da remoto. [29](#), [82](#), [91](#)

Base36: Sistema di numerazione in base 36 che utilizza le cifre 0–9 e le lettere A–Z. In RVC è usato per codificare i timestamp degli identificativi dei commit, garantendo che l'ordinamento lessicografico dei nomi file coincida con l'ordine cronologico. [17](#), [18](#), [23](#), [35](#), [56](#), [89](#)

Black-box: Approccio di analisi della sicurezza condotto senza accesso al codice sorgente, osservando esclusivamente il comportamento esterno del sistema in risposta agli input forniti. [5](#)

Branch: Linea di sviluppo parallela all'interno di un sistema di versionamento. Permette di lavorare su funzionalità o correzioni in modo isolato rispetto al ramo principale. [26](#), [27](#), [30](#), [34](#), [36](#), [47](#), [48](#), [49](#), [50](#), [53](#), [54](#), [59](#), [60](#), [66](#), [67](#), [68](#), [69](#)

CPL – CodePainter Language: Linguaggio di programmazione proprietario sviluppato da Zucchetti S.p.A. Interpretato, tipizzato staticamente, con sintassi simile al Pascal. Utilizzato per lo sviluppo di RVC e dei prodotti software Zucchetti. [v](#), [3](#), [5](#), [8](#), [9](#), [17](#), [19](#), [20](#), [87](#)

Ed25519: Algoritmo di firma digitale basato sulla curva ellittica Curve25519. Produce chiavi di 256 bit con un livello di sicurezza equivalente a RSA con chiavi da 3000 bit, risultando più efficiente e compatto. [v](#), [8](#), [13](#), [14](#)

Git: Sistema di versionamento distribuito open source, creato da Linus Torvalds nel 2005. Garantisce l'integrità della storia tramite una catena di hash crittografici. [v](#), [vi](#), [2](#), [16](#), [17](#), [22](#), [23](#)

Hash: Funzione matematica che trasforma un dato di dimensione arbitraria in una stringa di lunghezza fissa. Una minima modifica dell'input produce un output completamente

diverso, garantendo l'integrità dei dati. [v](#), [2](#), [9](#), [12](#), [17](#), [18](#), [21](#), [22](#), [24](#), [35](#), [37](#), [39](#), [42](#), [43](#), [51](#), [53](#), [56](#), [57](#), [59](#), [60](#), [65](#), [66](#), [68](#), [69](#), [70](#), [78](#), [83](#), [84](#), [86](#)

JIT – Just-In-Time: Tecnica di compilazione che traduce il codice intermedio in codice macchina durante l'esecuzione, migliorando le prestazioni rispetto alla sola interpretazione. [20](#)

Manifest: Nel contesto di RVC, file che descrive lo stato di tutti i file tracciati in un progetto in un dato momento: nomi, dimensioni, hash e versione di ogni file. [20](#)

Namespace: Nel contesto della firma SSH, stringa che identifica il contesto d'uso di una firma e previene attacchi di riutilizzo: una firma prodotta con un certo namespace non può essere usata in un contesto diverso. [14](#)

Passphrase: Sequenza di parole o caratteri utilizzata per proteggere una chiave privata. Più lunga e complessa di una password tradizionale, viene richiesta ogni volta che si utilizza la chiave. [14](#)

Repository: Archivio che contiene la storia completa delle modifiche di un progetto sotto controllo versione, inclusi tutti i commit, i branch e i tag. [v](#), [3](#), [8](#), [9](#), [16](#), [17](#), [20](#), [22](#), [24](#), [25](#), [29](#), [30](#), [31](#), [32](#), [33](#), [34](#), [35](#), [37](#), [38](#), [39](#), [43](#), [45](#), [47](#), [48](#), [51](#), [52](#), [53](#), [54](#), [55](#), [56](#), [61](#), [62](#), [64](#), [66](#), [67](#), [69](#), [70](#), [78](#), [80](#), [81](#), [82](#), [83](#), [84](#), [85](#), [86](#), [87](#), [90](#), [92](#), [94](#), [98](#), [102](#), [103](#), [105](#), [109](#)

RVC – Repositoryless Version Control: Sistema di versionamento distribuito sviluppato da Zucchetti S.p.A. come alternativa a Git. Garantisce l'integrità dei contenuti tramite verifica crittografica degli hash di ogni commit, permettendo a qualsiasi utente di accertare autonomamente l'autenticità della repository ricevuta. [v](#), [vi](#), [vii](#), [xi](#), [2](#), [3](#), [4](#), [5](#), [8](#), [9](#), [11](#), [14](#), [17](#), [18](#), [19](#), [20](#), [21](#), [23](#), [32](#), [33](#), [34](#), [36](#), [45](#), [46](#), [51](#), [55](#), [56](#), [59](#), [60](#), [61](#), [62](#), [74](#), [80](#), [83](#), [84](#), [87](#), [109](#), [111](#)

Social engineering: Tecnica di manipolazione psicologica utilizzata dagli attaccanti per indurre le vittime a divulgare informazioni riservate o a compiere azioni che compromettono la sicurezza, come cliccare su link malevoli o fornire credenziali. Spesso sfrutta la fiducia, l'urgenza o la curiosità delle persone. [38](#)

Spear phishing: Variante di phishing mirata a un individuo o a un gruppo specifico, in cui l'attaccante personalizza i messaggi per aumentare le probabilità di successo. Può essere utilizzato per ottenere credenziali di accesso o indurre la vittima a eseguire azioni dannose. [38](#)

SSH – Secure Shell: Protocollo crittografico per la comunicazione sicura su reti non affidabili. Utilizza crittografia asimmetrica per l'autenticazione e la cifratura del canale. [v](#), [vi](#), [xi](#), [2](#), [3](#), [4](#), [8](#), [9](#), [11](#), [13](#), [14](#), [17](#), [18](#), [19](#), [22](#), [23](#), [25](#), [35](#), [36](#), [39](#), [42](#), [44](#), [57](#), [60](#), [61](#), [62](#), [65](#), [66](#), [67](#), [69](#), [70](#), [71](#), [72](#), [74](#), [75](#), [80](#), [83](#), [84](#)

VCS – Version Control System: Sistema software che registra le modifiche ai file nel tempo, permettendo di recuperare versioni precedenti, confrontare cambiamenti e coordinare il lavoro di più sviluppatori. [15](#), [16](#)

White-box: Approccio di analisi della sicurezza con piena visibilità del codice sorgente e dell'architettura interna del sistema. Si contrappone all'analisi black-box, condotta senza accesso ai sorgenti. [5](#)

Bibliografia

- [1] S. Josefsson e I. Liusvaara, «Edwards-Curve Digital Signature Algorithm (EdDSA)». [Online]. Disponibile su: <https://datatracker.ietf.org/doc/html/rfc8032>
- [2] B. Harris e L. Velvindron, «Edwards-Curve Digital Signature Algorithm (EdDSA) for Secure Shell (SSH)». [Online]. Disponibile su: <https://datatracker.ietf.org/doc/html/rfc8709>
- [3] OpenBSD Project, «ssh-keygen: authentication key generation, management and conversion». [Online]. Disponibile su: <https://man.openbsd.org/ssh-keygen>
- [4] F. Valsorda, «age: simple, modern and secure file encryption». [Online]. Disponibile su: <https://age-encryption.org/v1>
- [5] L. Torvalds, «Git: Distributed Version Control System». [Online]. Disponibile su: <https://git-scm.com/>
- [6] S. Chacon e B. Straub, *Pro Git*, 2^o ed. Apress, 2014. [Online]. Disponibile su: <https://git-scm.com/book/en/v2>
- [7] MITRE Corporation, «MITRE ATT&CK: Adversarial Tactics, Techniques, and Common Knowledge». [Online]. Disponibile su: <https://attack.mitre.org/>
- [8] M. Haug e L. Mädje, «Typst: A new markup-based typesetting system». [Online]. Disponibile su: <https://typst.app/>